SUP'COM

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Diplôme d'Ingénieur en Télécommunications

Option :

Services Numériques & IA(SNI)

Rapport Final Du Projet De Fin d'Etudes

Thème :

Mise en place d'une plateforme Cloud PaaS OpenShift automatisant le cycle de vie des services avec CI/CD

Réalisé par :

Chaker Barkaoui

Superviseurs :

Mr. Seifallah Aouinti (Standard Sharing Software (3S))

Mr. Zied Choukair (Sup'Com)

Travail proposé et réalisé au sein :

Standard Sharing Software (3S)

Année académique : 2020-2021

مدينة تكنولوجيا المواصلات - طريق رواد كلم 3.5 - 2083 الغزالة أريانة - الهاتف : 000 857 71 216 + - الفاكس : 829 856 71 216 +
Cité Technologique des Communications - Route de Raoued Km 3,5 - 2083 El Ghazala Ariana
Tél : + 216 71 857 000 - Fax : + 216 71 856 829 - E-mail : contact@supcom.tn - Web : www.supcom.tn

Signatures des superviseurs

Superviseur Standard Sharing Software (3S) :
Mr. Seifallah Aouinti

Superviseur SUP'COM :
Mr. Zied Choukair

Remerciements

Je voudrais tout d'abord exprimer ma profonde gratitude à tous ceux qui m'ont offert la possibilité de mener à bien ce projet.

Une gratitude particulière que j'aime exprimer à mon superviseur Mr. Seifallah Aouinti, directeur du département Cloud et Automatisation de Standard Sharing Software (3S) pour les conseils et les encouragements qu'il m'a prodigués pendant mon stage.

Je tiens également à remercier Mme Imen Yaich, ingénieure cloud chez Standard Sharing Software (3S) pour son grand soutien. Elle écoute toujours attentivement mes questions et lorsque je rencontre un problème, elle m'a guidé pour le résoudre. Elle a toujours permis à ce projet d'être mon propre travail, mais m'a orientée dans la bonne direction chaque fois qu'elle pensait que j'en avais besoin. J'apprécie vraiment ses encouragements et elle m'aide à apprendre beaucoup de nouvelles choses.

Je suis très reconnaissant de faire mon stage dans cette entreprise et d'avoir M. Seifallah Aouinti et Mme Imen Yaich comme superviseurs. Sans leur participation passionnée et leur enquête cohérente, ce projet n'aurait pas pu être mené avec succès.

Un merci tout particulier à M. Zied Choukair, mon superviseur au SUP'COM pour son soutien constant, sa supervision et ses encouragements.

Je voudrais également exprimer ma plus profonde gratitude à tous mes professeurs pour leur formation pendant mes années d'études.

Enfin, je dois exprimer ma profonde gratitude à ma famille et à mes amis pour m'avoir apporté un soutien indéfectible et des encouragements continus tout au long de mes années d'études et pour avoir cru que je suis capable de le faire. Cet accomplissement n'aurait pas été possible sans eux.
Merci à tous



Résumé

L'approche CI/CD garantit une automatisation et un contrôle continue tout au long du cycle de vie des services.

Dans ce travail, j'essaye de déployer plusieurs catalogues de services en utilisant Openshift vue qu'il prend en charge des outils de développement bien connus et m'aide à rationaliser les flux de travail de construction et de livraison.

Dans ce travail je commence par la conception et l'implémentation d'une plateforme Cloud PaaS autour du Framework OpenShift. Puis j'arrive au développement de catalogue de services qui comprend des langages, des bases de données, des cadres de développement et des outils très variés. La partie suivante est l'automatisation des catalogues de services d'application multi-tiers. Enfin, l'administration d'un pipeline CI/CD avec les conteneurs.

Afin de déterminer si ces catalogues de services sont solides et performants, j'ai ajouté des outils d'analyse de code comme SonarQube, OpenShift Monitoring et Grafana pour contrôler les projets déployés.
Mots clés: Virtualisation, DevOps, Cloud Computing, PaaS, OpenShift, Ansible, Stockage, Containers, VMware.

Contents

Remerciements 3

Résumé 4

Contenu 5

Liste des figures 7

Liste des tables 9

Lexique 10

Introduction 11

1 Cadre général 13
1.1 Introduction . . . 13
1.2 Organisme d'accueil . . . 13
1.2.1 Activités et produits de 3S . . . 13
1.2.2 Département d'accueil . . . 14
1.3 Problématique et solution . . . 15
1.4 Objectifs et contributions . . . 15
1.5 Méthodologie de travail . . . 17
1.5.1 Gestion de projet : Scrum . . . 17
1.5.2 Méthodologie de développement : 2TUP . . . 19
1.6 Conclusion . . . 20

2 Etude préalable 21
2.1 Introduction . . . 21
2.2 Intégration continue . . . 21
2.3 Déploiement continu . . . 22
2.4 Éléments clés . . . 22
2.4.1 Docker . . . 22
2.4.2 Cluster . . . 22
2.4.3 Kubernetes . . . 23
2.4.4 Pod . . . 23
2.5 Étude fonctionnelle . . . 23
2.6 Étude comparative des solutions . . . 25
2.6.1 Etude de gestion de version : Git vs SVN . . . 25
2.6.2 Outil d'intégration continue : Jenkins vs Bamboo vs Travis . . . 27
2.6.3 Outil d'automatisation de configuration des ordinateurs : Chef vs Puppet vs Ansible . . . 27
2.6.4 Virtualisation : Machine virtuelle vs Docker . . . 28
2.6.5 Outil de gestion de dépôt : Nexus sonatype vs JFrog Artifactory . . . 29

2.6.6 Outil d'analyse de code : SonarQube vs Codacy . . . 30
2.6.7 Plateforme de déploiement : Openshift vs Cloud Foundry . . . 31
2.7 Synthèse et préconisation . . . 32
2.8 Conclusion . . . 32

3 Analyse et spécification **33**
3.1 Introduction . . . 33
3.2 Identification des acteurs . . . 33
3.3 Analyse des besoins . . . 34
3.3.1 Recensement des besoins fonctionnels . . . 34
3.3.2 Recensement des besoins techniques . . . 34
3.3.3 Besoins non fonctionnels . . . 35
3.4 Spécification UML – Diagramme des composants: . . . 35
3.5 Représentation des besoins "Diagrammes de cas d'utilisation"(DCU) : . . . 36
3.5.1 Intégration continue : . . . 39
3.5.2 Déploiement continu . . . 40
3.5.3 Pré-configuration . . . 41
3.5.4 Configuration . . . 42
3.5.5 Supervision . . . 43
3.6 Aspect dynamique de la solution : "Diagrammes d'activités" . . . 44
3.6.1 Processus d'Intégration continue . . . 44
3.6.2 Processus de déploiement d'une application . . . 45
3.7 Conclusion . . . 47

4 Réalisation **48**
4.1 Introduction . . . 48
4.2 Environnement de travail . . . 48
4.3 Implémentation: . . . 49
4.3.1 Pré-configuration : Mise en place d'un cluster OpenShift . . . 49
4.3.2 Configuration . . . 52
4.3.3 Pipeline d'intégration et de déploiement continu : . . . 54
4.4 Supervision : . . . 65
4.5 Intégration d'Openshift avec ManageIQ : . . . 66
4.6 Conclusion . . . 70

Conclusion générale **71**

Bibliography **73**

List of Figures

1.1 Objectifs et contributions : Project CI/CD Pipeline 16
1.2 Gestion de projet : Scrum 17
1.3 Méthodologie de développement : 2TUP 19

2.1 Docker 22
2.2 Kubernetes 23
2.3 Virtualisation : Machine virtuelle vs Docker 28

3.1 spécification UML :"Diagramme des composants du pipeline CI/CD" 35
3.2 Représentation des besoins : "Diagramme de cas d'utilisation général" . . . 37
3.3 Représentation des besoins : "diagramme de cas d'utilisation "Intégration continue" 39
3.4 Représentation des besoins : "Diagramme de cas d'utilisation "Déploiement continu" 40
3.5 Représentation des besoins : "Diagramme de cas d'utilisation "Pré-configuration". 41
3.6 Représentation des besoins : "Diagramme de cas d'utilisation "Configuration". 42
3.7 Représentation des besoins : "Diagramme de cas d'utilisation "Monitoring". 43
3.8 Vue dynamique du volet "Intégration continue" 44
3.9 Vue dynamique du volet "Déploiement continue" 46

4.1 "Mise en place d'un cluster OpenShift : Fichier inventory file" 49
4.2 "Mise en place d'un cluster OpenShift : Logs Ansible. Prerequisites" 50
4.3 "Mise en place d'un cluster OpenShift : "Logs Ansible. Deploy Cluster" . . 50
4.4 "Mise en place d'un cluster OpenShift : Openshift Console: Projets" 51
4.5 "Mise en place d'un cluster OpenShift : Etat du cluster Openshift" 51
4.6 "Configuration" : "Un modèle simple (YAML)" 52
4.7 "Configuration": "Openshift Console: Service Catalogue" 53
4.8 "Configuration": "Openshift CLI: Templates" 53
4.9 "Pipeline d'intégration et de déploiement continu : Création des volumes persistants" 55
4.10 "Pipeline d'intégration et de déploiement continu : Gogs Pods" 56
4.11 "Pipeline d'intégration et de déploiement continu : Gogs Web Console" . . 57
4.12 "Pipeline d'intégration et de déploiement continu : Jenkins Pods" 58
4.13 "Pipeline d'intégration et de déploiement continu : Jenkins Web Console" . 58
4.14 "Pipeline d'intégration et de déploiement continu : Nexus3 Pod" 59
4.15 "Pipeline d'intégration et de déploiement continu : Nexus3 Web Console" . 60
4.16 "Pipeline d'intégration et de déploiement continu : SonarQube Pod" 61
4.17 "Pipeline d'intégration et de déploiement continu : SonarQube Web Console" 61
4.18 "Pipeline d'intégration et de déploiement continu : Jenkins Job" 62
4.19 "Pipeline d'intégration et de déploiement continu : Pipeline Script" 63
4.20 "Pipeline d'intégration et de déploiement continu : Application Code Analyses" 63
4.21 "Pipeline d'intégration et de déploiement continu : Application Image - Build" 64

4.22 "Pipeline d'intégration et de déploiement continu : Application Web Interface" 64
4.23 "Supervision :""Grafana Web Console" 65
4.24 "Supervision :""Alert Manager Web Console" 66
4.25 "Intégration d'openshift avec ManageIQ :""Openshift Provider Token" . . . 67
4.26 "Intégration d'Openshift avec ManageIQ: ManageIQ Configurations 1" . . . 67
4.27 "Intégration d'Openshift avec ManageIQ: ManageIQ Configurations 2 . . . 68
4.28 "Intégration d'Openshift avec ManageIQ: ManageIQ Container Console" . . 68
4.29 "Intégration d'Openshift avec ManageIQ: Openshift Nodes" 69
4.30 "Intégration d'Openshift avec ManageIQ: Openshift Pods" 69
4.31 "Intégration d'Openshift avec ManageIQ: Openshift Projects" 70
4.32 "Inventory File All-in-one:" 74
4.33 "Inventory File : Master, Node, Etcd :" 74
4.34 "Déploiement de Nexus3" 75
4.35 "Add in the JBoss Repositories : jboss et jboss-ce" 75
4.36 "Maven Public Repository" 76
4.37 "Déploiement de Gogs" 76
4.38 "Gogs installation 1" 77
4.39 "Gogs installation 2" 77
4.40 "Pipeline Frontend NodeJS 1" 78
4.41 "Pipeline Frontend NodeJS 2" 78
4.42 "Frontend NodeJS Pod" 79
4.43 "Frontend NodeJS Web Interface" 79
4.44 "Pipeline Backend NodeJS + MongoDB 1" 80
4.45 "Pipeline Backend NodeJS + MongoDB 1" 80
4.46 "Backend NodeJS + MongoDB Pods" 81
4.47 "Backend NodeJS Web Interface" 81

List of Tables

1.1 Backlog produit 18

2.1 Plateformes et Fonctionnalités 24
2.2 Tableau comparatif Git vs SVN 26
2.3 Tableau comparatif Jenkins vs Bamboo vs Travis 27
2.4 Tableau comparatif Chef vs Puppet vs Ansible 28
2.5 Tableau comparatif machine virtuelle vs Docker 29
2.6 Tableau comparatif Nexus sonatype vs JFrog Artifactory 30
2.7 Tableau comparatif SonarQube vs Codacy 31
2.8 Tableau comparatif OpenShift Origin vs Cloud Foundry 31
2.9 Tableau récapitulatif des choix technologiques 32

Lexique

ASP: Active Server Pages de Microsoft

JSP: Java Server Pages de Sun

CD: Continuous Delivry

CI: Continuous Integration

CPU: Central Processing Unit

HTTP: HyperText Transfer Protocol

RAM: Random Access Memory

SSH: Secure SHell

TLS: Transport Layer Security

URL: Uniform Resource Locator

YAML: Yet Another Markup Language

IT: Information Technology

Paas: Platform as a service

SGBDR: Relational database

Introduction

Contexte académique

Ce stage coïncide avec la bourse de fin d'études obligatoire de fin d'études exigée d'un étudiant de l'école supérieure de communication de Tunis, Sup'Com, pour compléter les 3 ans de formation d'ingénieur. L'étudiant doit mettre en pratique des notions théoriques et académiques dispensées au cours des 5 premiers semestres d'études d'ingénieur pour travailler sur une problématique professionnelle ou de recherche en lien direct avec son domaine d'expertise.

Mon stage a été réalisé au sein de la société "Standard Sharing Software (3S)". Ce stage a duré quatre mois du 15 février au 15 juin à Tunis. Le sujet portait sur le cloud et l'automatisation.

Les concepts du Cloud Computing sont utilisés par les entreprises de développement logiciel pour améliorer leur rythme de production.
Au cours de développement logiciel, les développeurs utilisent souvent la méthodologie agile pour une meilleur gestion des projets. Un projet agile intègre d'une maniéré continue les changements des utilisateurs à chaque itération.
L'union de Cloud Computing et de la méthodologie agile fournit des services très riches en termes de productivité, sécurité et automatisation de déploiement.
Le DevOps se base sur deux parties : une partie opérationnelle et une partie de production. Ces deux parties aident également à aligner l'effort de tous les équipes sur un objectif commun qui aide à réduire les problèmes, à gagner en temps et à réduire le temps de livraison.

Notre rapport s'articule autour de quatre chapitres. Nous présentons, d'abord, un premier chapitre, intitulé "Cadre Général" où nous présentons l'organisme d'accueil, la problématique et la solution proposée. Nous enchaînons avec l'objectif du projet et notre contribution, pour finir avec la méthodologie de développement adoptée.
Notre deuxième chapitre intitulé "Étude Préalable" nous aide à détailler les composants de base de notre projet et de passer par la suite à la comparaison entre les technologies les plus utilisés sur le marché et peuvent satisfaire notre besoin. Finalement, nous procédons à la synthèse de choix et la préconisation.

Au cours du troisième chapitre qui est "Analyse et Spécification" nous identifions les acteurs de notre système, nous spécifions les besoins fonctionnels, techniques et non fonctionnels décelés. Ensuite, nous détaillons le diagramme des composants. Puis, nous procédons à la représentation de ces besoins et nous présentons la vue dynamique des deux volets " Intégration continue " et " Déploiement continu ".
Le chapitre "Réalisation" du système mis en place qui permet de présenter l'environnement matériel et logiciel de notre travail, de présenter les différentes technologies que nous avons utilisé pour aboutir à ce travail et pour finir, d'exposer quelques interfaces graphiques relatives à notre travail. Je clôture ce rapport par une conclusion générale.

Chapter 1
Cadre général

1.1 Introduction

Ce chapitre a pour objectif de situer notre travail par rapport à son cadre général. Nous commençons tout d'abord par la présentation de l'organisme d'accueil, ensuite nous passons à l'étude l'existant et aux constations. Nous enchaînons avec l'énonciation de l'objectif du projet avant de clôturer par la méthodologie de développement.

1.2 Organisme d'accueil

Fondée en 1988, Standard Sharing Software (3S) est un Groupe Intégrateur d'infrastructures et de services à valeur technologique, misant sur l'innovation et l'anticipation du futur. 3S offre des solutions de bout en bout, dans deux secteurs d'activités : IT et CleanTech. Il offre les meilleures solutions d'infrastructure IT et DATA Center à forte valeur technologique, grâce à leur expertise en sécurité informatique, communication unifiée et stockage de données, selon les standards internationaux. [1]
L'entreprise agile, est parvenue, au fil de ses 34 ans d'existence, à se positionner en tant qu'Intégrateur N°1 en Tunisie et en Afrique du Nord. L'entreprise a su jouer le rôle de précurseur dans un marché en mutation fulgurante. Avec 450 compétences, hautement qualifiées et aguerries, dont près des deux tiers sont ingénieurs ou techniciens dotés de plus de 320 certifications. [1]

1.2.1 Activités et produits de 3S

L'activité de 3S s'articule principalement sur 6 axes: [1]

- Architecture, intégration et déploiement IT
- Consulting et stratégie IT
- Support et maintenance
- Outsourcing et mission en régie
- Audit et mesures des performances
- Formation et Transfert de compétences

3S fournit pour ses clients des produits de référence dans différents domaines : [1]

1. Cloud Computing : L'accompagnent dans la phase de design, de conception et de réalisation d'un Cloud à travers :
   - Des services à la demande et sur mesure.
   - Haute disponibilité des services.
   - Déploiement instantané de nouvelles ressources (serveurs, applications, postes de travail)
2. Stockage et Sauvegarde :

   Des solutions de stockage fiable et totalement adaptées aux besoins des clients.
3. Cybersécurité :

   3S garantit l'intégrité, la confidentialité et la disponibilité de Système d'information des clients.
4. Solution IPTV :
   Les solutions IP TV garantissent une intégration bidirectionnelle avec les leaders de systèmes d'information hôtelier et hospitalier.

1.2.2 Département d'accueil

Durant mon stage, j'étais affecté au département « Cloud and Automation ». Leurs principales missions consistent à déployer et mettre en œuvre des solutions de stockage, de virtualisation, de sauvegarde et de Cloud pour leurs clients.

1.3 Problématique et solution

L'équipe "Cloud and Automation" trouve beaucoup des problèmes dans le déploiement des projets dans des machines virtuelles ou dans des serveurs physiques vue la limitation des ressources, le risque de perdre les services en cas d'arrêt du matériel, la consommation de beaucoup des ressources systèmes et la compatibilité des bibliothèques.
On peut résoudre tous ces problèmes en migrant vers la containerisation grâce çà des technologies comme Docker et Kubernetes. Et pour améliorer de plus le développement des projets informatiques au sein de notre département et vue que nous sommes partenaires de groupe Red Hat. Nous avons choisi d'utiliser la plateforme Openshift pour l'orchestration des containeurs et le déploiement des projets.

1.4 Objectifs et contributions

L'objectif de ce projet consiste à la conception et l'implémentation d'une plateforme Cloud PaaS autour du Framework OpenShift. Une fois le cluster est déployé, Nous arrivons à la deuxième partie qui consiste à l'automatisation des catalogues de services d'application multi-tiers. Nous passons par la suite à la dernière partie : l'administration d'un pipeline d'intégration et de déploiement continu avec les conteneurs.
La mise en place de la plateforme Openshift est basée sur Ansible. Openshift-Ansible contient des rôles et des playbooks open sources pour installer, mettre à niveau et gérer les clusters OpenShift.
OpenShift peut s'exécuter sur des produits Red Hat comme Red Hat Enterprise Linux (RHEL), CentOS, ou Fedora. Cela peut être sur une infrastructure cloud publique ou privée, directement sur matériel, ou à l'aide de machines virtuelles. Ayant Kubernetes comme base, OpenShift hérite de toutes les constructions de base pour l'orchestration des conteneurs et, en outre, les étend à l'aide des extensions.
Une bonne affaire de ces extensions provienne de l'ajout de capacités ou de fonctionnalités ne fait pas partie de la base de Kubernetes mais qui sont nécessaires pour faire fonctionner correctement la plateforme. D'autres extensions proviennent de l'application des meilleures pratiques prescriptives conçues pour se conformer à la stabilité et aux réglementations requises sur les environnements d'entreprise (c'est-à-dire, RBAC, CI / CD Pipelines, etc.) [3].
L'architecture N-tier est une architecture client-serveur. Les différents composants logiciels sont reliés pour exécuter une ou plusieurs applications. Une architecture 4-tier par exemple comprend les couches suivantes :

- La couche de présentation contenant les clients.
- La couche applicative contient les traitements (facturation, calcule d'amortissement...).
- La couche d'objets métier (Facture, Client...).
- La couche d'accès aux données (SGBDR, Objet, Fichiers, ...).

Les catalogues de services sont essentiellement des applications Web 2-tiers ou 3-tiers de différents langages comme JAVA, Python et Java Script (NodeJS).
Le pipeline d'intégration continu englobe la mise en place :

- D'un serveur de gestion de version "Gogs".
- D'un serveur d'orchestration "Jenkins".
- D'un serveur d'analyse de code "SonarQube".
- D'un serveur de gestionnaire de dépôt maven "Nexus sonatype".

Le pipeline de déploiement continu comprend deux étapes principaux qui sont:

- Déploiement : "build and deployment"
- Vérification de déploiement : "verify deployment".

La figure 1.1 illustre le pipeline d'intégration continu et de déploiement continu que nous voulons mettre en place.

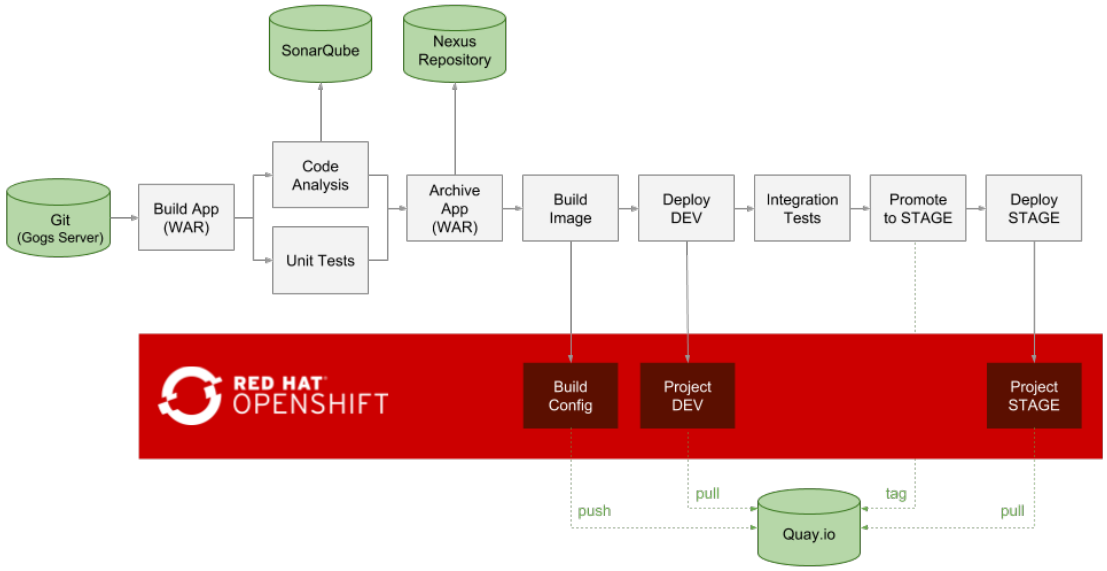


Figure 1.1: Objectifs et contributions : Project CI/CD Pipeline "Project CI/CD Pipeline"

1.5 Méthodologie de travail

Dans cette section, nous allons exposer la méthodologie de travaille utilisée pour la gestion et réalisation de ce projet.

1.5.1 Gestion de projet : Scrum

Pour la gestion de notre projet j'avais suivi un model inspiré de Scrum sans l'appliquer pour toutes ses facettes. Scrum est un cadre de travail (Framework) aidant à livrer des produits en évitant des problèmes complexes. [6]

La figure 1.2, illustre le fonctionnement de Scrum, l'équipe se réunit quotidiennement pour faire le point sur ce qui a été fait et à faire. Chacun de mon sprint a duré une semaine. Chaque sprint a produit un livrable.

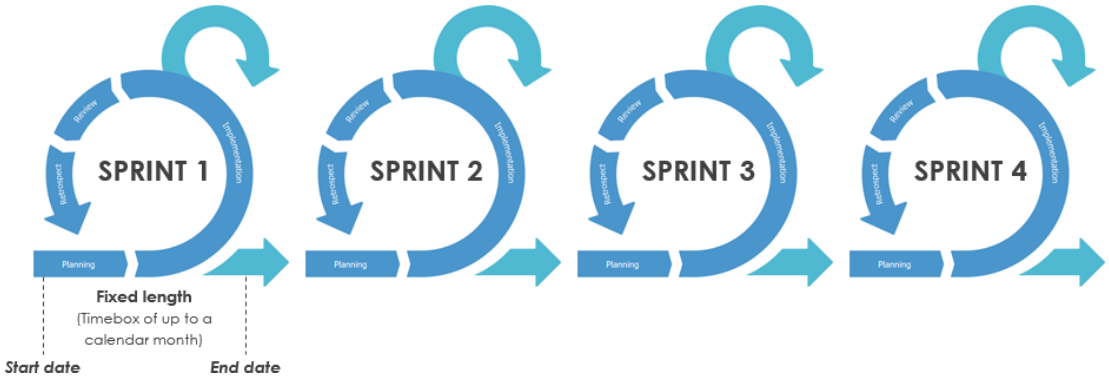


Figure 1.2: Gestion de projet : Scrum
" Présentation de la méthode Scrum"

Le backlog de produit présenté dans le tableau suivant contient les spécifications fonctionnelles exiges par le client. L'évaluation de difficulté est divisée en quatre : facile, moyen, assez difficile et difficile.

Numéro	User story	Description	Priorité	Difficulté	Durée
1	Mettre en place un cluster OpenShift	En tant que configurateur je peux mettre en place un cluster OpenShift	2	Assez difficile	4 semaines.
2	Automatisation du catalogue de services	En tant que dévloppeur je peux créer un catalogue de services. En tant que configurateur je peux automatiser le catalogue de services	1	Moyen	4 semaines.
3	Mettre en place une chaîne CI/CD	En tant que configurateur je peux mettre en place une chaîne d'intégration continue. En tant que développeur je peux créer les configurations nécessaires pour chaque projet et déployer le projet sur une plateforme de déploiement continu.	1	Difficile	4 semaines.
4	Intégration de Openshift avec ManageIQ	En tant que configurateur je peux intégrer Openshift avec ManageIQ	3	Assez difficile	4 semaines.

Table 1.1: Backlog produit

1.5.2 Méthodologie de développement : 2TUP

Pour la réalisation de ce projet, la méthodologie en Y ou 2TUP m'a semblé la plus approprié.
2TUP est un processus unifié qui a pour but d'apporter une réponse aux contraintes de changement fonctionnelles et techniques qui s'imposent aux systèmes d'information. 2TUP propose un cycle de développement qui dissocie les aspects techniques des aspects fonctionnels. Il part du constat que toute évolution imposée au système d'information peut se décomposer et se traiter parallèlement, suivant un axe fonctionnel et un axe technique. [10]
Il distingue ainsi deux branches (fonctionnelle et technique) dont les résultats sont fusionnés pour réaliser le système. On obtient un processus de développement en Y comme l'illustre la figure 1.3. Il faut noter que le processus 2TUP commence d'abord par une étude préliminaire. Dans cette étude préliminaire, il s'agit d'identifier les acteurs qui vont interagir avec le système, les messages qu'échangent les acteurs et le système, puis à produire un cahier de charges et enfin à modéliser le contexte. [10]

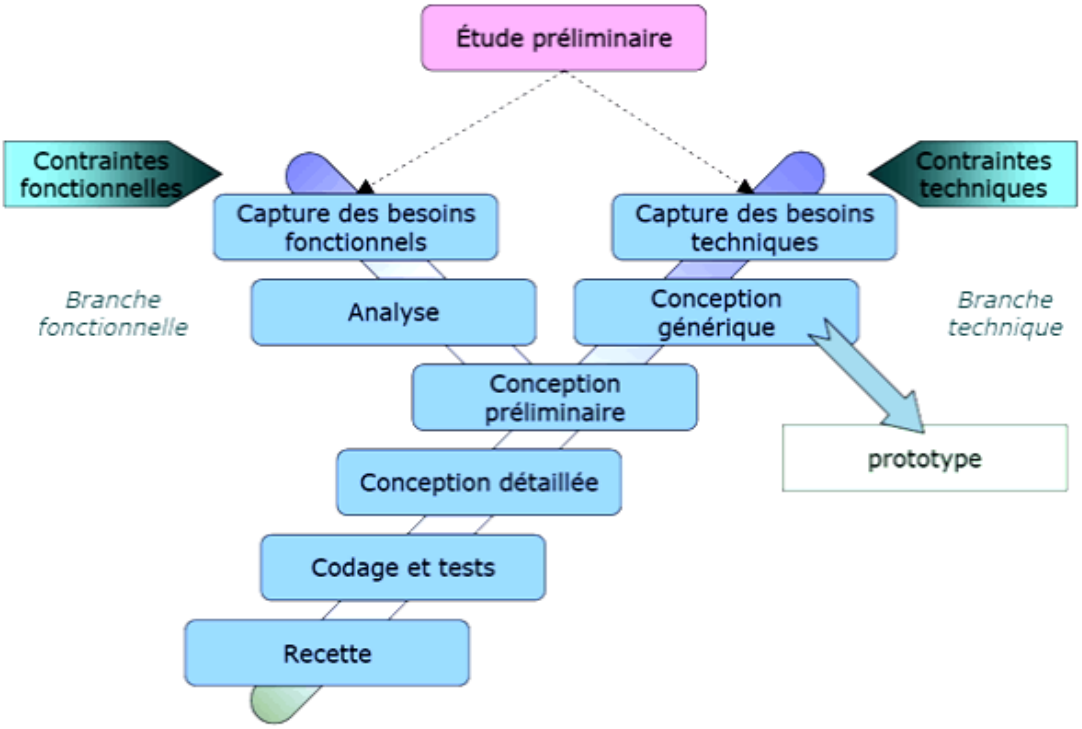


Figure 1.3: Méthodologie de développement : 2TUP
"Le processus 2TUP"

- Branche technique : correspond à la partie où les besoins techniques sont fixés et où l'architecture matérielle et logicielle est énoncée. Au cours de cette partie, j'avais établi, les prérequis matériels tel que les caractéristiques techniques des serveurs à utiliser et les logiciels avec lesquelles je vais mettre en place les pipelines.
- Branche fonctionnelle : permet la capture et l'analyse des besoins fonctionnels pour déterminer les exigences demandées. J'ai commencé par l'étude de l'existant et ensuite j'ai passé par identifier les besoins de notre système pour éviter les problèmes de production.

- Branche conception et développement logiciel : permet d'intégrer les deux branches précédemment expliquées afin d'aboutir au système final qui correspond à l'objectif de mon projet. Le fusionnement des deux branches a abouti à la mise en place d'un pipeline d'intégration et de déploiement continus et l'exploitation des outils de monitoring qui donne des informations sur l'état des différentes composantes qui interviennent dans le pipeline. Pour la documentation de ces volets, j'avais choisi de préciser toutes les exigences dans chaque description de cas d'utilisation.

1.6 Conclusion

Dans cette partie de notre rapport, nous avons présenté l'organisme d'accueil. Nous avons présenté notre problématique et la solution proposée. Nous avons aussi énoncé l'objectif de notre projet avant de clôturer avec la méthodologie de développement adoptée. Ce qui nous prépare bien à avancer dans ce projet pour que nous commençons à détailler les besoins attendus.

Chapter 2
Etude préalable

2.1 Introduction

Dans ce chapitre, nous commençons par définir l'intégration continue et le déploiement continu. Ensuite, nous identifions des composants clés dans notre système avant de passer à une étude comparative. Enfin, nous faisons la synthèse de nos choix technologiques et nos préconisations.

2.2 Intégration continue

L'intégration continue est un ensemble de pratiques consistant à tester de manière automatisée chaque révision de code avant de le déployer en production.
L'intégration continue est un ensemble de pratiques qui testent automatiquement chaque révision de code avant de la déployer en production. Lorsque le développeur a codé la fonction, il a également conçu le test pertinent et l'a ajouté à son référentiel de code. Le serveur d'intégration exécutera ensuite tous les tests pour vérifier si des régressions ont été introduites dans le code source en raison de cet ajout. Si aucune erreur n'est détectée, le serveur intégré peut déployer directement le code en production. [12]
Le but souhaité à travers cette approche est de localiser les bugs automatiquement avant la mise en production du projet. Identifions les points faibles et forts de l'équipe nous permet d'avoir une vue complète du logiciel. Faisant face aux problèmes rencontrés rend notre intervention plus efficace et avec plus de réactivité pour terminer plusieurs projets et gagner en temps pendant tous les phases de production.
Pour bénéficier des avantages qu'apporte le concept d'intégration continue, il est nécessaire de suivre quelques règles de bonne pratique comme le maintien d'un unique dépôt pour le code source visionné, effectuer plusieurs commits par jour et par développeur, automatiser la compilation du code, et le maintien d'un de compilation court. L'architecture d'un pipeline d'intégration continu, fait intervenir plusieurs éléments :

- Un gestionnaire de code.
- Un serveur d'orchestration ou d'intégration continu.
- Un serveur de gestion de dépôt
- Un serveur d'analyse de code

2.3 Déploiement continu

Après l'intégration continue, on passe à la phase de déploiement continu (CD) afin de mettre en production nos projets vue que les tests de développement sont validés dans la phase précédente. Cette phase consiste à automatiser les commandes de déploiements réalisés manuellement au passé. L'union de ces deux étapes forme la fameuse chaine CI/CD.
Pour arriver à cette étape, il faut s'assurer de l'environnement de production et faire les tests nécessaires qui garantissent le bon fonctionnement de notre nouvelle fonctionnalité. [13]

2.4 Éléments clés

Dans cette section, nous définissons quelques éléments clés, à savoir docker, cluster, Kubernetes et Pod.

2.4.1 Docker

Le logiciel « Docker » est une technologie de conteneurisation qui permet la création et l'utilisation de conteneurs Linux.
Avec la technologie Docker, nous pouvons traiter les conteneurs comme des machines virtuelles très légères et modulaires. En outre, ces conteneurs nous offrent une grande flexibilité : nous pouvons les créer, déployer, copier et déplacer d'un environnement à un autre, ce qui nous permet d'optimiser nos applications pour le cloud. [14]

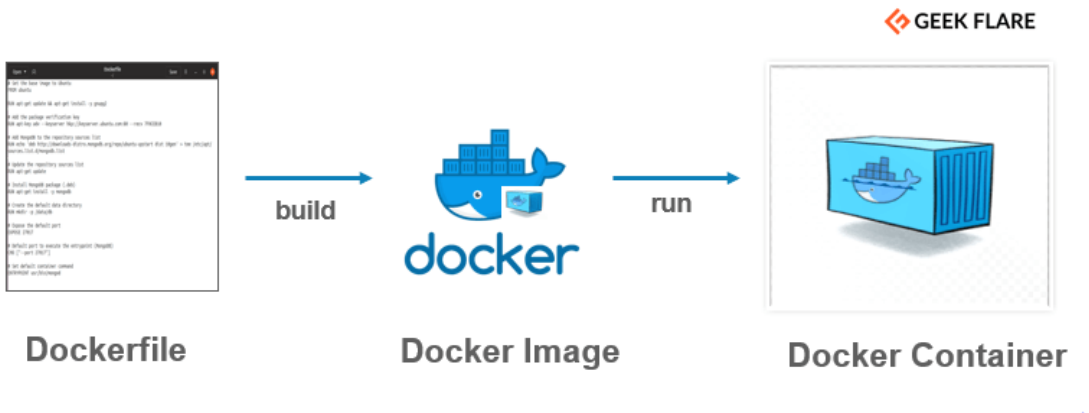


Figure 2.1: Docker
"Les containeurs docker"

2.4.2 Cluster

Dans un système informatique, un cluster, est définit comme étant un groupe de ressources, telles que des serveurs ou des ordinateurs. Ce groupe se comporte comme un seul et même système. Il en résulte ainsi une disponibilité élevée, et parfois, des fonctions de traitement parallèle et d'équilibrage de la charge. C'est la notion de clustering. [8]

2.4.3 Kubernetes

Kubernetes est un programme open source dédié à la gestion de conteneurs Linux. Il automatise le déploiement, l'évolution, la maintenance, la planification et le fonctionnement de nombreux conteneurs d'application sur des clusters. Il contient des outils d'orchestration, de catalogages de services et équilibrage de charge utilisables avec les conteneurs Docker. Kubernetes exécute les conteneurs dans des Pods.[15]

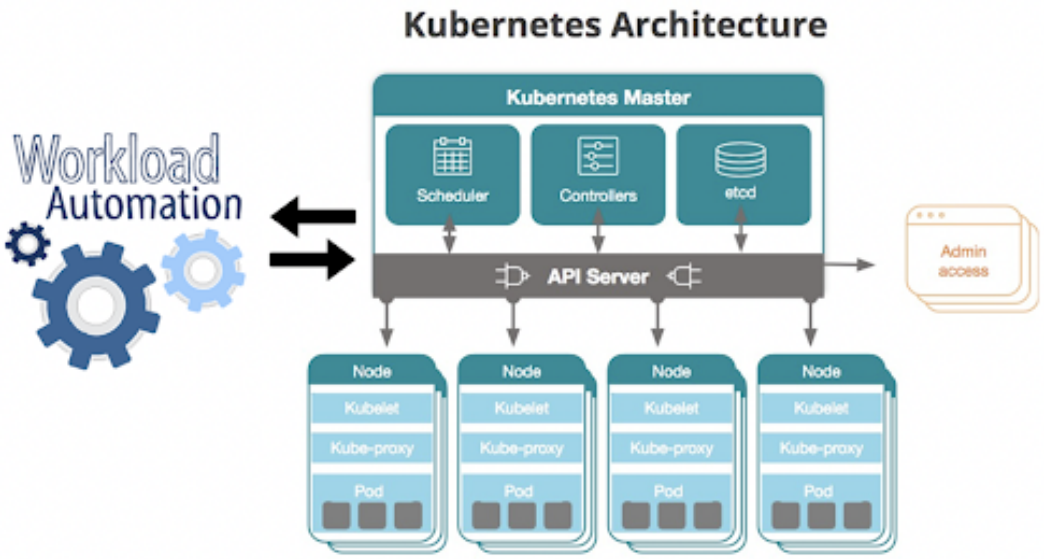


Figure 2.2: Kubernetes
"Kubernetes Orchestration"

2.4.4 Pod

Un Pod est l'unité qui héberge un ou plusieurs conteneurs. Ces derniers, partagent des ressources. Pour chaque Pod, Kubernetes choisit une machine ayant une capacité de traitement suffisante et lance les conteneurs associés. [15]

2.5 Étude fonctionnelle

Les principales solutions sur le marché pour le déploiement des applications sur des containeurs sont Red Hat OpenShift, VMware Tanzu, OpenStack, Cloud Foundry.
Le tableau 2.1 explique les fonctionnalités de chaque solution.

Plateforme	Fonctionnalités
Red Hat OpenShift est une entreprise qui développe une famille de logiciels de conteneurisation.	• Container Orchestration • Contrôles/Permissions d'accès • Gestion du déploiement • Intégration continue • Runtime Container Security
VMware Tanzu est un fournisseur de solutions permettant de créer, exécuter et gérer des applications sur Kubernetes.	• Exécution de Kubernetes dans Vsphère. • Exécution de kubernetes sur plusieurs Clouds. • Adopter la livraison DevSecOps des charges de travail sur Kubernetes.
OpenStack est une société proposant la plateforme de cloud computing standard ouverte pour les clouds publics et privés.	• Une utilisation qui nécessite peu de matériel, les services du cloud peuvent généralement être utilisés avec n'importe quel appareil compatible avec Internet. • Haute sécurité et fiabilité des données. • Évolutivité du volume de stockage et de la puissance. • Les services Cloud permettent un travail collaboratif.
Cloud Foundry est une plate-forme ouverte en tant que service offrant un choix de clouds, de cadres de développement et de services d'application.	• Portabilité des applications. • Mise à l'échelle automatique des applications. • Administration centralisée de la plateforme. • Intégration avec des composants de journalisation externes comme Elasticsearch et Logstash.

Table 2.1: Plateformes et Fonctionnalités

Cette comparaison a été faîtes pour un but académique, le choix de la plateforme de déploiement avec lequel nous allons travailler a été fait par l'entreprise hôte 3S., il s'agit de Red Hat OpenShift vue que notre département est partenaire avec Red Hat.

2.6 Étude comparative des solutions

Pour la mise en place d'un pipeline d'intégration et de déploiement continus et du tableau de supervision que présente l'état de notre solution, nous avons besoin d'un ensemble d'outils spécifiques comme Grafana et Prometheus, un gestionnaire de version, un orchestrateur, un serveur de dépôt maven, un serveur d'analyse de code et un serveur de déploiement continu. De plus, pour faciliter la tâche de la mise en place de cluster Openshift, nous avons aussi besoin d'un outil d'automatisation de configuration des ordinateurs.
Dans cette section du rapport nous procédons à une étude comparative entre les différents outils existant sur le marché et pouvant répondre à notre besoin.

2.6.1 Etude de gestion de version : Git vs SVN

Pour la gestion de version, nous comparons Git et SVN vue qu'ils sont les outils les plus répandus dans le marché. Voici un tableau qui résume les fonctionnalités de ces deux outils qui nous aide à choisir l'outil qui satisfait notre besoin. [4]
Le tableau 2.2 compare entre les deux outils de gestion de versions en se référant aux fonctionnalités de base de ces deux outils :

- architecture,
- single point of failure,
- contrôle d'accès,
- stockage du contenu,
- branches,
- espace mémoire utilisé.

.

Aspect	SVN	Git
Architecture	Centralisé, uniquement le répertoire central a l'historique complet des changements. Les utilisateurs doivent communiquer via le réseau avec le répertoire central pour obtenir l'historique. Les backups sont gérés et maintenu indépendamment du SVN.	Distribué, tous les développeurs qui vérifient le code à partir d'un dépôt / serveur central auront leur propre référentiel cloné installé sur leur machine. Ce qui permet les développeurs de pouvoir créer une nouvelle branche, de faire un « commit » sur un fichier et de revoir une version même en absence de connexion.
Single point of failure	Si le répertoire central est endommagé, alors uniquement les données enregistrées dans le dernier backup sont récupérable.	La notion de « single point of failure » n'existe pas car il y a autant de copie du dossier qu'il y'en a d'utilisateur si ce n'est plus.
Contrôle d'accès	Nécessité d'un « commit access » due au fait de la centralisation.	Non nécessite d'un « Commit access » puisque le système est distribué. Il faut juste décider de fusionner quoi à partir de quel utilisateur.
Stockage de contenu	Stockage des métadonnées des fichiers sous forme de dossier caché .svn	Stockage de tout le contenu dans un dossier .git , c'est le dépôt du code cloné sur la machine client.
Les branches	Les branches sont gérées comme étant un autre dossier dans le dépôt du code, elles ne sont pas historiées. Pour connaître s'il y a eu fusion entre des branches, il faut explicitement exécuter une commande. Cette façon de gérer les branches démultiplie les chances de créer des branches « orpheline ». Une fois la fusion est faite on ne peut plus savoir ce qui a été modifié.	La gestion des branches est assez simple, on peut facilement naviguer entre les différentes branches d'un même répertoire de travail. Chaque répertoire de travail existant chez un utilisateur est considéré comme une branche. La fusion de la branche inclut les informations suivantes : auteur, heure, date, informations relatives à la branche et à la révision,les changements effectués par rapport à la version antécédente. La fusion suivante, commence automatiquement là où la dernière fusion a été faite.
Espace	Le répertoire de travail contient deux copies de chaque fichier, un pour travailler avec et un caché dans le .svn, ce qui rend l'espace nécessaire relativement grand.	Taille du dépôt de code et répertoire de travail relativement petit (30X inférieur à celui de SVN). Ceci est due au fait que pour Git, le répertoire de travail nécessite seulement un fichier d'index de taille 100 bits pour chaque fichier.

Table 2.2: Tableau comparatif Git vs SVN

Suite à cette comparaison, nous avons opté pour le serveur Git, qui grâce à son architecture décentralisée et le gain en espace qu'il fournit (dépôt 30 fois inférieur à celui de SVN) répond au mieux aux nouvelles attentes de 3S.

2.6.2 Outil d'intégration continue : Jenkins vs Bamboo vs Travis

Jenkins, Travis et Bamboo sont des outils d'orchestration bien connu sur le marché, chacun d'entre eux présente ses avantages et inconvénients. [7]
Le tableau 2.3 permet de comparer entre ces outils, la comparaison se base sur :

- l'aspect Open source du code,
- le nombre de plugins,
- la simplicité d'utilisation,
- l'interfaçage avec les différents outils de gestion de version.

Aspect	Jenkins	Bamboo	Travis
Open source	Oui	Non	Oui
Plugins	Oui	Cher	Peu
Simplicité	Non	Oui	Non
Outil de versionning	Tout	Tout	Github

Table 2.3: Tableau comparatif Jenkins vs Bamboo vs Travis

L'étude comparative entre ces différents serveurs d'orchestration a dirigé notre choix vers Jenkins, qui grâce à ses nombreux plugins, permet un interfaçage facile avec de nombreux outils dont j'avais besoin tel que Git et OpenShift.

2.6.3 Outil d'automatisation de configuration des ordinateurs : Chef vs Puppet vs Ansible

La mise en place des serveurs peut s'avérer une tâche fastidieuse répétitive et sujette aux erreurs de manipulation. L'apparition de "Chef", "Puppet" et "Ansible" a été longuement attendu. Grâce à ces outils il n'est plus nécessaire de configurer chaque poste individuellement, il suffit de lancer un script, et des dizaines, centaines ou voir millier de machines sont configuré toutes en mêmes temps, réduisant ainsi l'effort fourni et les risques d'erreur dû à la manipulation. [17]

Ici je dresse un tableau comparatif entre ces trois outils de configuration basé sur les aspects suivants :

- le code (open source ou non),
- l'infrastructure,
- le langage,
- l'architecture de chaque outil.

Aspect	Chef	Puppet	Ansible
Open source	Oui	Oui	Oui
Infrastructure	Mutable	Mutable	Mutable
Langage	Procédural	Déclaratif	Procédural
Architecture	Client/serveur	Client/serveur	Client uniquement

Table 2.4: Tableau comparatif Chef vs Puppet vs Ansible

Le fait qu'Ansible ne nécessite aucune installation coté client j'ai insisté à le choisir en tant que serveur de configuration d'ordinateur.

2.6.4 Virtualisation : Machine virtuelle vs Docker

Une machine virtuelle est un système très similaire à un ordinateur. Utiliser des machines virtuelles c'est très proche d'avoir plusieurs ordinateurs dans un seule poste. Chaque machine virtuelle nécessite son système d'exploitation qui nous aide à virtualiser le matériel. [2]

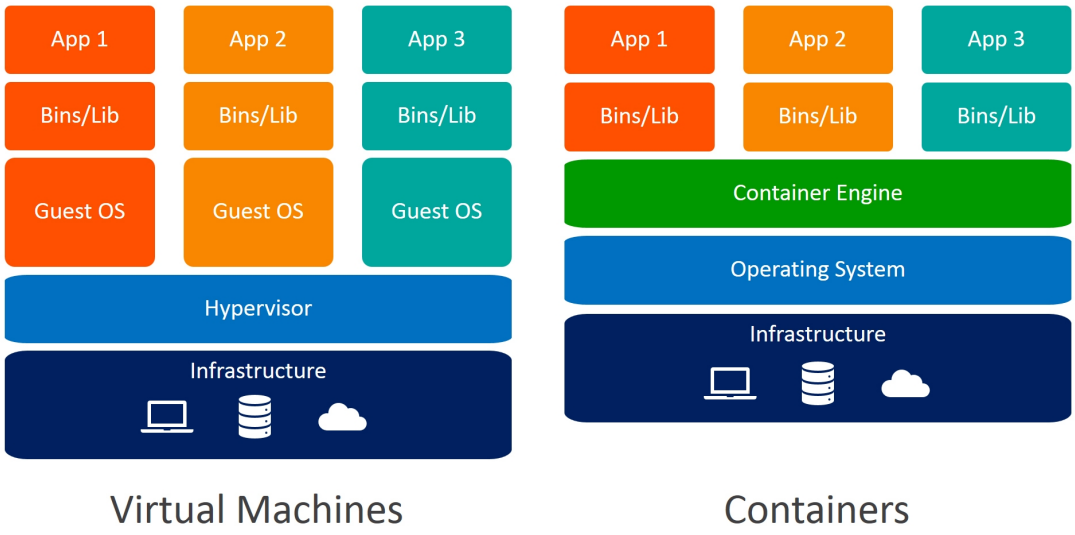


Figure 2.3: Virtualisation : Machine virtuelle vs Docker
"Containers vs. VMs"

Le tableau ci-dessous illustre une comparaison entre la machine virtuelle et Docker basé sur :

- la virtualisation,
- la plateforme de virtualisation,
- le système d'exploitation hôte,
- l'overhead,
- le chargement du système d'exploitation,
- l'isolation,
- le démarrage,
- le nombre d'instances.

Aspect	Machine virtuelle	Docker
virtualisation	Hardware	Kernel
Plateforme de virtualisation	hyperviseur	Docker Engine
Système d'exploitation hôte	Tout (distribution linux, Windows, mac)	Distribution linux
Overhead	Oui	moins important
Chargement système d'exploitation	Son propre système d'exploitation	Aucun
Isolation	Oui	moins importante
démarrage	Prend quelques minutes	Rapide
Nombre d'instances	Quelques-unes	important

Table 2.5: Tableau comparatif machine virtuelle vs Docker

A l'issue de cette comparaison nous déduisons que nous avons besoin de combiner les deux technologies pour répondre à nos besoins.

2.6.5 Outil de gestion de dépôt : Nexus sonatype vs JFrog Artifactory

L'utilisation d'un gestionnaire de dépôt maven a plusieurs utilités, le dépôt peut servir d'un backup sur les applications dont les artefacts sont déployés sur ces serveurs, les artefacts peuvent être utilisé comme des bibliothèques et peuvent être importé par les différents développeurs sur d'autres applications. Nexus et JFrog Artifactory sont les deux gestionnaires de dépôts maven les plus répandu sur le marché.

Le tableau 2.6 est un tableau comparatif sur les deux outils JFog Artifactory et Nexus sonatype, cette comparaison est basée sur : [5]

- Le code (Open source ou non).
- Installation à base de Docker.
- Docker file.
- Taille du jar.

Aspect	JFrog Artifactory	Nexus sonatype
Open source	Oui	Oui
Installation à base de Docker	Moyen	Facile
Docker file	Non disponible	Disponible
Taille du jar	37M	13M

Table 2.6: Tableau comparatif Nexus sonatype vs JFrog Artifactory

Cette comparaison a été faîtes pour un but académique, le choix du gestionnaire du dépôt maven avec lequel nous allons travailler a été fait par l'entreprise hôte 3S., il s'agit de Nexus sonatype.

2.6.6 Outil d'analyse de code : SonarQube vs Codacy

L'utilisation d'un serveur d'analyse de code est très importante dans notre cas pour détecter les bugs et les problèmes dans notre code. SonarQube et Codacy sont les outils d'analyse de code les plus répandu sur le marché.
Le tableau 2.7 est un tableau comparatif sur les deux outils SonarQube et Codacy, cette comparaison est basée sur : [16]

- Automatisation
- Prise en charge du code
- Analyse
- Etendue

Aspect	SonarQube	Codacy
Automatisation	La meilleure chose à SonarQube est qu'il offre une instance en ligne (SonarCloud) où nous pouvons exécuter un projet open source en forçant un référentiel github.	Vérifie régulièrement vos validations de code. Par exemple, vous transmettez du code à Git, il mettra automatiquement à jour vos notes de code
Prise en charge du code	Fournit une analyse détaillée des piles pour lesquelles il vérifie les bogues et les problèmes dans les piles de code.	Il prenne en charge tous les principaux langages, par exemple Python, PHP, JAVA. Ainsi, il devient rapide et facile d'obtenir une qualité en temps réel pour votre code.
Analyse	Fournit une bonne quantité de documentation sur la configuration et l'installation et comment l'utiliser.	Une interface utilisateur intuitive et un tableau de bord simple donnent un affichage clair de votre base de code.
Etendue	Fournit une forte intégration avec azure devops et jenkins pour la création de pipelines DSL.	Codacy est très populaire et la seule bonne source de solution de qualité de code.

Table 2.7: Tableau comparatif SonarQube vs Codacy

Cette comparaison a été faîtes pour un but académique, le choix d'analyseur de code avec lequel nous allons travailler a été fait par l'entreprise hôte 3S., il s'agit de SonarQube.

2.6.7 Plateforme de déploiement : Openshift vs Cloud Foundry

Pour le déploiement des applications modernes sur des modèles cloud, les deux solutions PAAS les plus connus sur le marché sont Cloud Foundry et Openshift. On peut intégrer une chaîne CI/CD en utilisant ces outils ce qui facilitent beaucoup notre conception.
Le tableau comparatif sur les deux outils OpenShift Origin et Cloud Foundry se base sur les éléments suivants : [11]

- Cloud.
- Outils supportés.
- Systèmes d'exploitation

Aspect	OpenShift	Cloud Foundry
Cloud	PAAS	PAAS
Outils supportés	Data Dog, Jenkins, Eclipse, Jboss studio.	Github, Eclipse, Spring.io, CloudBees
Systèmes d'exploitation	Windows, Linux, Mac	Windows, Mac

Table 2.8: Tableau comparatif OpenShift Origin vs Cloud Foundry

Cette comparaison a été faîtes pour un but académique, le choix de la solution a été fait par l'entreprise hôte 3S., il s'agit d'OpenShift Origin.

2.7 Synthèse et préconisation

À L'issu d'une étape d'étude comparative, j'avais procédé aux choix technologiques énoncé mais que je vais récapituler. Comme serveur de gestion de version j'avais opté pour Git, et pour l'outil d'orchestration, mon choix s'est porté sur Jenkins. Pour la configuration de mon cluster, j'avais utilisé l'outil de configuration d'ordinateur Ansible. Comme technique de virtualisation, j'avais choisi la combinaison entre virtualisation matérielle avec les machines virtuelles et la virtualisation du Kernel avec Docker. L'outil de gestion de dépôt maven que j'avais gardé est Nexus sonatype. L'outil d'analyse de code est SonarQube. Enfin, j'avais choisi OpenShift Origin comme plateforme de déploiement continu. Le tableau 2.9 récapitule mes choix.

Fonctionnalité	Outil
Serveur de gestion de version	Git
Serveur d'orchestration	Jenkins
Configuration du cluster	Ansible
Outil de gestion de dépôt maven	Nexus sonatype
Outil d'analyse de code	SonarQube
Plateforme de déploiement	OpenShift Origin

Table 2.9: Tableau récapitulatif des choix technologiques

2.8 Conclusion

Au cours de ce chapitre, nous avons défini l'intégration continue et le déploiement continu, nous avons aussi présenté quelques éléments clés. Nous avons, par la suite, fait une étude comparative sur les outils qui peuvent répondre à nos besoins et enfin, nous avons fait la synthèse de mes choix technologique. Le prochain chapitre portera sur l'analyse et spécifications ou nous essayons de détailler la conception de notre solution. Vue l'expérience acquise, nous préconisons d'améliorer chaque sous-système de notre solution pour arriver finalement à une solution générale qui respecte les besoins demandés.

Chapter 3
Analyse et spécification

3.1 Introduction

Dans le présent chapitre, nous intéressons aux fonctionnalités que notre système doit satisfaire. Nous commençons par les composants et les acteurs intervenant dans notre solution. Ensuite, nous procédons à l'analyse et la présentation des besoins que le système doit satisfaire. Cette représentation est faite avec UML comprenant le diagramme des composants. Pour les besoins de ce chapitre, nous utilisons les diagrammes de cas d'utilisation accompagné de description textuelle et pour l'architecture dynamique de la solution, nous utilisons les diagrammes d'activités.

3.2 Identification des acteurs

Pour ce système, nous avons identifié 5 participants :

- Développeur : Il est un utilisateur GIT, il peut enregistrer ou récupérer du code depuis le référentiel. C'est le premier acteur à déclencher le pipeline.
- Client : Il s'agit de l'utilisateur final, il doit avoir une URL de production qui lui permet d'accéder Vers l'application.
- Chef de projet : C'est l'utilisateur concerné par le monitoring, il dispose d'un tableau de supervision qui lui permet de connaître l'état du cluster.
- Testeur : C'est un acteur qui intervient dans la phase du test du logiciel, il dispose d'une URL de test pour tester les fonctionnalités de l'application.
- Configurateur : Il intervient dans la phase de configuration du système et a pour rôle la mise en place du pipeline.

Ce système inclut aussi certains composants comme étant des acteurs :

- Jenkins : c'est un orchestrateur et un outil important dans l'intégration continue, il permet de déclencher des builds.
- Nexus : c'est une gestionnaire de dépôt maven, il permet d'exposer des artefacts.
- Docker registry : c'est un composant qui permet le stockage des images docker et leur récupération.
- SonarQube : est un logiciel libre permettant de mesurer la qualité du code source en continu.

3.3 Analyse des besoins

Dans cette partie, nous commençons par analyser les besoins fondamentaux de notre système à travers la partie besoins fonctionnels, les besoins techniques et pour finir avec les besoins non fonctionnels à considérer pour ce système.

3.3.1 Recensement des besoins fonctionnels

- Le système permet au développeur d'exposer ou récupérer un code d'une application sur un serveur de dépôt de code.
- Le système permet au développeur de créer un script pipeline CI/CD.
- Le système permet au développeur de créer un Template pour le build et le déploiement de l'application sur OpenShift.
- Automatiser le déclenchement d'un build sur Jenkins suite à un dépôt de code sur Git (ou Gogs).
- Visionner le build sur OpenShift.
- Récupérer les artefacts générés par Nexus3 suite à un build réussit.
- Automatiser le déploiement des applications sur OpenShift.
- Afficher au sein d'OpenShift, les différentes étapes par lesquelles passe le code avant d'être déployé.
- Le système permet au client d'accéder à l'application via une URL de production qu'il expose par Openshift.
- Le système permet au testeur de tester le bon fonctionnement de l'application en y accédant via l'URL de test qu'il expose.

Le système permet au chef de projet de superviser le pipeline, pour ce faire :

- Le système permet au chef de projet de surveiller l'usage CPU, de la RAM et la pression entrée/sortie sur le réseau par les pods grâce à l'outil Grafana.
- Le chef de projet peut savoir l'état des différentes applications existant sur OpenShift (container creating, running, pending, etc.) grâce à l'outil Prometheus.

3.3.2 Recensement des besoins techniques

Les besoins techniques sont :
* 3 machines virtuelles :

- Master (8 GB RAM)
- Node (16 GB RAM)
- Etcd (8 GB RAM)

* Image ISO CentOS 7.

3.3.3 Besoins non fonctionnels

En combinant les exigences fonctionnelles ci-dessus, mon système doit respecter de nombreuses normes offrent une meilleure qualité de solution.
Dans ces normes, je cite :

- Performance : le temps de réponse du système doit être minimal, et la table de supervision doit s'afficher immédiatement lorsque l'URL est cliquée.
- Scalabilité : Le système doit être capable de supporter l'évolution et l'évolutivité de ses composants, Par conséquent, nous pouvons ajouter autant de nœuds que nécessaire à notre cluster OpenShift.
- Convivialité : le tableau de monitoring doit être compris de façon intuitive, c'est-à-dire Les indicateurs de performance doivent être clairement affichés et commentés.

3.4 Spécification UML – Diagramme des composants:

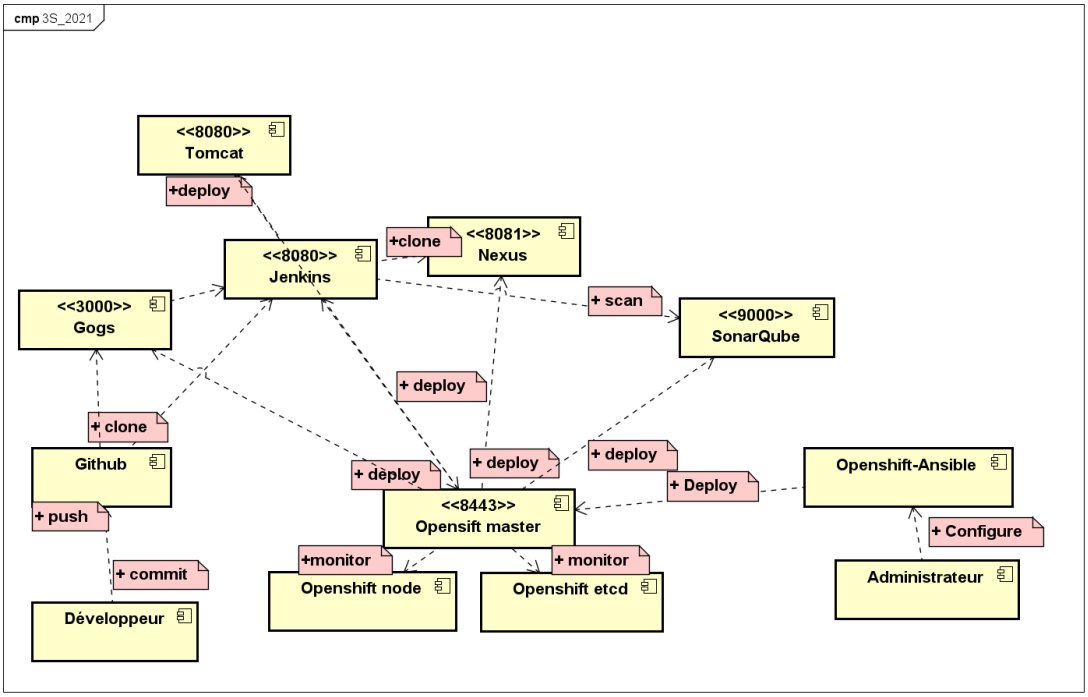


Figure 3.1: spécification UML :"Diagramme des composants du pipeline CI/CD"
"Diagramme des composants du pipeline CI/CD"

Ce diagramme montre le déroulement des opérations les plus importantes effectuées par notre pipeline CI/CD En fait, un administrateur configure Openshift-Ansible pour déployer un cluster Openshift.
Avec Openshift on peut déployer les différents outils de pipeline et on peut aussi déployer nos projets de développement.
Chaque développeur travaille localement : il actualise GitHub pour chaque nouvelle modification du code. Un « build » sera démarré automatiquement sous Jenkins, ce dernier télécharge le code, le compile, puis le scanne à l'aide de SonarQube, et envoie enfin les livrables générés au serveur Nexus. Dans le même temps, un autre job Jenkins sera lancé à Openshift pour créer une image et déployer cette image en conteneurs dans les nœuds d'Openshift.

3.5 Représentation des besoins "Diagrammes de cas d'utilisation"(DCU) :

Au niveau de cette partie, nous essayons de présenter les besoins dégagés sous forme des diagrammes de cas d'utilisation. Le diagramme de cas d'utilisation suivant, nous permet de présenter les principales fonctionnalités auxquels le système doit satisfaire. Les sous-systèmes qui constituent notre solution sont:

- CI : "Continuous Integration" ou intégration continue.
- CD : "Continuous Deployement" ou déploiement continu.
- Configuration.
- Supervision.
- Pré-configuration.

La figure 3.2 illustre le diagramme de cas d'utilisation général de notre système.

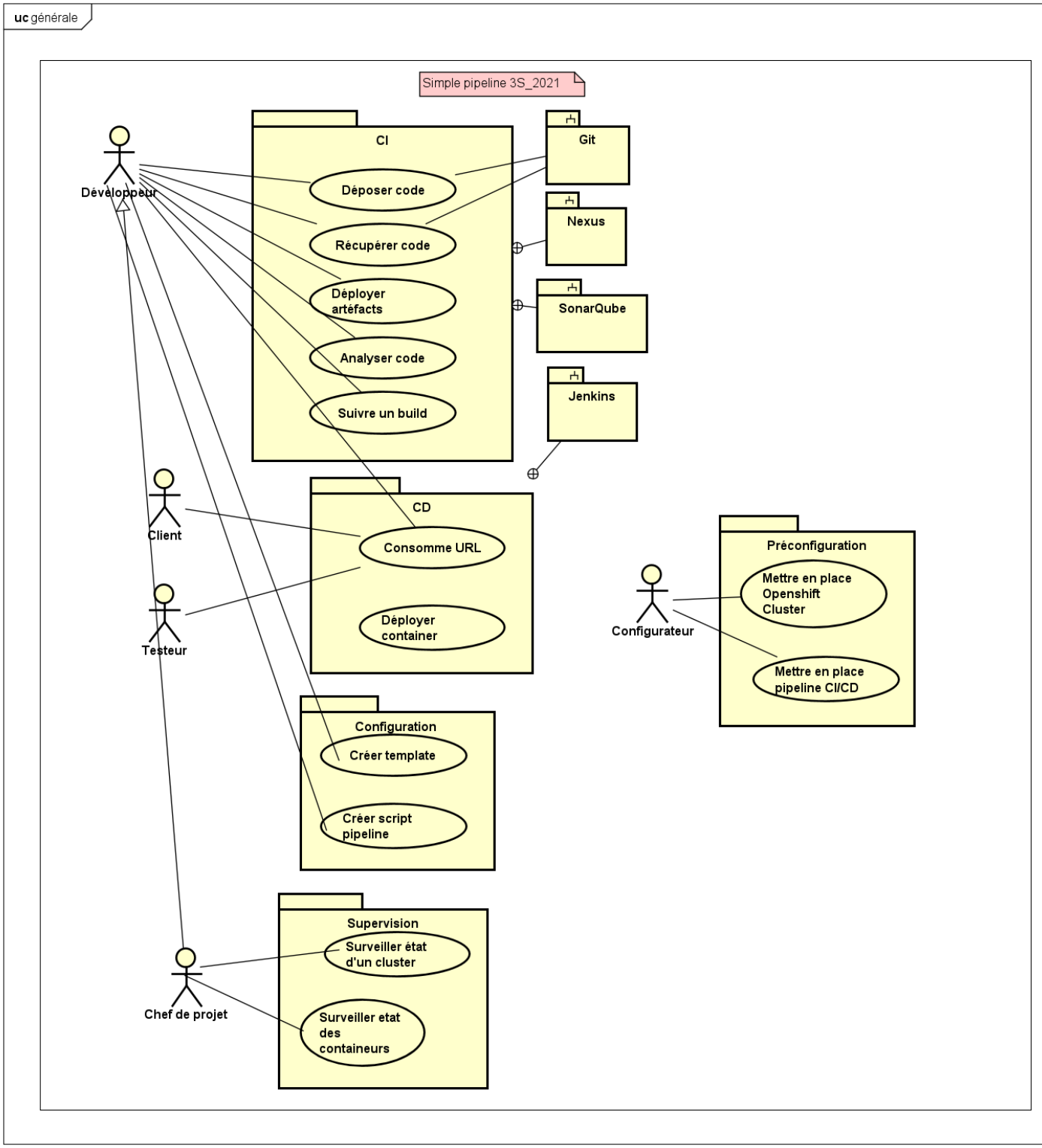


Figure 3.2: Représentation des besoins : "Diagramme de cas d'utilisation général"
"Diagramme de cas d'utilisation général"

Dans ce système, nous distinguons trois profiles principaux : Développeur, Configurateur et Chef de projet.
Le développeur peut agir sur la plateforme avec plusieurs façons.
Il peut :

- Déposer ou récupérer un code à partir de GitHub ou Gogs.
- Suivre un build sur Jenkins ou sur Openshift.
- Déployer artefacts sur Nexus.
- Analyser le code sur SonarQube.
- Accéder à l'URL de l'application déployée sur OpenShift.
- Créer un Template pour les projets qui va être déployé sur OpenShift et qui nécessite une modification de la configuration.
- Créer un script pipeline qui décrit les étapes par lesquelles passe le code (test, build, déploiement).

Le chef de projet est un développeur qui hérite les cas d'utilisation du développeur et peut aussi :

- Surveiller l'état du cluster Openshift (RAM, CPU, état des pods).
- Surveiller l'état des projets à partir de ManageIQ ou à partir d'Openshift Monitoring. Les clients et les testeurs peuvent vérifier l'état des applications et l'avancement de développement grâce aux URLs donnés par Openshift.

Le configurateur peut créer et mettre en place un cluster OpenShift et peut aussi configurer un pipeline d'intégration continue.

3.5.1 Intégration continue :

L'intégration continue se déclenche lorsqu'un développeur expose son code sur un serveur de gestion de version GIT et prend fin lorsque le code est analysé par SonarQube et les artefacts du code sont déployé sur le serveur Nexus.
Le diagramme de cas d'utilisation « Intégration continue » représenté par la figure 3.3 explique les fonctionnalités exigées dans cette partie du travail.

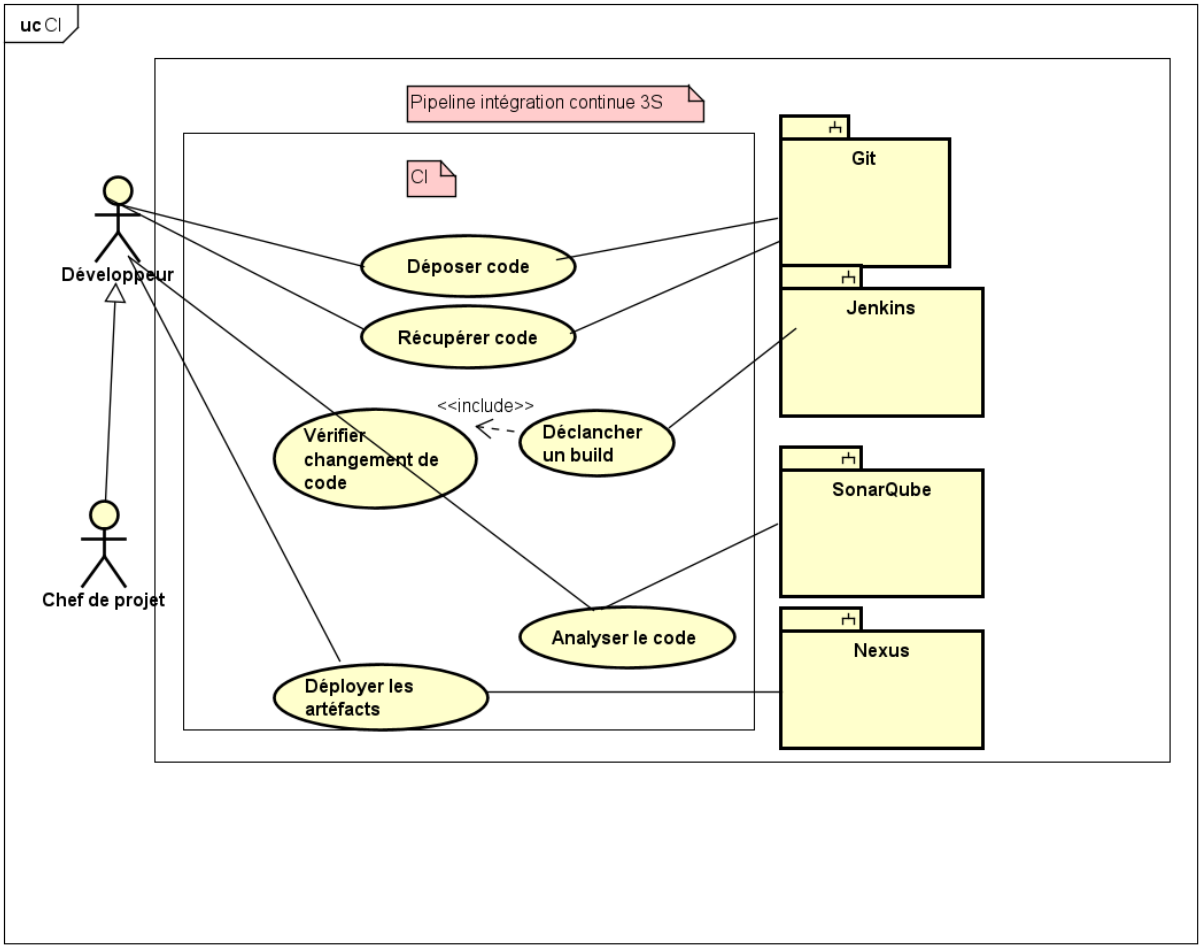


Figure 3.3: Représentation des besoins : "diagramme de cas d'utilisation "Intégration continue"

"diagramme de cas d'utilisation "Intégration continue"."

3.5.2 Déploiement continu

Le volet de déploiement continu, se traduit en un pipeline qui inclut le serveur Jenkins, OpenShift et docker registry.
Le diagramme de cas d'utilisation « déploiement continu » représenté par la figure 3.4 explique les fonctionnalités offertes par ce module de notre projet.

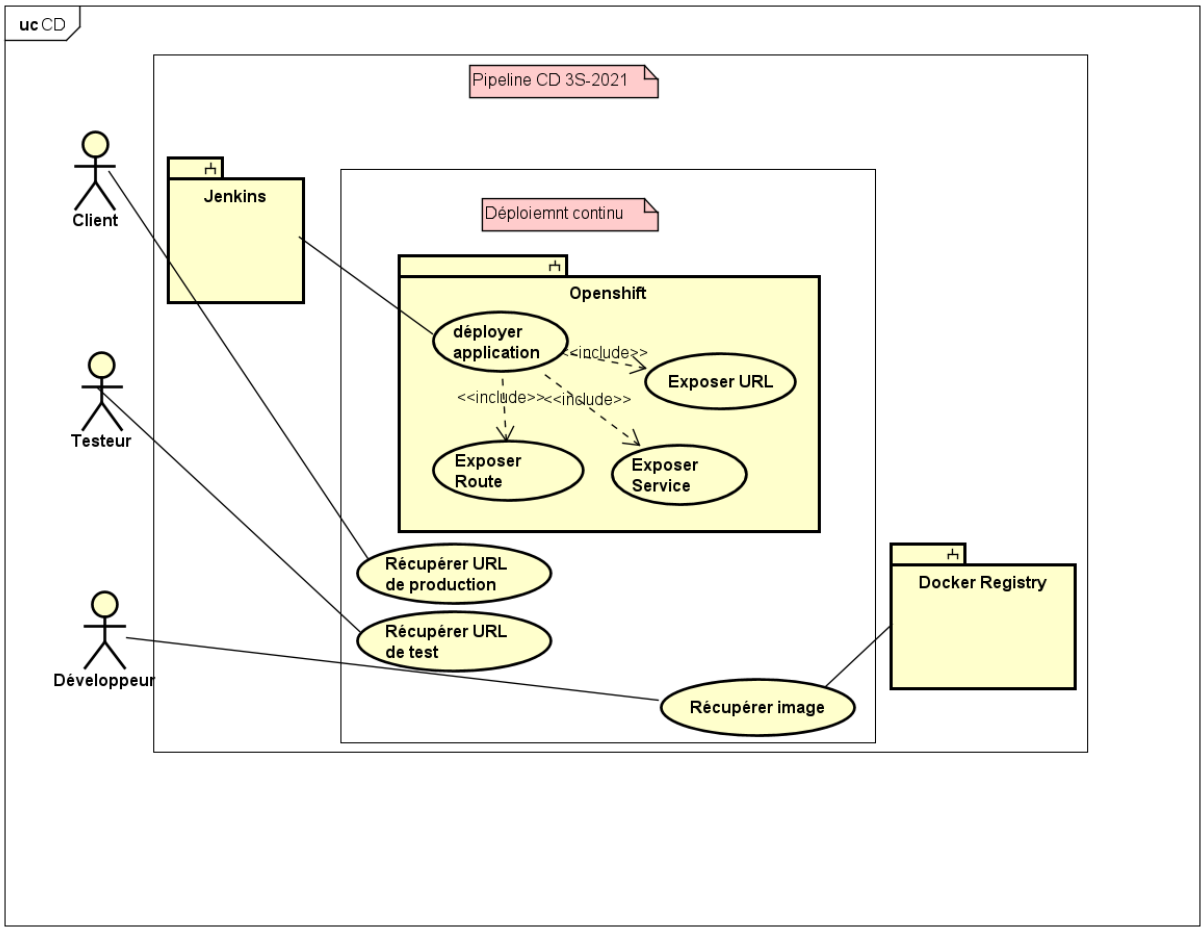


Figure 3.4: Représentation des besoins : "Diagramme de cas d'utilisation "Déploiement continu"

"Diagramme de cas d'utilisation "Déploiement continu"

3.5.3 Pré-configuration

Le volet pré-configuration nous permet de mettre en place une chaîne CI/CD et de mettre en place un cluster Openshift qui est notre outil de déploiement des projets. La mise en place d'un pipeline d'intégration continue a été fait en déployant chaque serveur comme un conteneur Docker sur OpenShift et en intégrant ces outils avec Jenkins.
Le diagramme 3.7 présente les différents cas d'utilisation pour le volet pré-configuration.

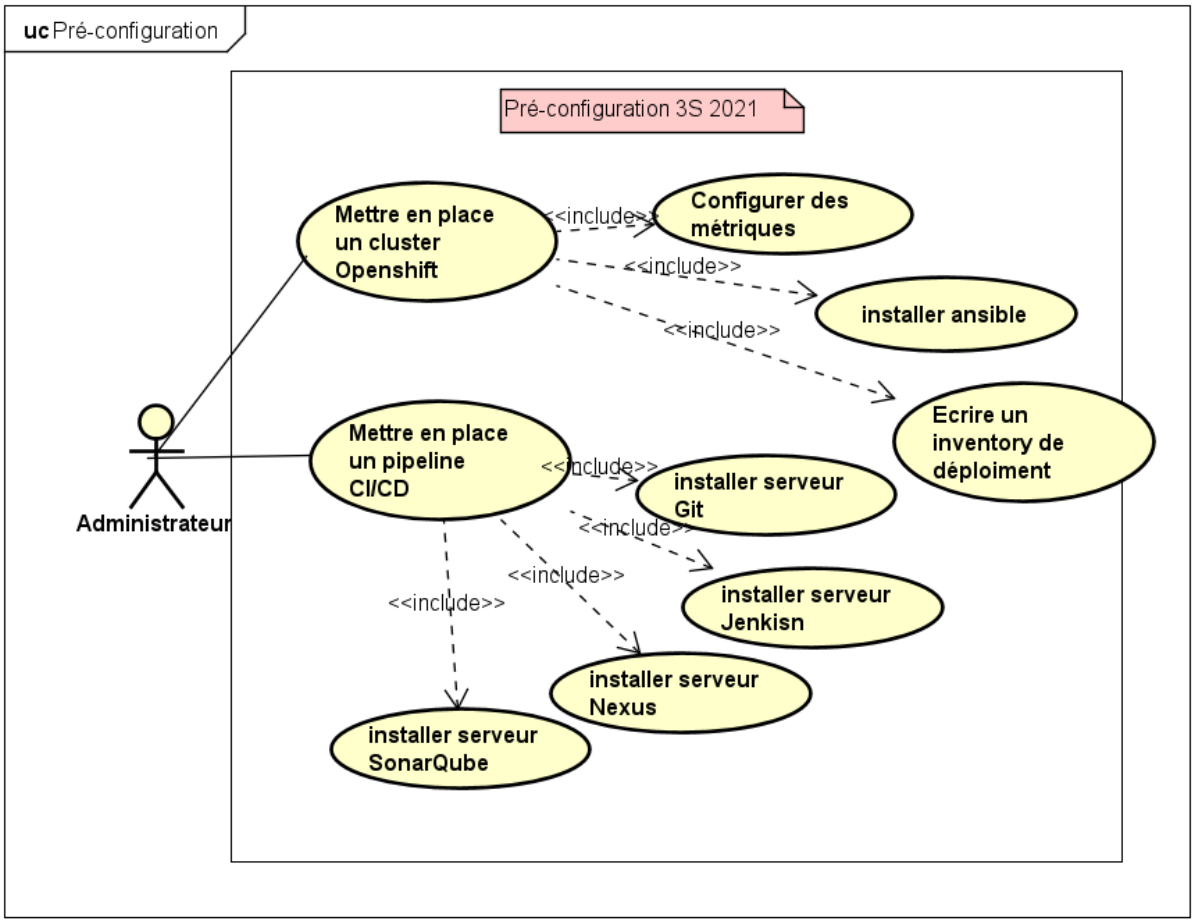


Figure 3.5: Représentation des besoins : "Diagramme de cas d'utilisation "Pré-configuration". "Diagramme de cas d'utilisation "Pré-configuration"."

3.5.4 Configuration

Le module de configuration consiste en la création d'un Template pour chaque projet qui contient les configurations nécessaires pour son build et déploiement sur OpenShift et en la création d'un script pipeline qui définit les étapes par lesquelles passe le projet (développement, tests, déploiement, etc.).
La figure 3.5 est le diagramme de cas d'utilisation illustrant la configuration.
Sa description est récapitulée dans le tableau 3.10 pour la création du Template et le tableau 3.11 pour la création du script.

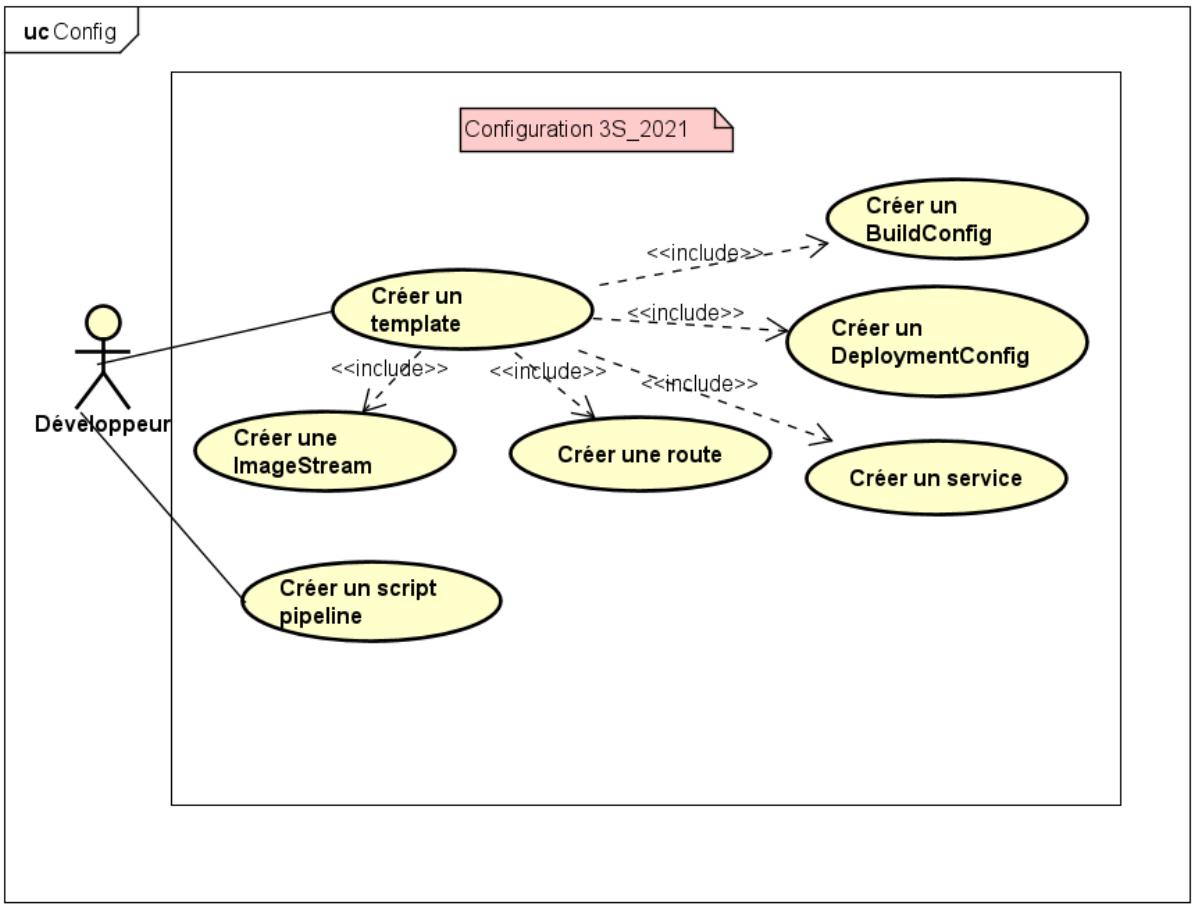


Figure 3.6: Représentation des besoins : "Diagramme de cas d'utilisation "Configuration". "Diagramme de cas d'utilisation "Configuration"."

3.5.5 Supervision

Le module de supervision comprend deux parties, la première est la surveillance de l'état du cluster sur les ressources CPU et RAM, la deuxième est la supervision des états des conteneurs (up, down, suspendu, attente, création de conteneur).
La figure 3.6 présente le diagramme de cas d'utilisation du module « monitoring ».

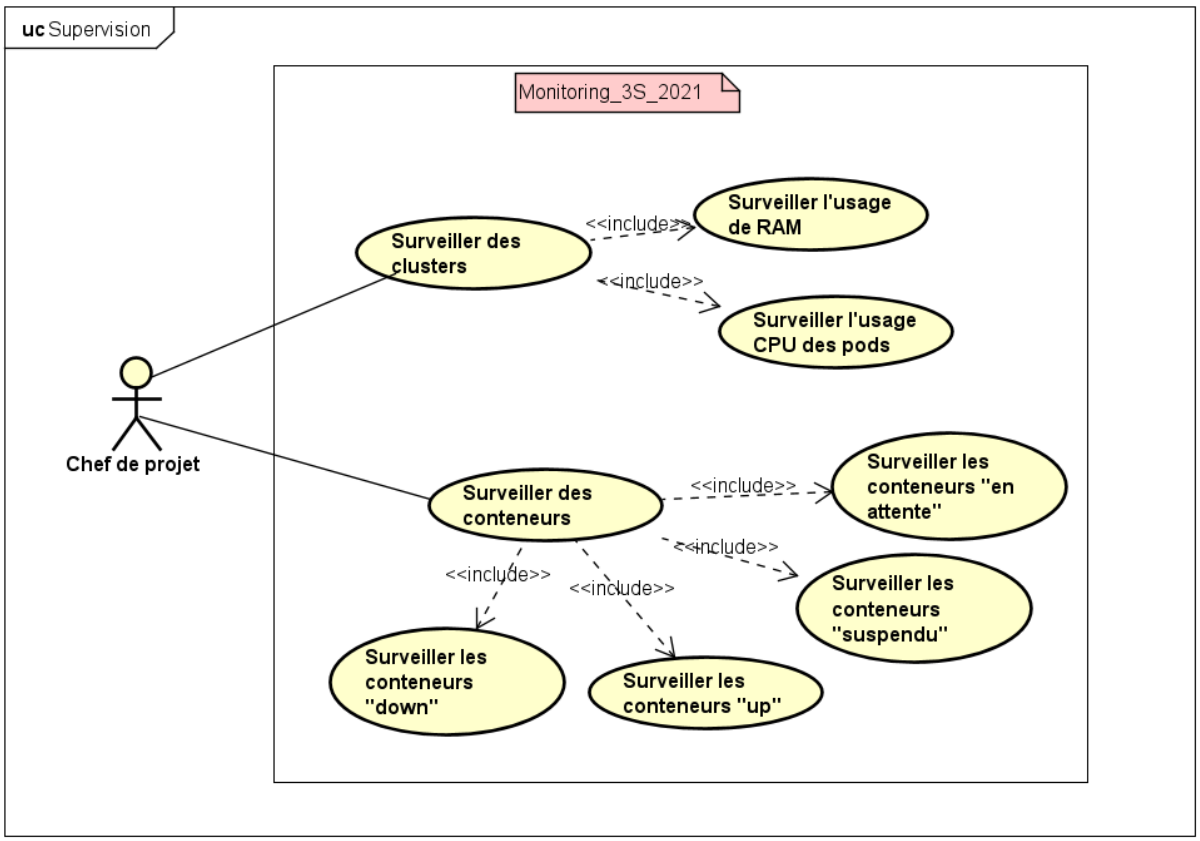


Figure 3.7: Représentation des besoins : "Diagramme de cas d'utilisation "Monitoring". "Diagramme de cas d'utilisation "Monitoring"."

3.6 Aspect dynamique de la solution : "Diagrammes d'activités"

Pour mieux comprendre l'interaction entre les différents composants de la solution, nous présentons la vue dynamique de l'architecture de ce système comportant les deux volets CI et CD.

3.6.1 Processus d'Intégration continue

Le diagramme d'activité illustré dans la figure 3.8, explique le fonctionnement et l'interaction entre les différents composants du pipeline intégration continue.

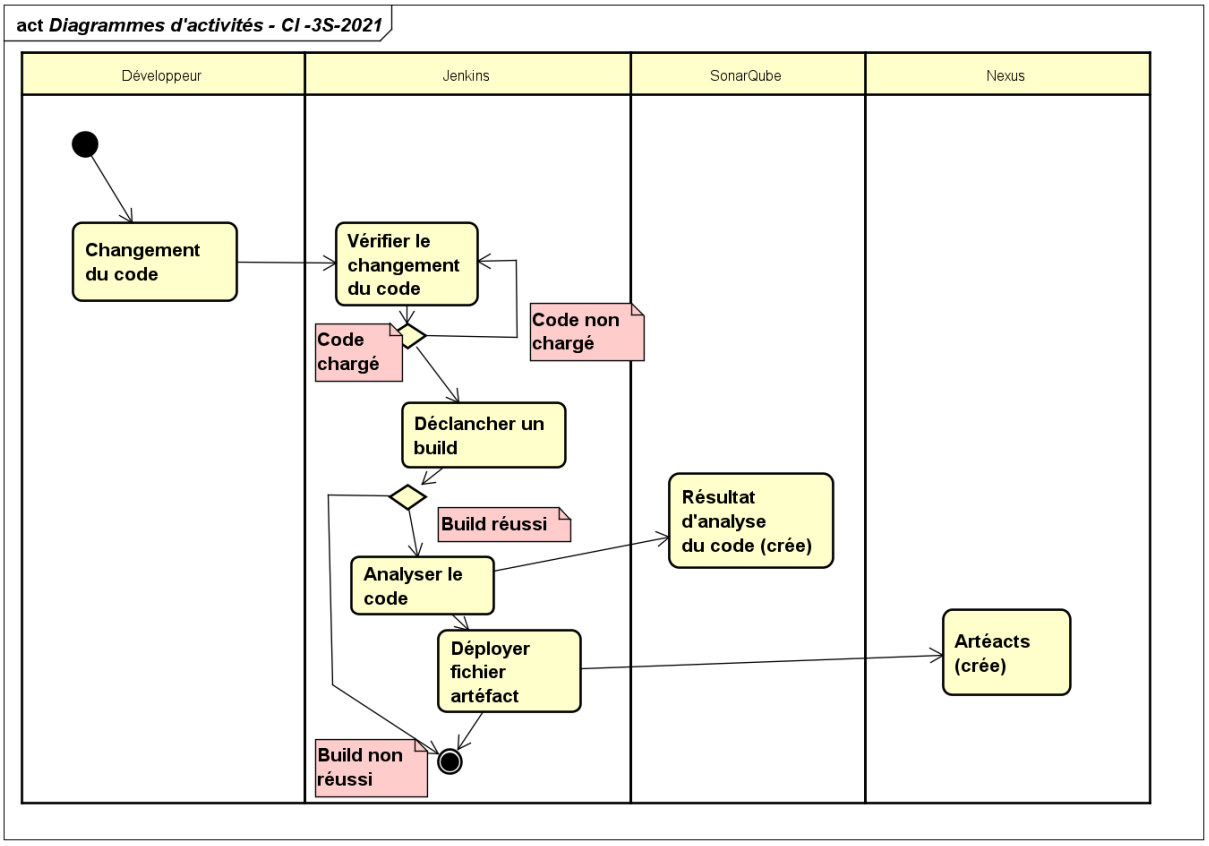


Figure 3.8: Vue dynamique du volet "Intégration continue"
"Processus d'Intégration continue"

Pendant les deux activités Analyser le code et Déployer le fichier artéfacts, Jenkins vérifie automatiquement les adresses des serveurs SonarQube et Nexus et parfois Gogs si le code est enregistré localement dans un serveur Gogs.

3.6.2 Processus de déploiement d'une application

Le diagramme d'activité représenté dans la figure 3.9, illustre la vue dynamique du pipeline de déploiement. En précondition, le build précédemment déclenché doit être effectué avec succès, ainsi, le master nœud d'OpenShift choisit le nœud worker au sein duquel le Pod sera créé en tenant compte des ressources nécessaires pour la création du Pod et des ressources disponibles dans les nœuds du cluster.
Une fois le nœud choisi, une adresse IP interne est alors assignée au Pod, cette adresse sera reconnue uniquement à travers le cluster OpenShift. Ensuite, la plateforme OpenShift instancie l'image docker qu'il a préalablement récupéré du docker registry suite au build réussi, ainsi un conteneur est créé. OpenShift monte le service du conteneur et lui crée une route externe pour qu'il soit accessible aux clients externes.

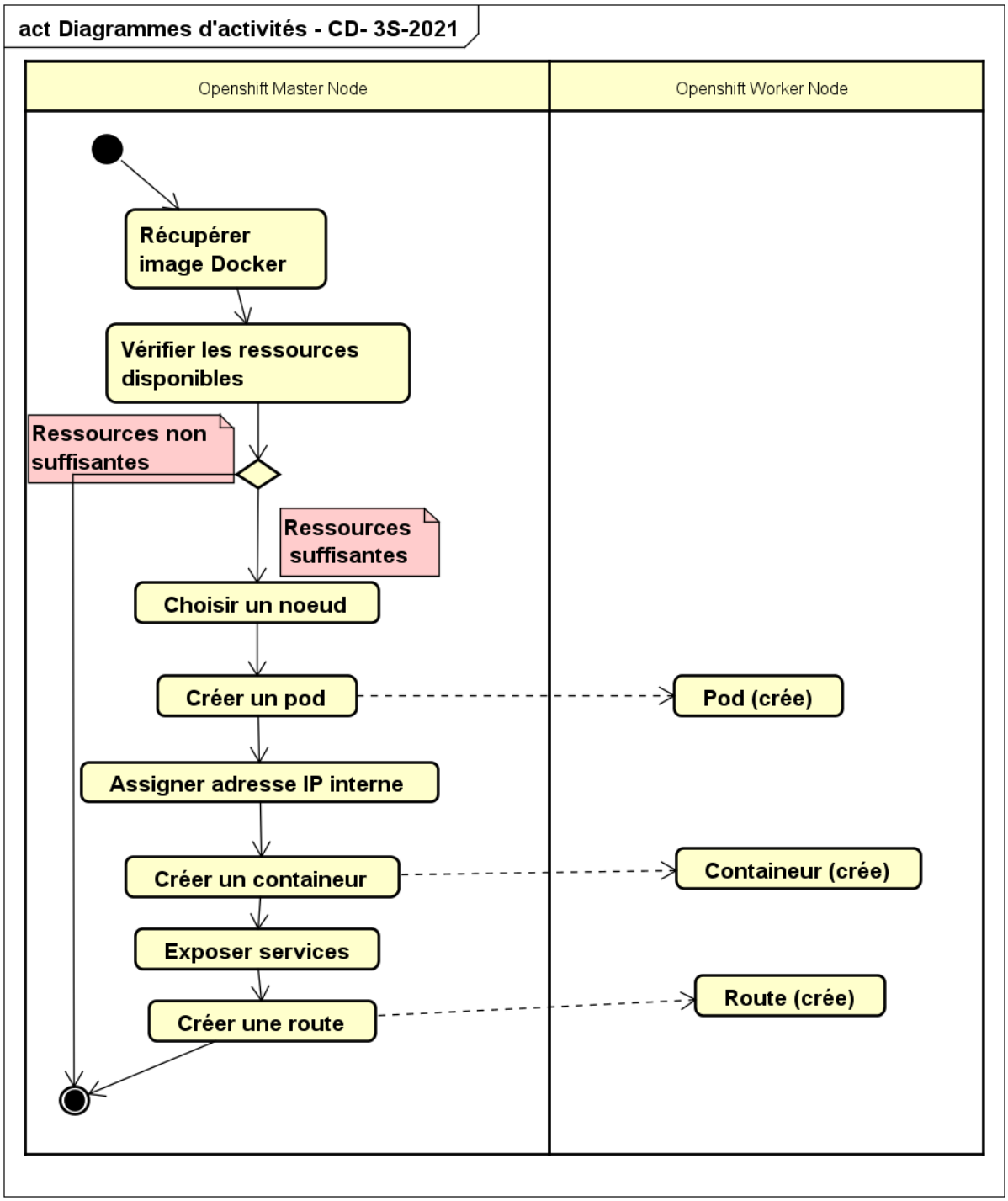


Figure 3.9: Vue dynamique du volet "Déploiement continue"
"Processus de déploiement d'une application"

3.7 Conclusion

Cette partie nous a permis dans un premier lieu d'identifier les acteurs de notre système. Ensuite, nous avons spécifié les besoins fonctionnels et non fonctionnels de notre système, présenter l'analyse des besoins, détailler le diagramme des composants de notre pipeline et nous clôturons l'aspect dynamique de cette dernière. À la vue de ce qui a été réalisé, nous pouvons maintenant passer à la réalisation de la solution en respectant la conception détaillée de chaque sous-système.

Chapter 4
Réalisation

4.1 Introduction

Après avoir traité la partie conceptuelle, nous entamons la partie réalisation de ce projet où nous présentons l'environnement logiciel et matériel de ce travail, ensuite nous exposons les différentes technologies utilisées pour la mise en place de notre système. Enfin, nous présentons les principaux développements de ce projet à travers l'essentiel des imprimes écran de mes interfaces.

4.2 Environnement de travail

Pour mener à bien mon projet, nous avons utilisé 3 machines virtuelles dont les caractéristiques techniques sont les suivantes :
*Master Node :

- Mémoire : 8GB RAM
- Système d'exploitation : CentOS 7.
- Disque dur : 100GB

*Etcd Node :

- Mémoire : 8GB RAM
- Système d'exploitation : CentOS 7.
- Disque dur : 100GB

*Worker Node :

- Mémoire : 16GB RAM
- Système d'exploitation : CentOS 7.
- Disque dur : 200GB

Nous avons aussi utilisé "Mobaxtrem" pour pour accéder aux VMs.

4.3 Implémentation:

4.3.1 Pré-configuration : Mise en place d'un cluster OpenShift

Dans cette partie nous avons effectué les configurations nécessaires pour la mise en place d'un cluster OpenShift grâce à l'outil de configuration d'ordinateurs Ansible.
Le Cluster OpenShift a été mis en place grâce à l'outil de gestion d'ordinateurs « Ansible », pour ce faire, j'avais créé un fichier appelé « myinventory » qui contient la liste des nœuds cibles, les paramètres et les configurations nécessaires au bon déploiement. Les fichiers playbooks sont déjà créés car dans ce projet on travaille avec la version communautaire d'Openshift appelée « OKD ».
Une fois le playbook lancé, Ansible se met à récupérer des informations sur les machines cibles et commence par la suite à effectuer les tâches mentionnées dans le playbook.
La figure 4.1 représente le fichier "Inventory File" que j'ai créé pour lancer les playbooks.

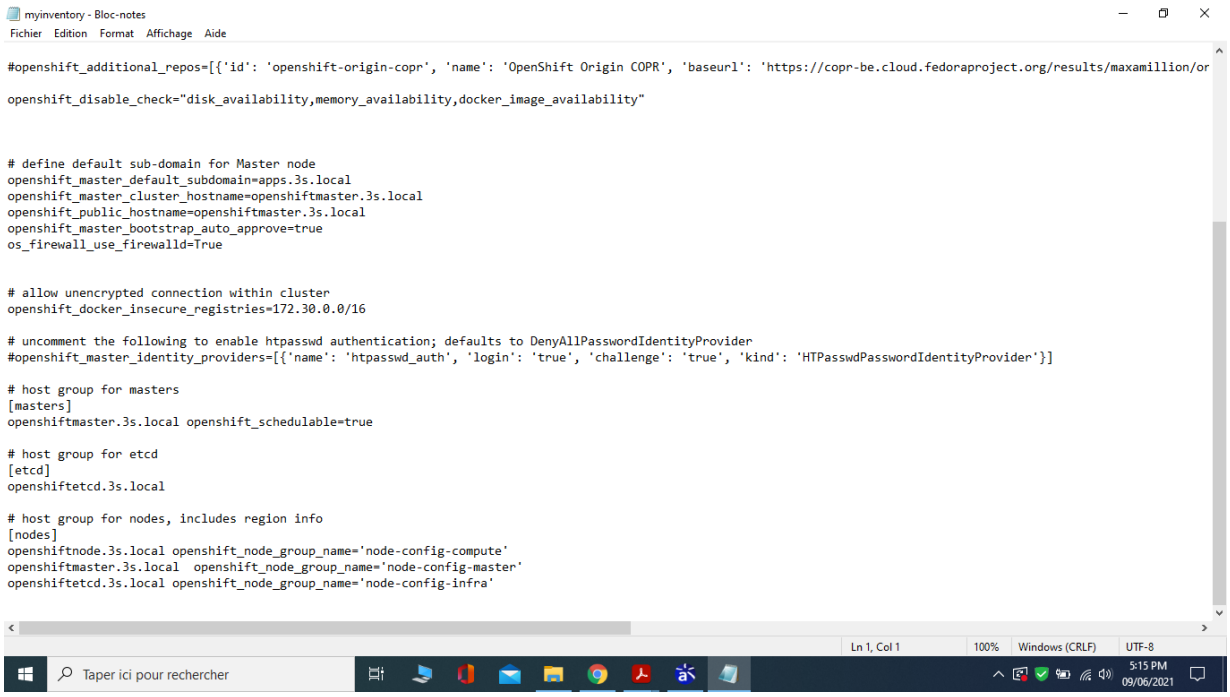


Figure 4.1: "Mise en place d'un cluster OpenShift : Fichier inventory file"
"Fichier inventory file"

La figure 4.2 présente un extrait des logs affiché par Ansible lors de lancement du premier playbook appelée "Prerequisities" qui vise à tester si les packages nécessaires pour le déploiement du cluster Openshift sont valables.

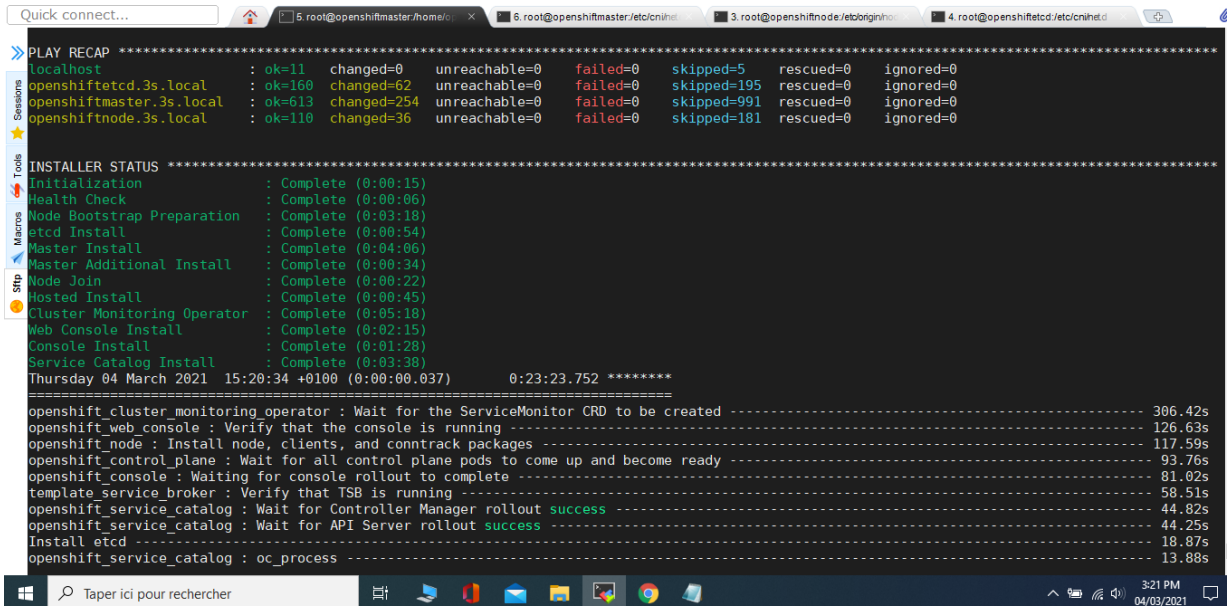


Figure 4.2: "Mise en place d'un cluster OpenShift : Logs Ansible. Prerequisites"
"Logs Ansible. Prerequisites"

La figure 4.3 présente un extrait des logs affiché par Ansible lors de lancement du premier playbook appelée "deploy-cluster" qui installe les packages et les configurations nécessaires et déploie le cluster Openshift.

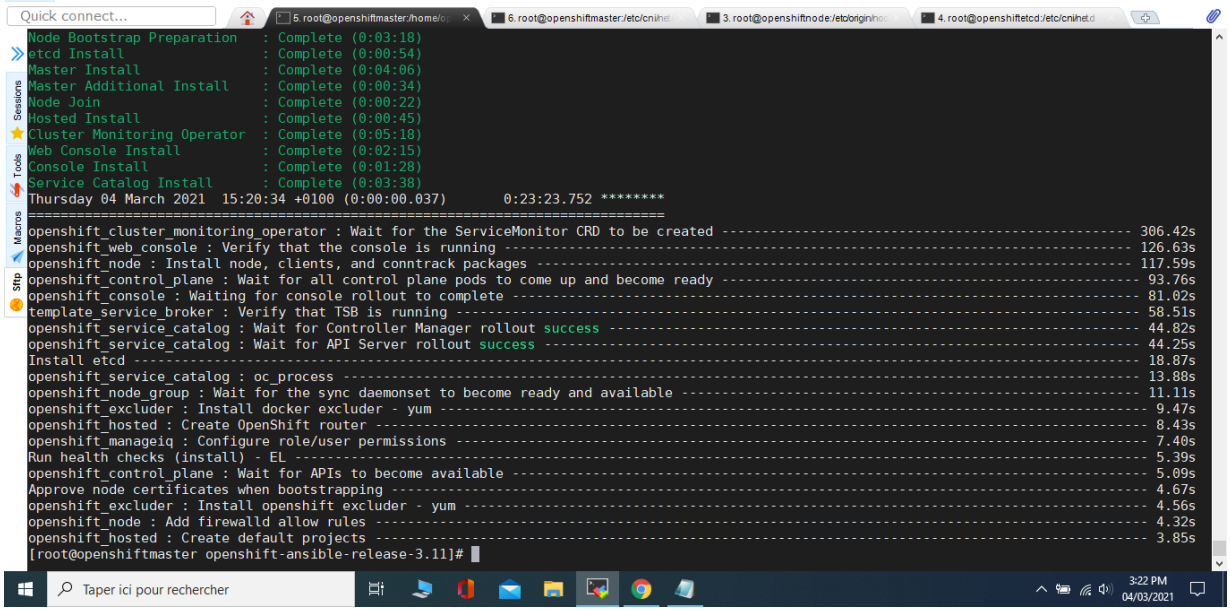


Figure 4.3: "Mise en place d'un cluster OpenShift : "Logs Ansible. Deploy Cluster"
"Logs Ansible. Deploy Cluster"

L'imprime écran que représente la figure 4.4, est la page d'accueil d'OpenShift contenant la liste des projets présents.

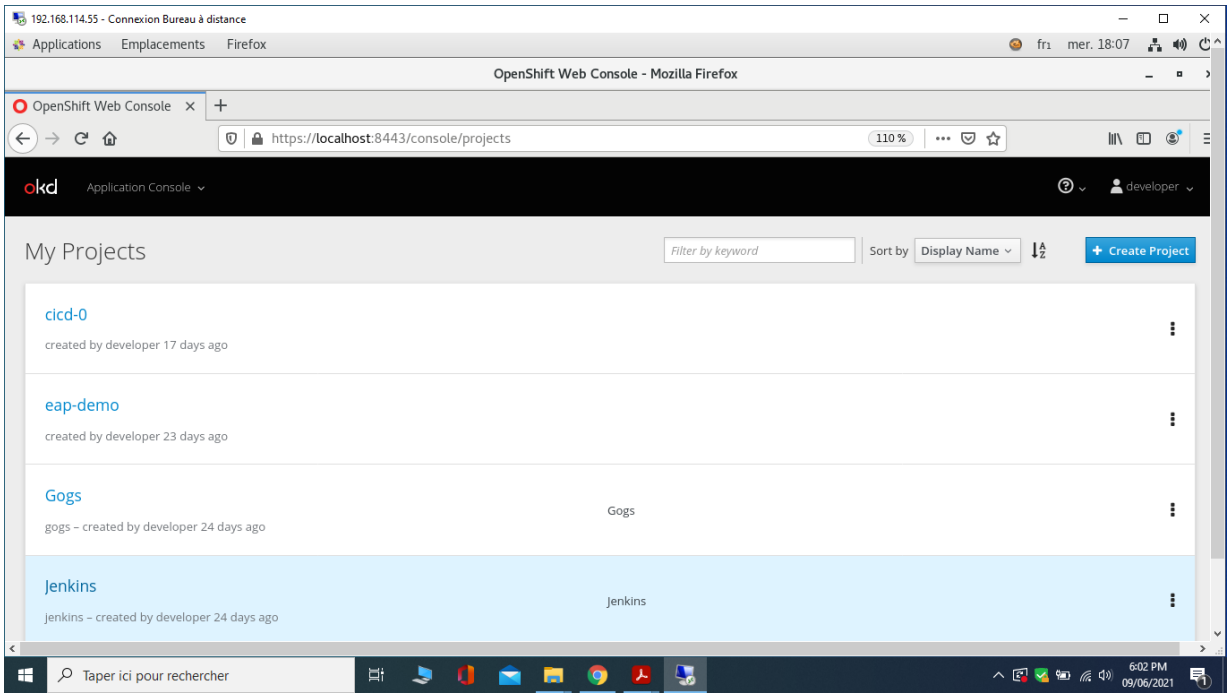


Figure 4.4: "Mise en place d'un cluster OpenShift : Openshift Console: Projets" "Openshift Console: Projets"

L'imprime écran présent dans la figure 4.5, représente un extrait de l'état des nœuds et des namespaces.

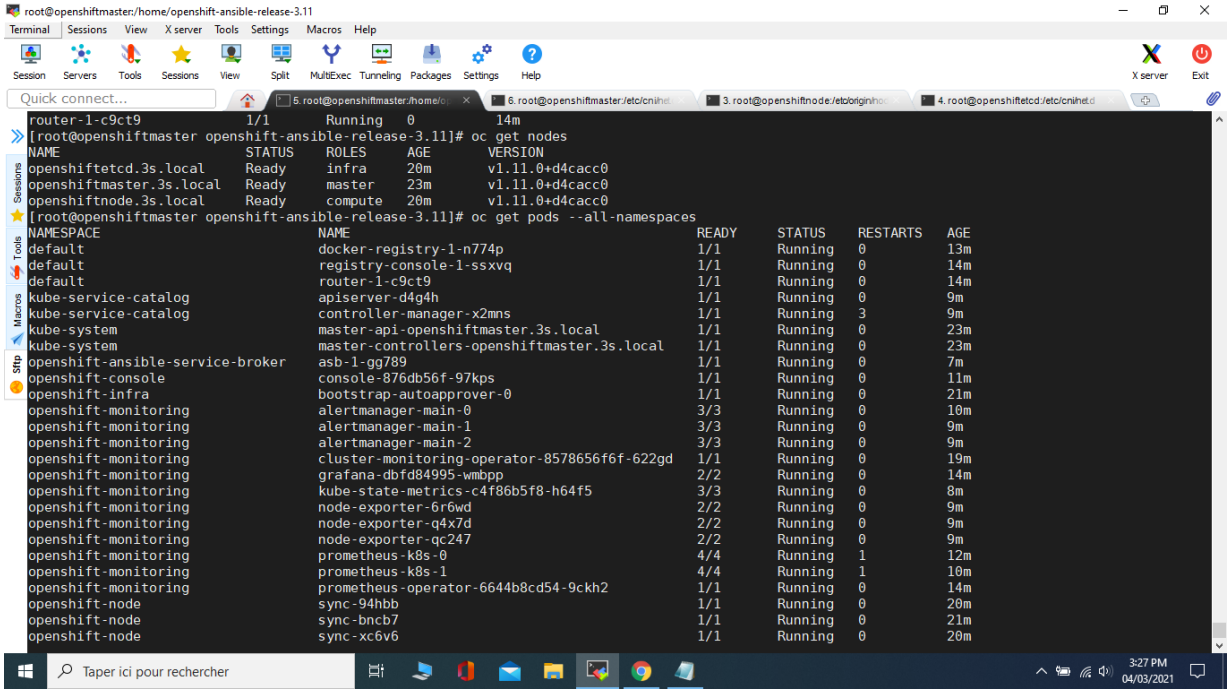


Figure 4.5: "Mise en place d'un cluster OpenShift : Etat du cluster Openshift" "État du cluster Openshift"

4.3.2 Configuration

Le module de configuration est nécessaire au bon déploiement d'une application sur OpenShift. Nous avons besoin dans chaque projet sur Openshift d'un Template et du source code de l'application. Le Template est un fichier YAML ou JSON. Le modèle définira les objets qu'il crée ainsi que des métadonnées pour guider la création de ces objets.
La figure 4.6 représente un modèle d'une application Redis.

```yaml
apiVersion: v1
kind: Template
metadata:
  name: redis-template
  annotations:
    description: "Description"
    iconClass: "icon-redis"
    tags: "database,nosql"
objects:
- apiVersion: v1
  kind: Pod
  metadata:
    name: redis-master
  spec:
    containers:
    - env:
      - name: REDIS_PASSWORD
        value: ${REDIS_PASSWORD}
      image: dockerfile/redis
      name: master
      ports:
      - containerPort: 6379
        protocol: TCP
parameters:
- description: Password used for Redis authentication
  from: '[A-Z0-9]{8}'
  generate: expression
  name: REDIS_PASSWORD
labels:
  redis: master
```

Figure 4.6: "Configuration" : "Un modèle simple (YAML)"
"Un modèle simple (YAML)"

Nous pouvons également utiliser les Template qui existe déjà dans notre plateforme Openshift à travers deux manières :

- Création d'application à partir de modèles à l'aide de la CLI.
- Création d'application à partir de modèles à l'aide de la console Web.

La figure 4.7 représente le catalogue de service qu'on peut l'utiliser dans la console web pour créer directement une application.

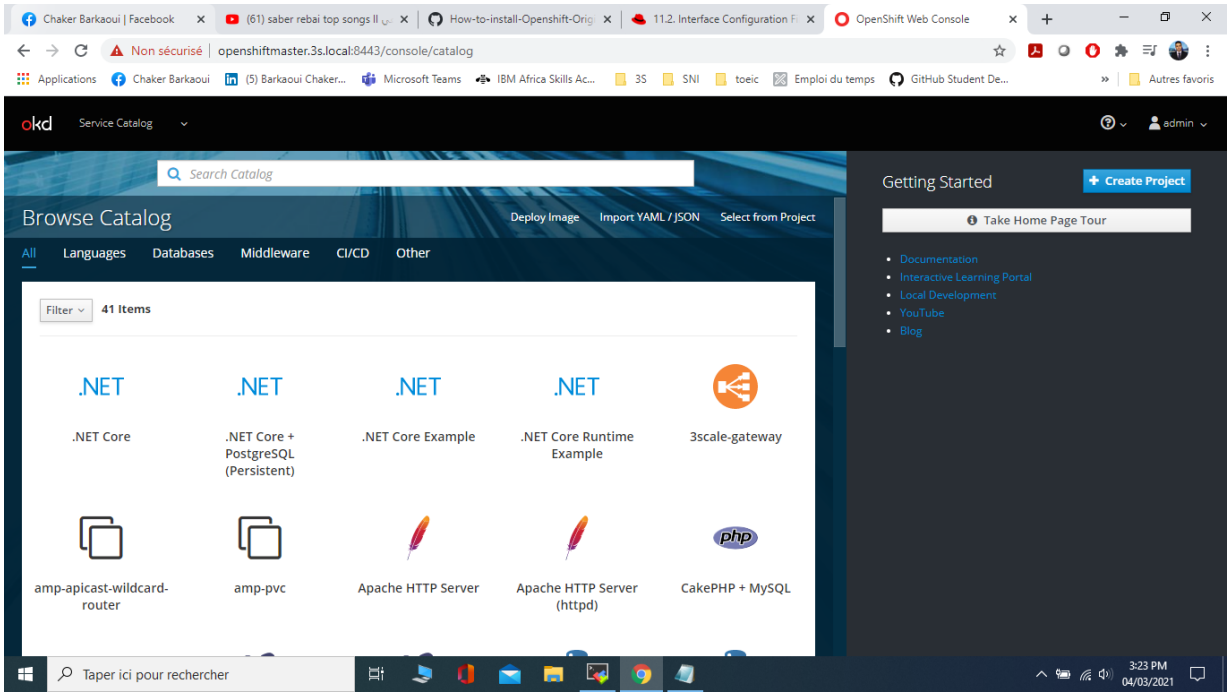


Figure 4.7: "Configuration": "Openshift Console: Service Catalogue"
"Openshift Console: Service Catalogue"

Les différents modèles sont enregistrés dans une base appelée "is". La figure 4.8 représente les différents Template crées dans ce projet.

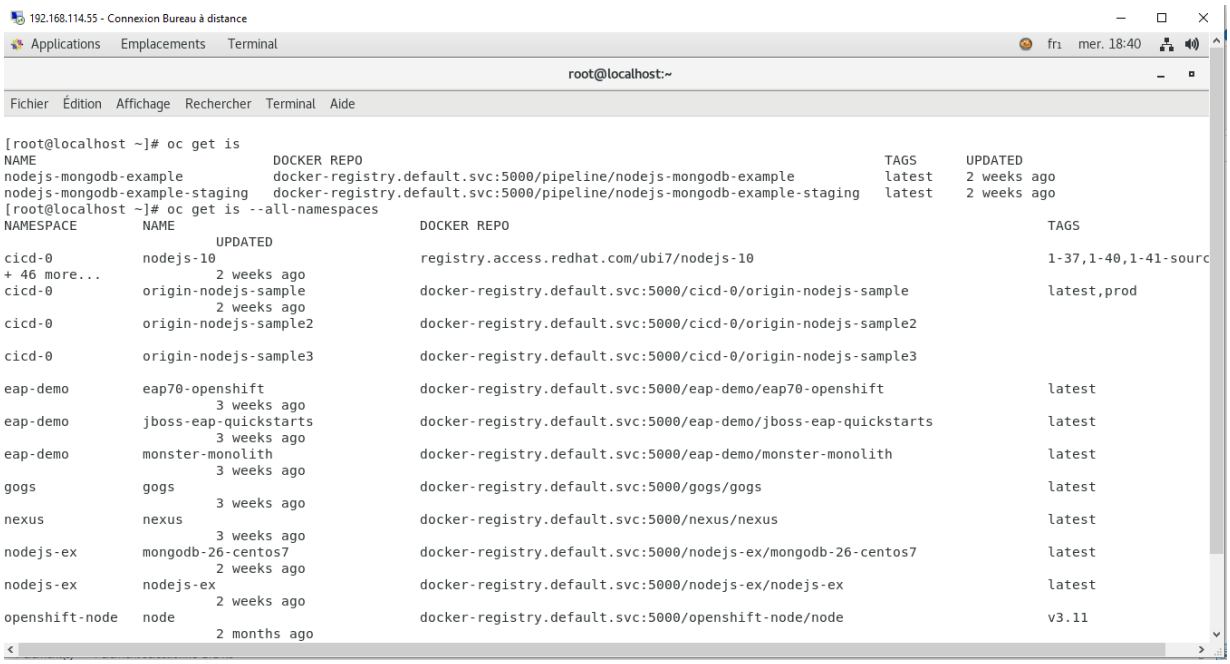


Figure 4.8: "Configuration": "Openshift CLI: Templates"
"Openshift CLI: Templates"

4.3.3 Pipeline d'intégration et de déploiement continu :

Dans cette partie nous allons détailler les différents outils utilisés dans le pipeline et le rôle de chaque outil. Pour la mise en place du pipeline d'intégration continu, nous allons commencer par expliquer la mise en place du serveur Git suivie de celle du serveur Jenkins et SonarQube pour conclure avec la mise en place du serveur Nexus.

Installation Serveur Git ("Gogs") à base de conteneurs :
Nous voulons réutiliser notre serveur Gogs pour certaines situations, nous devons utiliser la version persistante du modèle. Tout d'abord, nous devons avoir 2 volumes persistants.
La figure 4.9 explique comment nous pouvons créer des volumes persistants.

```
$ cat <<-EOF | oc create -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  nfs:
    path: /var/exports/openshift/pv0001
    server: 192.168.56.11
  persistentVolumeReclaimPolicy: Recycle
EOF

$ cat <<-EOF | oc create -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0002
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  nfs:
    path: /var/exports/openshift/pv0002
    server: 192.168.56.11
  persistentVolumeReclaimPolicy: Recycle
EOF
```

Figure 4.9: "Pipeline d'intégration et de déploiement continu : Création des volumes persistants"

"Création des volumes persistants"

Nous devons ensuite créer un nouveau projet à partir duquel nous allons servir les Gogs et nous allons répéter la même étape pour tous les autres outils.

oc new-project gogs

Et enfin, le déploiement d'un Pod de gestionnaire de référentiel Gogs git auto-hébergé à partir du modèle fourni. Cela créera un PersistentVolumeClaim dans lequel stocker le code source et la configuration.

oc project gogs

oc new-app -f https://raw.githubusercontent.com/eformat/gogs-openshift-docker/master/persistent.yaml

La figure 4.10 représente les pods installés de Gogs:

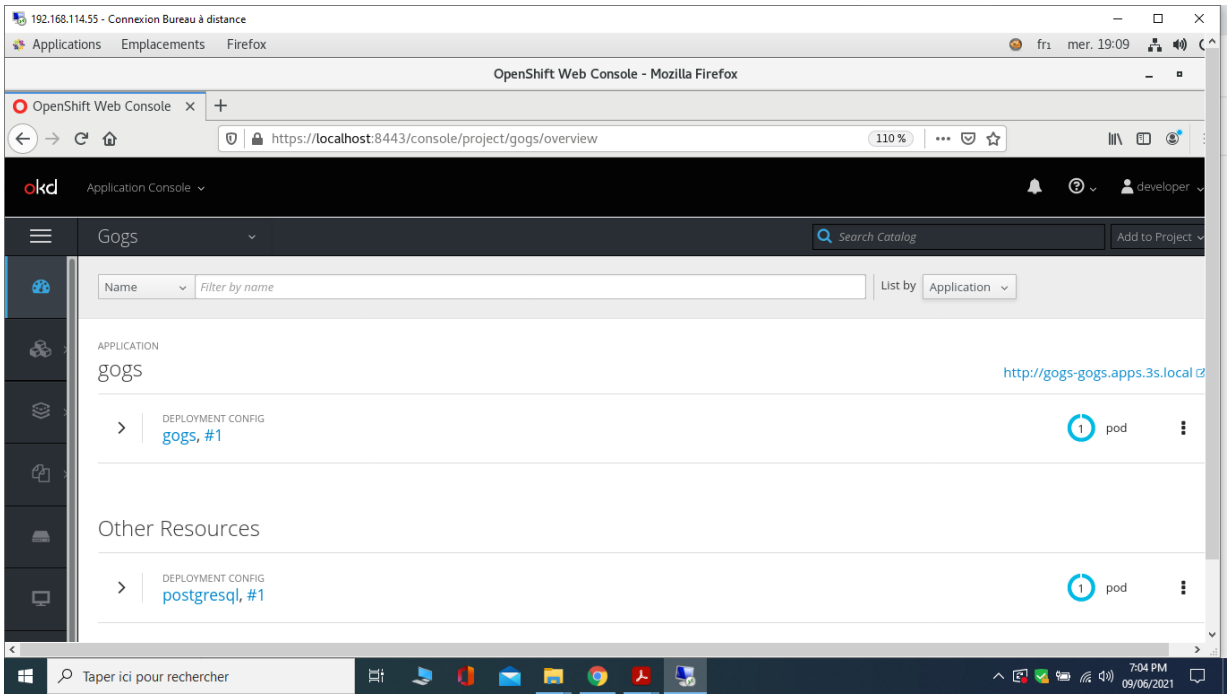


Figure 4.10: "Pipeline d'intégration et de déploiement continu : Gogs Pods"
"Gogs Pods"

Et enfin l'interface suivant est l'application Gogs déployé sur Openshift.

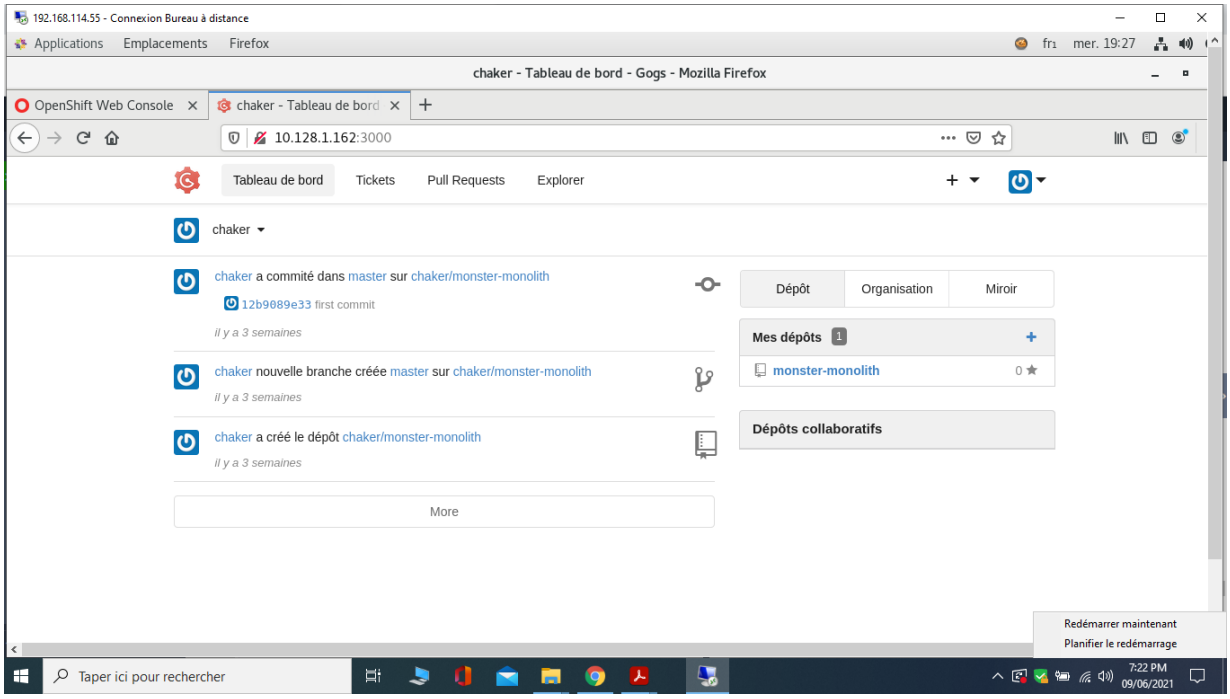


Figure 4.11: "Pipeline d'intégration et de déploiement continu : Gogs Web Console" "Gogs Web Console"

Installation Serveur Jenkins à base de conteneur : Nous allons déployer un Pod Jenkins à partir du modèle fourni. Cela créera un PersistentVolumeClaim pour stocker les Jenkins principaux et la configuration du travail et l'historique de construction.
oc project jenkins
oc new-app -f https://raw.githubusercontent.com/eformat/openshift-jenkins/master/jenk

La figure 4.12 représente les pods installés de Jenkins :

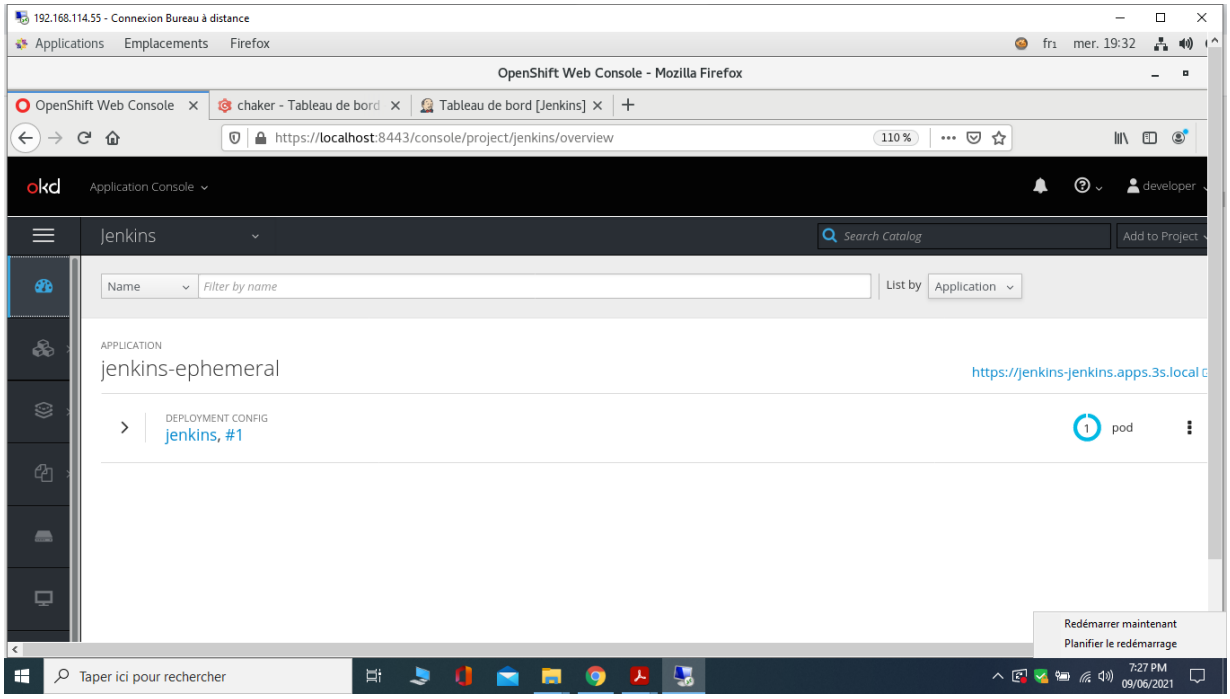


Figure 4.12: "Pipeline d'intégration et de déploiement continu : Jenkins Pods"
"Jenkins Pods"

Et l'interface graphique de Jenkins est la suivante:

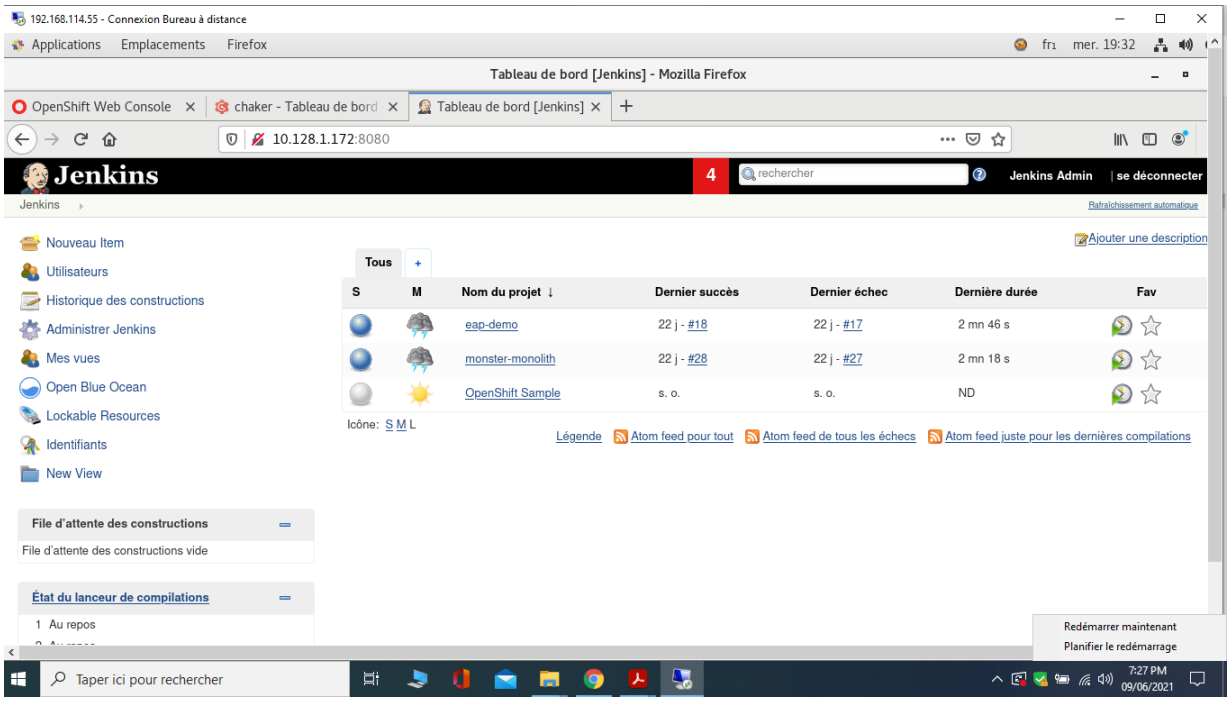


Figure 4.13: "Pipeline d'intégration et de déploiement continu : Jenkins Web Console"
"Jenkins Web Console"

Installation Serveur Nexus3 à base de conteneur :
De la même manière, nous allons déployer un Pod Nexus3 à partir du modèle fourni. Cela créera un PersistentVolumeClaim dans lequel stocker les images et les ressources mises en cache.
oc project nexus
oc new-app -f https://raw.githubusercontent.com/eformat/openshift-nexus/master/nexus
La figure 4.14 représente les pods installés de Nexus3 :

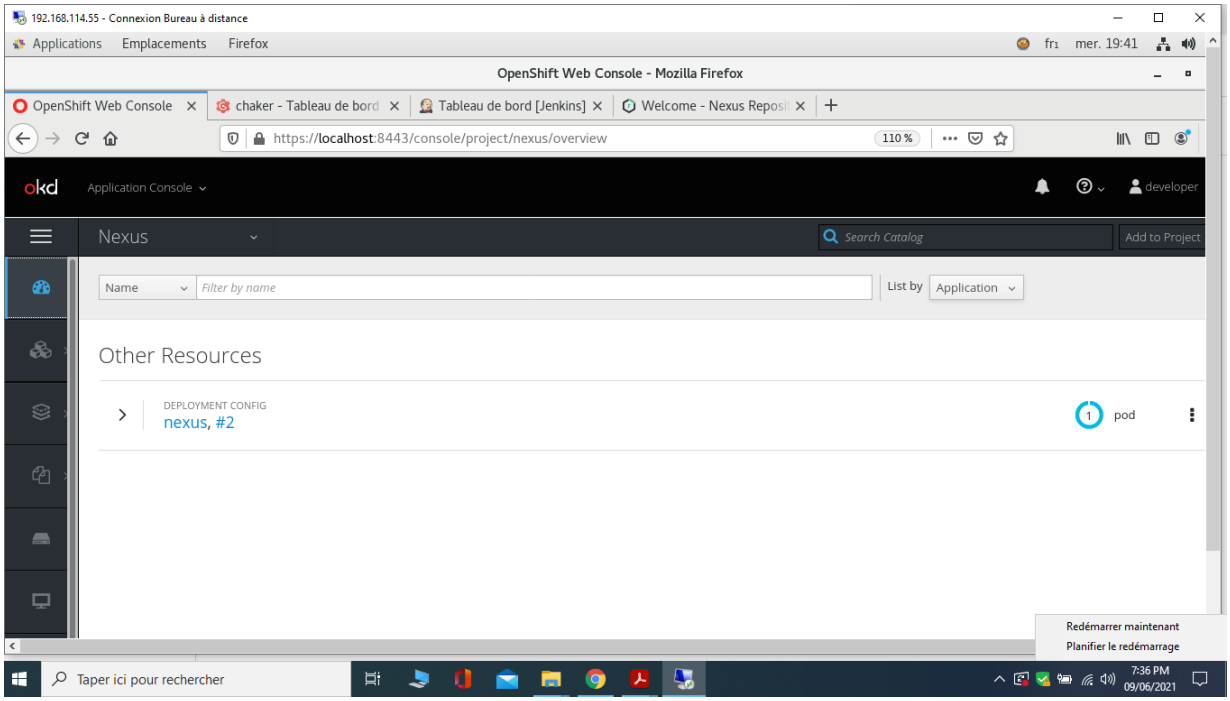


Figure 4.14: "Pipeline d'intégration et de déploiement continu : Nexus3 Pod"
"Nexus3 Pod"

Et nous pouvons de cette façon récupérer directement l'URL de l'interface web de Nexus3 présenté par la figure suivante :

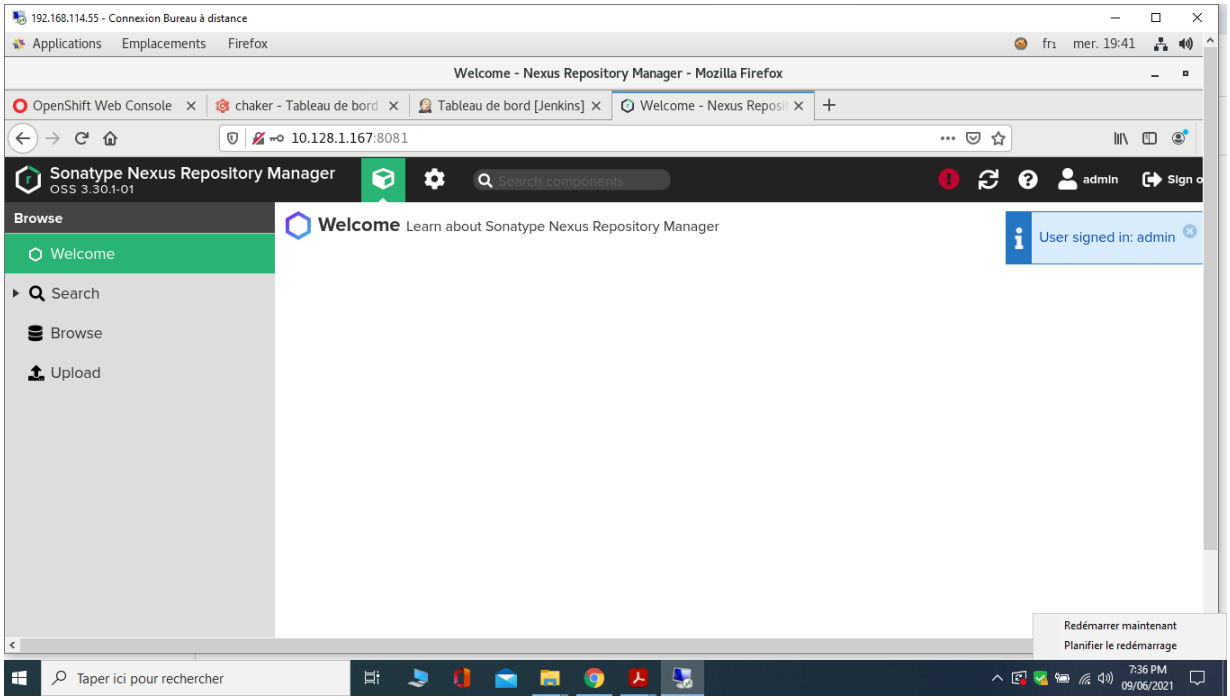


Figure 4.15: "Pipeline d'intégration et de déploiement continu : Nexus3 Web Console" "Nexus3 Web Console"

Installation Serveur SonarQube à base de conteneur :
Nous avons déployé un Pod SonarQube à partir du modèle fourni. Cela créera un PersistentVolumeClaim dans lequel stocker les données mises en cache et les résultats de l'analyse du code.
oc new-project sonarqube –display-name="SonarQube" –description="SonarQube"
oc new-app -f https://raw.githubusercontent.com/eformat/openshift-sonarqube/master/s

La figure 4.16 représente les pods installés de SonarQube :

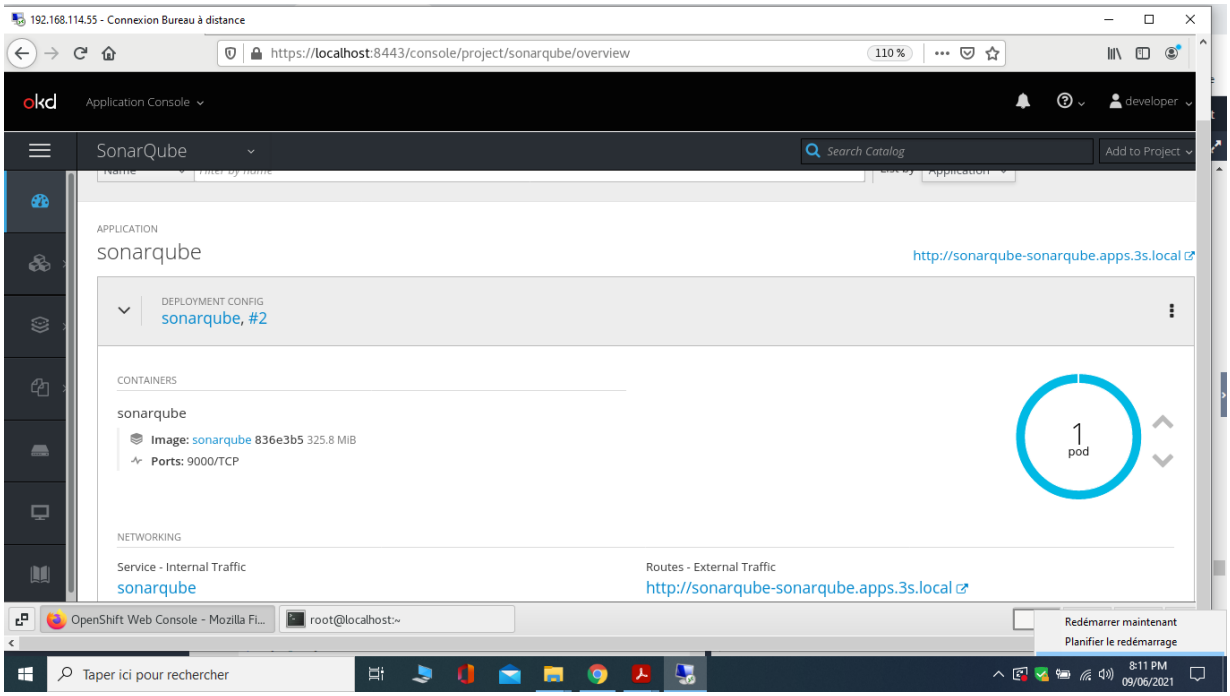


Figure 4.16: "Pipeline d'intégration et de déploiement continu : SonarQube Pod" "SonarQube Pod"

Et nous pouvons de cette façon récupérer directement l'URL de l'interface web de Nexus3 présenté par la figure suivante :

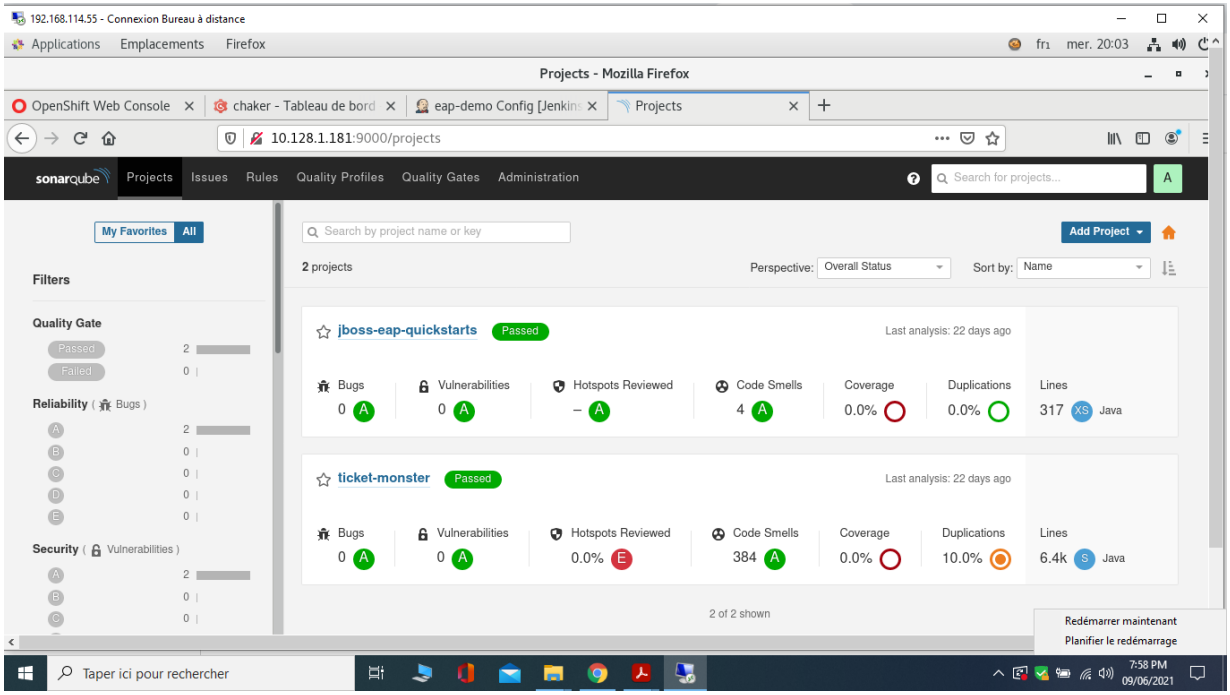


Figure 4.17: "Pipeline d'intégration et de déploiement continu : SonarQube Web Console" "SonarQube Web Console"

Nous avons intégré le pipeline d'intégration continu avec celui du déploiement continu. Ainsi les imprimes écran qui suivent, illustrent l'enchaînement du processus d'intégration et déploiement continu.
La figure 4.18 comporte un imprime écran sur un job Jenkins contenant un script pipeline définissant l'analyse de code après son importation, le build et le deploy, le déploiement des artéfacts sur Nexus et finalement la vérification du déploiement.

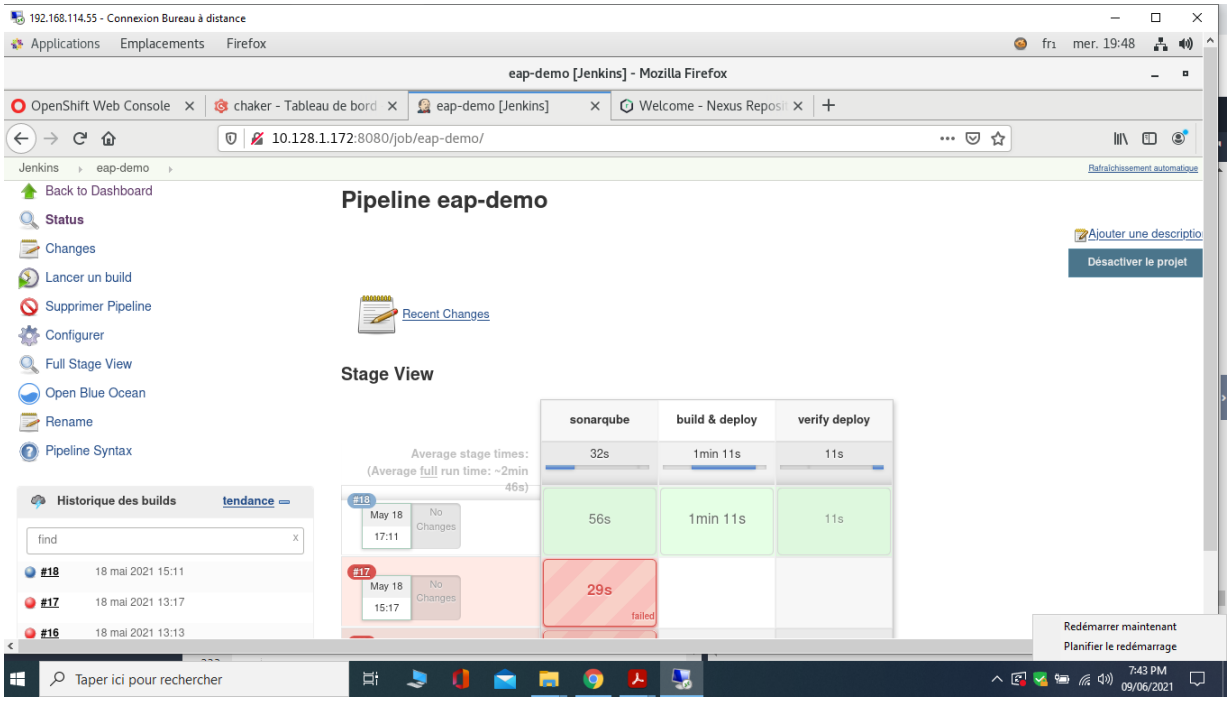


Figure 4.18: "Pipeline d'intégration et de déploiement continu : Jenkins Job"
"Jenkins Job"

La figure 4.19 présente le script du pipeline.

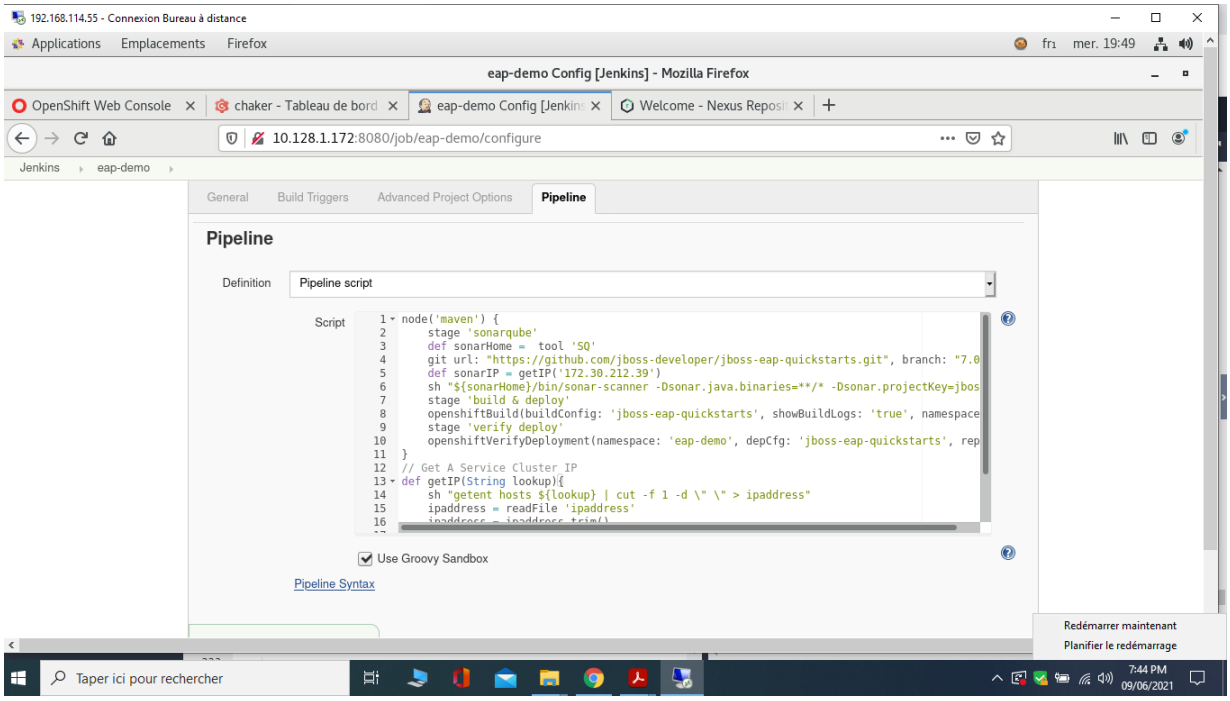


Figure 4.19: "Pipeline d'intégration et de déploiement continu : Pipeline Script" "Pipeline Script"

La figure 4.20 présenta le résultat d'analyse de code sur SonarQube

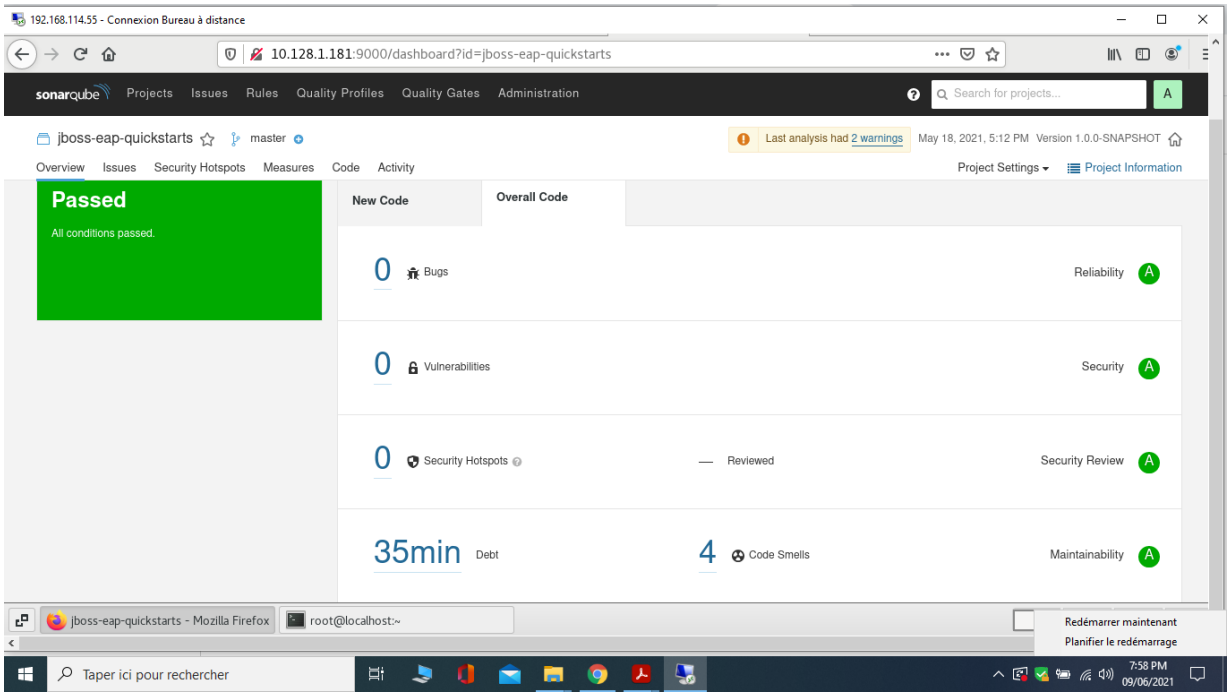


Figure 4.20: "Pipeline d'intégration et de déploiement continu : Application Code Analyses" "Application Code Analyses"

La figure 4.21 montre le build et l'image déclenchés par Jenkins et s'est déroulé sur Openshift.

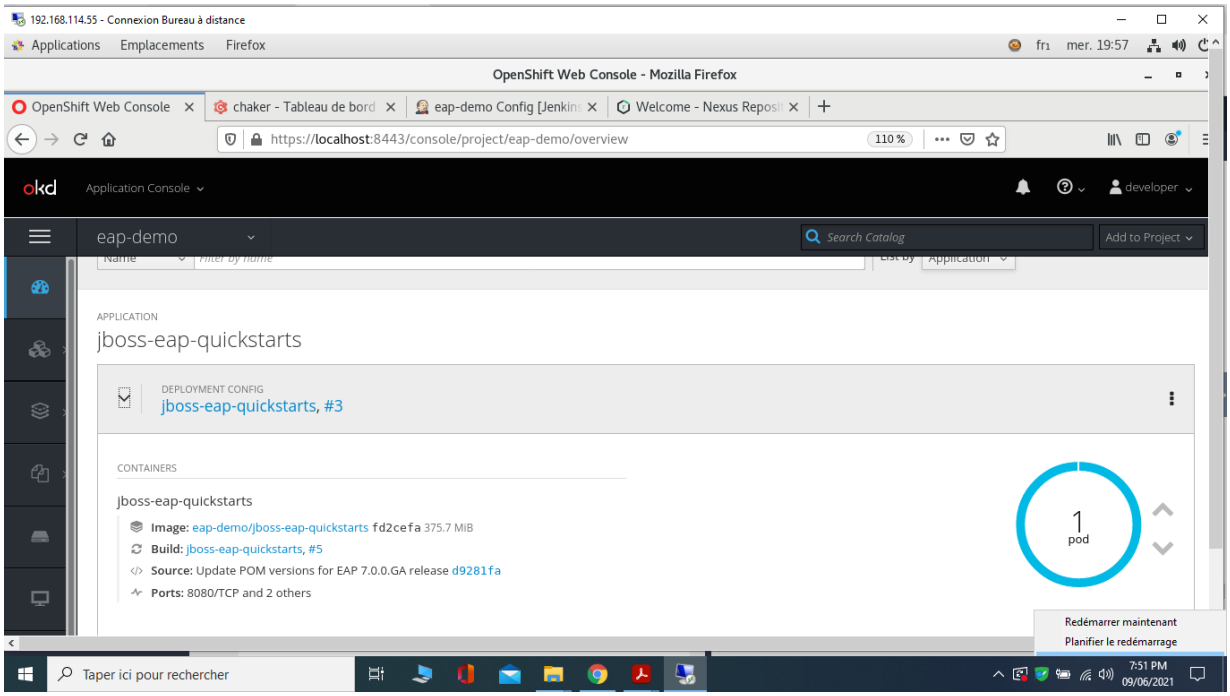


Figure 4.21: "Pipeline d'intégration et de déploiement continu : Application Image - Build" "Application Image - Build"

La figure 4.22 affiche l'interface de notre application JBoss Quick Start :

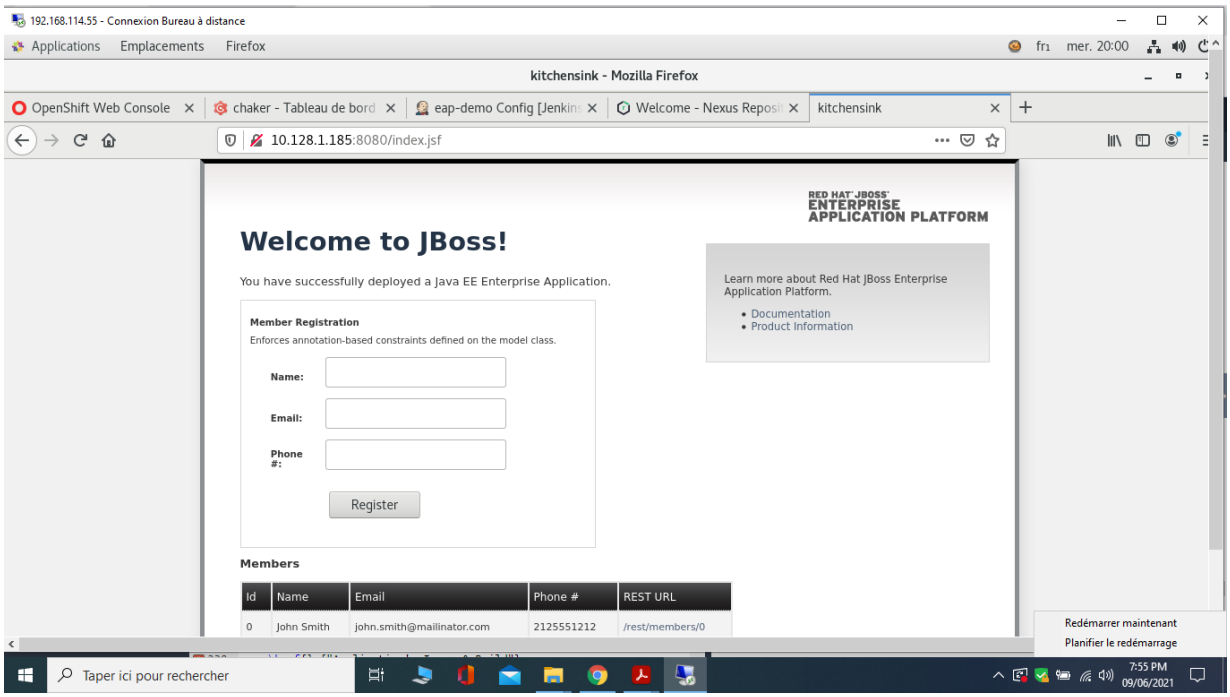


Figure 4.22: "Pipeline d'intégration et de déploiement continu : Application Web Interface" "Application Web Interface"

4.4 Supervision :

Le module de supervision permet de récupérer des métriques sur les composantes CPU, RAM, et pression entrée/sortie sur le réseau des pods du cluster. Il permet aussi de surveiller l'état des conteneurs.
La figure 4.23, montre l'usage CPU et RAM ainsi que la pression des entrées/sortie sur les conteneurs visualisés à l'aide de Grafana : Une plateforme qui existe déjà dans la version Openshift 3.11.

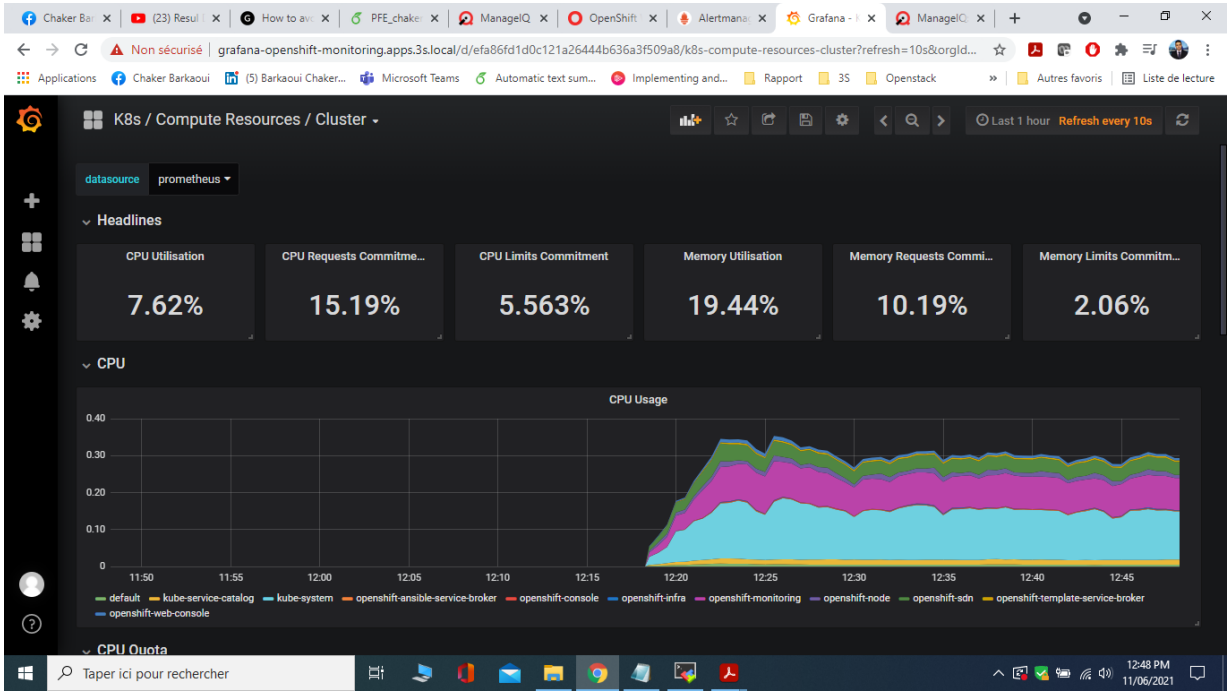


Figure 4.23: "Supervision :""Grafana Web Console"
"Grafana Web Console"

La figure 4.24 montre les alertes du cluster Openshift visualisé à l'aide d'Alert Manager.

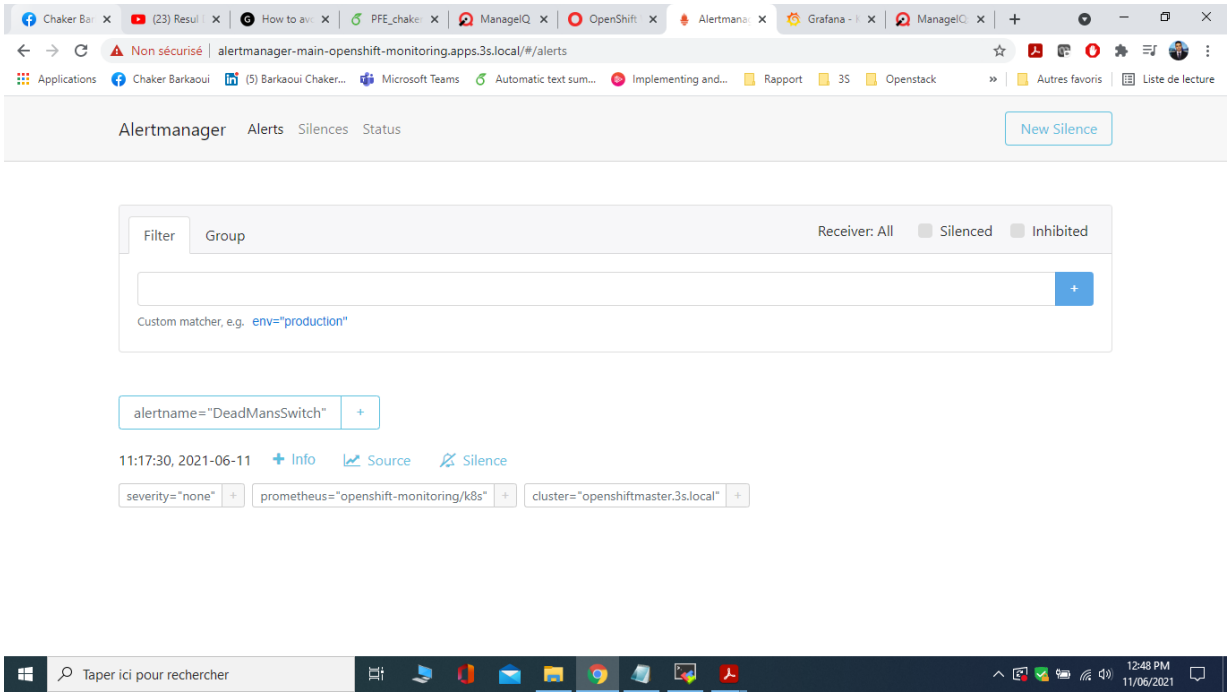


Figure 4.24: "Supervision :""Alert Manager Web Console"
"Alert Manager Web Console"

4.5 Intégration d'Openshift avec ManageIQ :

Dans cette partie nous allons utiliser le service Prometheus l'équivalent de Hawkular dans Openshift 3.11.
ManageIQ est une plate-forme de gestion open source pour l'informatique hybride. Il peut gérer des environnements petits et grands et prend en charge plusieurs technologies telles que les machines virtuelles, les clouds publics et les conteneurs.

Avec ManageIQ nous pourrons : [9]

- Découvre en permanence l'état le plus récent de notre environnement.
- Implémenter le libre-service pour nos utilisateurs finaux.
- Faire respecter la conformité dans l'ensemble de l'environnement.
- Optimiser les performances et l'utilisation de notre environnement.

La figure 4.25 montre les tokens d'Openshift et l'URL de service Prometheus nécessaire pour la configuration des métriques.

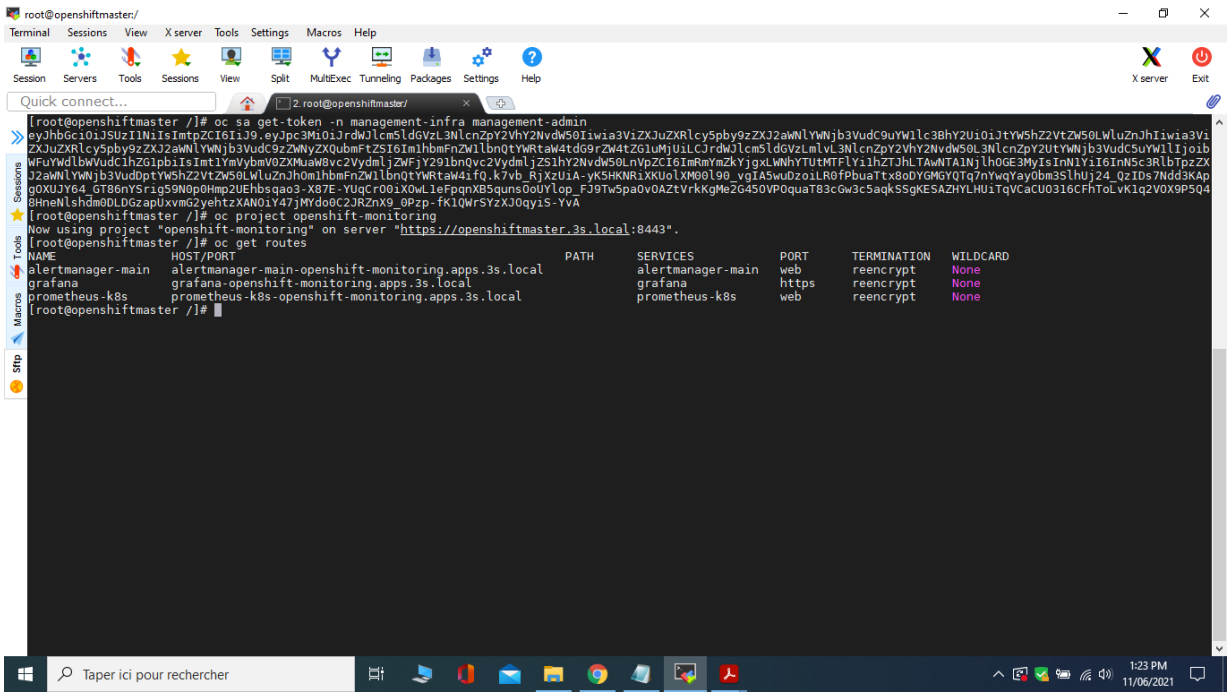


Figure 4.25: "Intégration d'openshift avec ManageIQ :""Openshift Provider Token" "Openshift Provider Token"

Les figures 4.26 et 4.27 présente les configurations nécessaires pour ManageIQ:

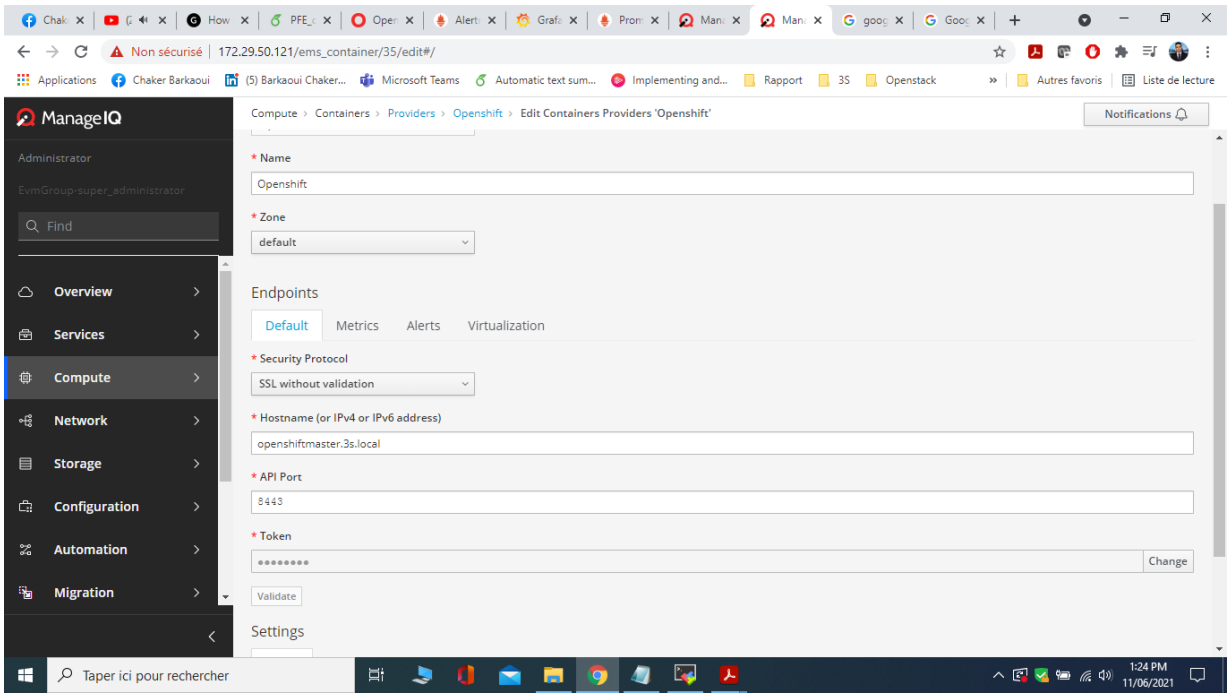


Figure 4.26: "Intégration d'Openshift avec ManageIQ: ManageIQ Configurations 1" "ManageIQ Configurations 1"

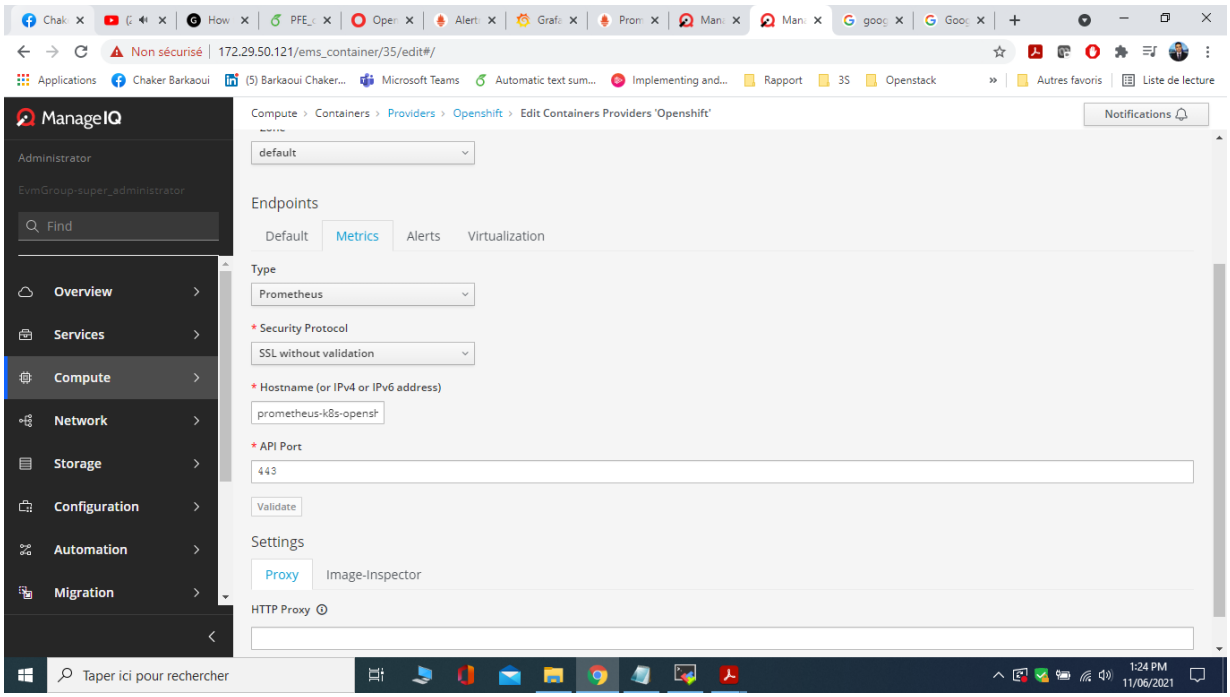


Figure 4.27: "Intégration d'Openshift avec ManageIQ: ManageIQ Configurations 2 "ManageIQ Configurations 2"

Les figures suivantes représentent les différentes informations sur le Cluster Openshift avec les métriques d'utilisation.

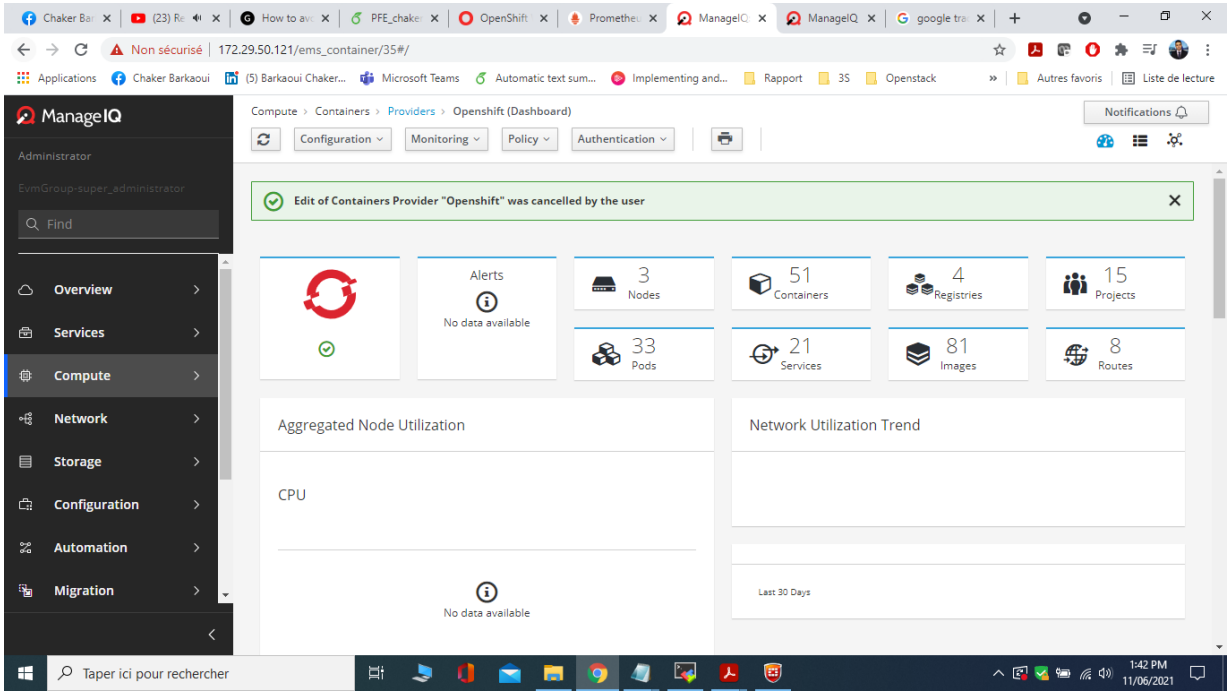


Figure 4.28: "Intégration d'Openshift avec ManageIQ: ManageIQ Container Console" "ManageIQ Container Console"

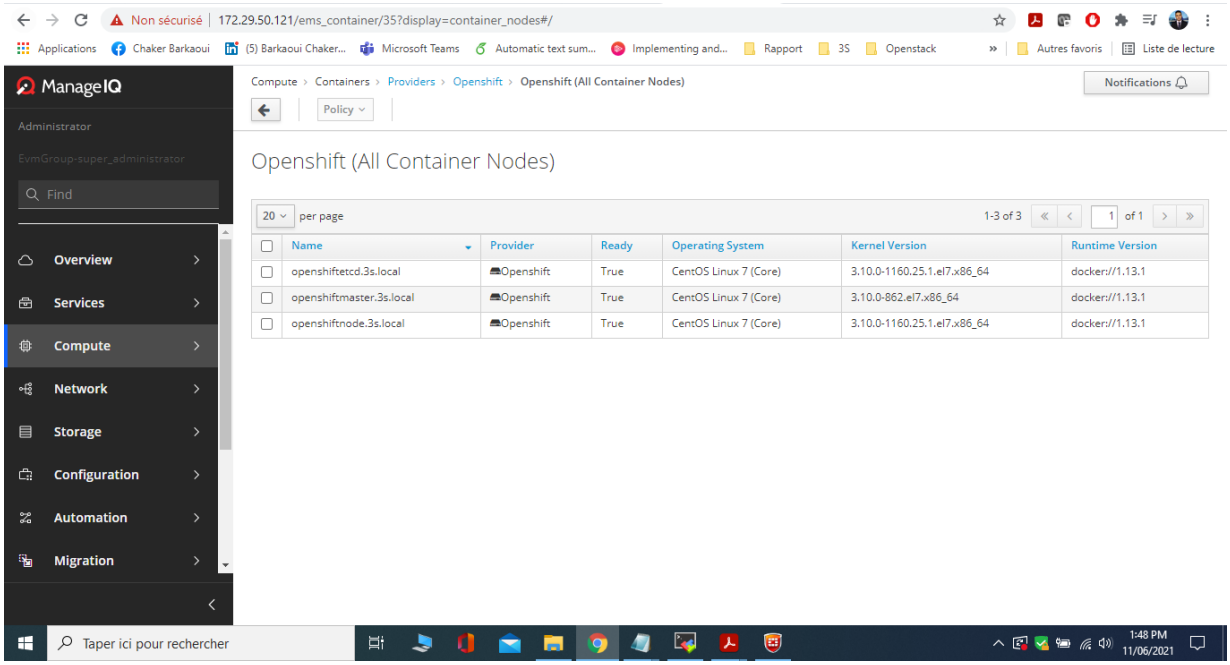


Figure 4.29: "Intégration d'Openshift avec ManageIQ: Openshift Nodes"
"Openshift Nodes"

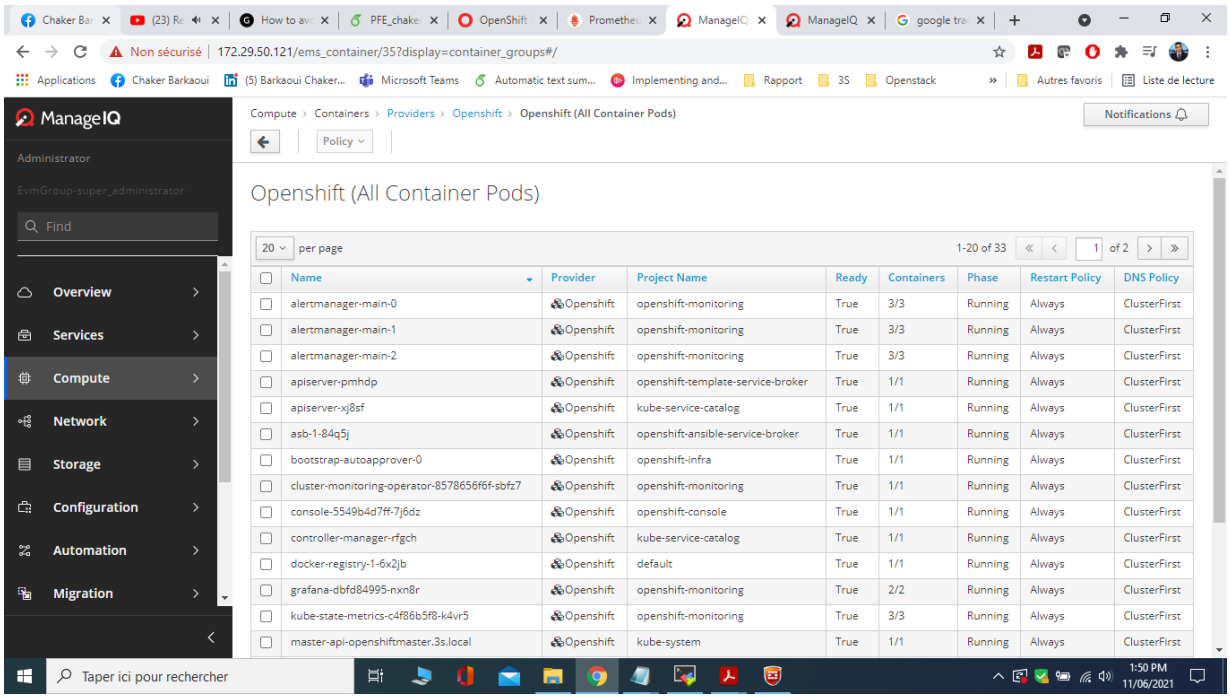


Figure 4.30: "Intégration d'Openshift avec ManageIQ: Openshift Pods"
"Openshift Pods"

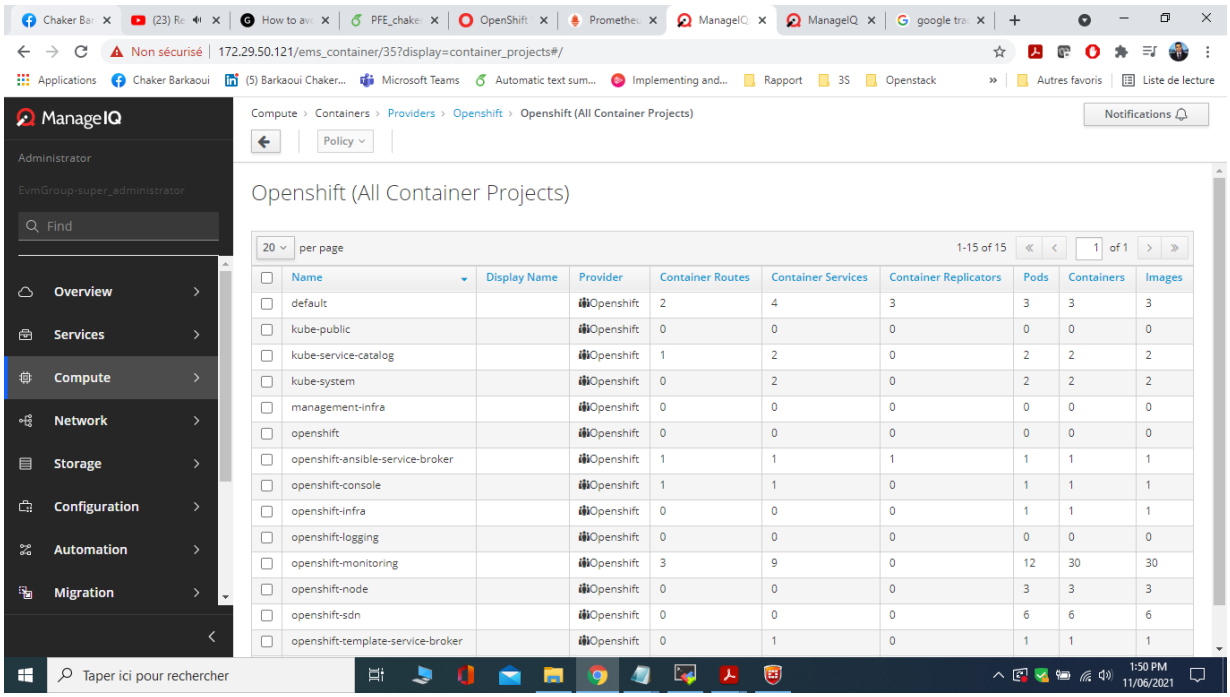


Figure 4.31: "Intégration d'Openshift avec ManageIQ: Openshift Projects" "Openshift Projects"

4.6 Conclusion

Ce dernier chapitre porte sur la réalisation de notre travail, il nous a permis de présenter l'environnement matériel et logiciel de notre projet, il nous a aussi permis de présenter les différentes technologies que nous avons utilisé pour l'aboutissement de ce travail, et enfin, nous avons exposé quelques interfaces graphiques relatives à ce projet.

Conclusion générale

Ce travail s'inscrit dans le cadre de projet de fin d'études, il a été réalisé au sein de la société Standard Sharing Software (3S) et consiste en la mise en place d'une plateforme Cloud PaaS OpenShift et l'automatisation du cycle de vie des services via l'approche CI/CD. J'avais décomposé ce travail sur cinq modules, un module de d'intégration continue, un module de déploiement continu, un module de configuration, un module de monitoring et un module de pré-configuration.
Pour aboutir à ce travail, j'avais d'abord commencé par présenter le cadre général de mon projet qui inclut la présentation de l'organisme d'accueil, l'étude de l'existant et constatations, objectifs et contributions, et enfin l'application de la méthodologie adoptée. Ensuite, j'ai passée à l'étude préalable qui a permis de définir les concepts clés pour notre projet, de comparer les différentes solutions techniques qui s'offrent à moi pour enfin faire synthèse de mes choix techniques.
Puis, je me trouve intéressé à l'analyse et spécification, un chapitre qui a fait l'objet sur l'identification des acteurs, l'analyse des besoins et leurs représentations et clôturer par décortiquer son aspect dynamique. Enfin, j'avais entamé la partie réalisation qui a porté sur la présentation de l'environnement de travail qu'il soit matériel ou logiciel, la présentation des différentes technologies utilisées pour réaliser ce travail, et pour finir l'exposition de quelques interfaces graphiques relatives à mon projet.
Grâce à cette expérience, j'avais pu, d'une part, découvrir de nouvelles technologies telle que docker, Kubernetes et OpenShift et d'autres part acquérir de nouvelles compétences telle que l'administration système et le déploiement d'un cluster.
Par ailleurs, ce projet m'a aussi été bénéfique de point de vue personnel, il m'a permis d'appréhender le travail au sein d'une grande société et dans une hiérarchie professionnelle. Le travail de groupe, la gestion de projet de façon méthodique et la répartition des tâches et des efforts sont des qualités que j'avais acquis grâce à ce stage. Je tiens à présent à souligner quelques extensions intéressantes pour améliorer mon projet. J'estime que l'ajout que l'intégration d'Openshift avec OpenStack et CloudFoundry peut améliorer les performances du projet. Aussi, l'utilisation de la version avec licence de Redhat Openshift va aider mieux pour arriver aux objectifs.
Pour conclure, je souhaite que ce travail apporte la satisfaction aux responsables de l'entreprise hôte « Standard Sharing Software (3S) » et aux membres du Jury.

Bibliography

[1] Groupe 3S. Standard sharing software (3s). *"https://www.3s.com.tn/","Accès le 27/05/2021"*.

[2] Avi. Docker vs virtual machine - comprendre les différences. *"https://geekflare.com/fr/docker-vs-virtual-machine/ ","Accès le 26/05/2021"*.

[3] William Caban. *Architecting and Operating OpenShift Clusters*, volume 1. William Caban, 2019.

[4] Ionos Community. Git ou svn : comparaison d'outils de versionning. *"https://www.ionos.fr/digitalguide/sites-internet/developpement-web/git-ou-svn-comparaison-doutils-de-versionning/ ","Accès le 26/05/2021"*, Mars 12, 2020.

[5] Ori Dar. Maven repository manager: Nexus vs. artifactory. *"https://dzone.com/articles/maven-repository-manager-nexus ","Accès le 26/05/2021"*, Dec 14, 2019.

[6] Ken Schwaber et Jeff Sutherland. *Le Guide Scrum*, volume 1. Ken Schwaber et Jeff Sutherland, 2017.

[7] Rahul Jain. Jenkins vs travis vs bamboo vs teamcity: Clash of the titans. *"https://www.lambdatest.com/blog/jenkins-vs-travis-vs-bamboo-vs-teamcity/ ","Accès le 26/05/2021"*, October 22, 2020.

[8] Communauté le mag IT. Cluster. *"http://www. lemagit.fr/definition/Cluster","Accès le 26/05/2021"*, jui.2015.

[9] ManageIQ. Get started. *"https://www.manageiq.org/docs/get-started/"*.

[10] Frédéric MILLOGO. *Mise en place d'une application webmapping de géolocalisation des points d'intérêt de la ville de Ouagadougou.* "Accès le 26/05/2021".

[11] Communauté openvswitch. Compare openshift vs cloud foundry. *"https://comparisons.financesonline.com/openshift-vs-cloud-foundry ","Accès le 26/05/2021"*, Dec 2016.

[12] Juliette Palmer. Devops : qu'est-ce que l'intégration continue (ci) ? premiers pas vers le ci/cd. *"https://www.padok.fr/blog/devops-integration-continue","Mise à jour 13/05/2019","Accès le 26/05/2021"*, 2019.

[13] Clémence Plu. Devops : Qu'est-ce que le déploiement continu et la livraison continue (cd). *"https://www.padok.fr/blog/devops-deploiement-continu","Mise à jour 14/01/2020","Accès le 26/05/2021"*, 2020.

[14] Inc. Red Hat. Docker, qu'est-ce que c'est ? *"https://www.redhat.com/fr/topics/containers/what-is-docker","Accès le 26/05/2021"*.

[15] M. Rouse. What is kubernetes. *" http://www. lemagit.fr/definition/Kubernetes. ","Accès le 26/05/2021"*, Mars. 2016.

[16] Communauté TrusTradius. Codacy vs sonarqube. *"https://www.trustradius.com/compare-products/codacy-vs-sonarqube ","Accès le 26/05/2021"*.

[17] Veritis. Chef vs puppet vs ansible – comparison of devops configuration management tools. *"https://www.veritis.com/blog/chef-vs-puppet-vs-ansible-comparison-of-devops-management-tools/ ","Accès le 26/05/2021"*.

Annexes

Inventory File: All-in-one

```
# Create an OSEv3 group that contains the masters and nodes groups
[OSEv3:children]
masters
nodes
etcd

# Set variables common for all OSEv3 hosts
[OSEv3:vars]
# SSH user, this user should allow ssh based auth without requiring a password
ansible_ssh_user=root

openshift_master_default_subdomain=apps.3s.local
openshift_public_hostname=localhost

# If ansible_ssh_user is not root, ansible_become must be set to true
ansible_become=true

openshift_deployment_type=origin
deployment_type=origin
openshift_portal_net=172.30.0.0/16

openshift_disable_check=disk_availability,memory_availability,docker_storage
openshift_node_groups=[{'name': 'node-config-all-in-one', 'labels': ['node-role.kubernetes.io/master=true', 'node-role.kubernetes.io/infra=true', 'node-role.kubernetes.io/compute=true']}]

# uncomment the following to enable htpasswd authentication; defaults to DenyAllPasswordIdentityProvider
#openshift_master_identity_providers=[{'name': 'htpasswd_auth', 'login': 'true', 'challenge': 'true', 'kind': 'HTPasswdPasswordIdentityProvider'}]

# host group for masters
[masters]
localhost ansible_connection=local

# host group for etcd
[etcd]
localhost ansible_connection=local

# host group for nodes
[nodes]
# openshift_node_group_name should refer to a dictionary with matching key of name in list openshift_node_groups.
localhost ansible_connection=local openshift_node_group_name="node-config-all-in-one"
```

Figure 4.32: "Inventory File All-in-one:"

Inventory File : Master, Node, Etcd :

```
[OSEv3:children]
masters
nodes
etcd

# Set variables common for all OSEv3 hosts
[OSEv3:vars]
# SSH user, this user should allow ssh based auth without requiring a password
ansible_ssh_user=root

# If ansible_ssh_user is not root, ansible_become must be set to true
#ansible_become=true

openshift_deployment_type=origin


#openshift_additional_repos=[{'id': 'openshift-origin-copr', 'name': 'OpenShift Origin COPR', 'baseurl': 'https://copr-be.cloud.fedoraproject.org/results/maxamillion/origin-next/epel-7-$basearch/', 'enabled': 1, 'gpgcheck':

openshift_disable_check="disk_availability,memory_availability,docker_image_availability"


# define default sub-domain for Master node
openshift_master_default_subdomain=apps.3s.local
openshift_master_cluster_hostname=openshiftmaster.3s.local
openshift_public_hostname=openshiftmaster.3s.local
openshift_master_bootstrap_auto_approve=true
os_firewall_use_firewalld=True


# allow unencrypted connection within cluster
openshift_docker_insecure_registries=172.30.0.0/16

# uncomment the following to enable htpasswd authentication; defaults to DenyAllPasswordIdentityProvider
#openshift_master_identity_providers=[{'name': 'htpasswd_auth', 'login': 'true', 'challenge': 'true', 'kind': 'HTPasswdPasswordIdentityProvider'}]

# host group for masters
[masters]
openshiftmaster.3s.local openshift_schedulable=true

# host group for etcd
[etcd]
openshiftetcd.3s.local

# host group for nodes, includes region info
[nodes]
openshiftnode.3s.local openshift_node_group_name='node-config-compute'
openshiftmaster.3s.local  openshift_node_group_name='node-config-master'
openshiftetcd.3s.local openshift_node_group_name='node-config-infra'
```

Figure 4.33: "Inventory File : Master, Node, Etcd :"

Partie Nexus :

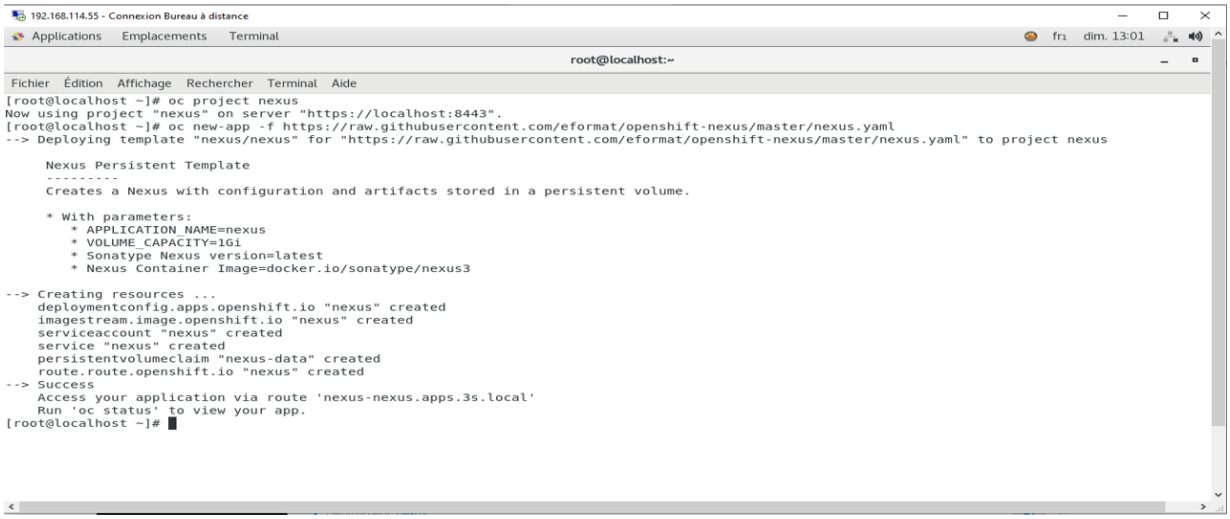


Figure 4.34: "Déploiement de Nexus3"

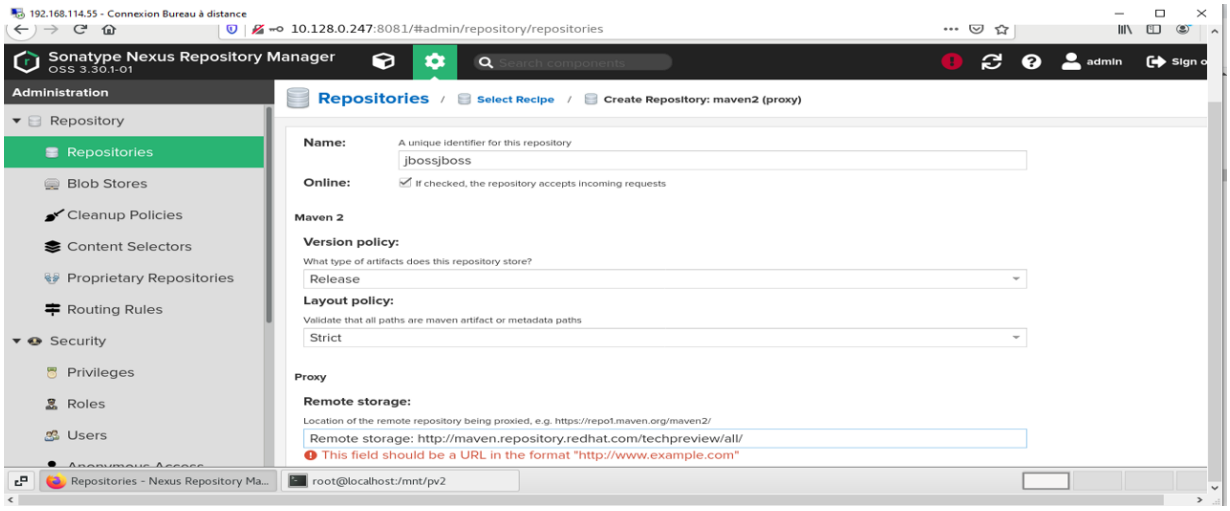


Figure 4.35: "Add in the JBoss Repositories : jboss et jboss-ce"

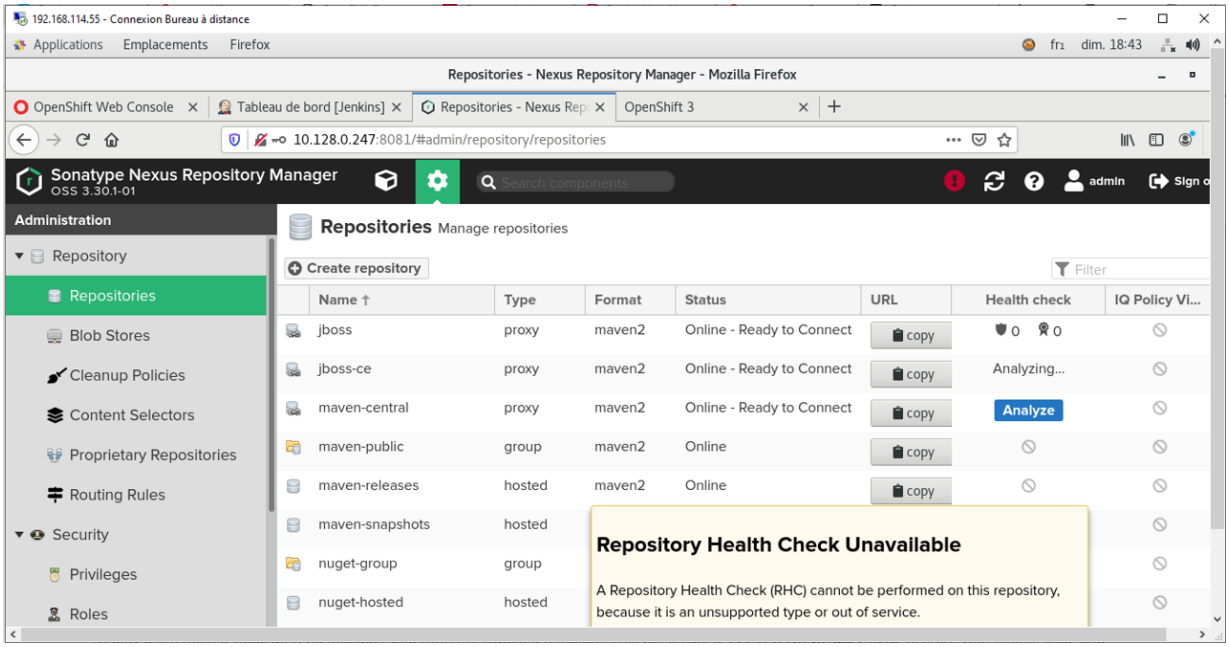


Figure 4.36: "Maven Public Repository"

Partie Gogs :

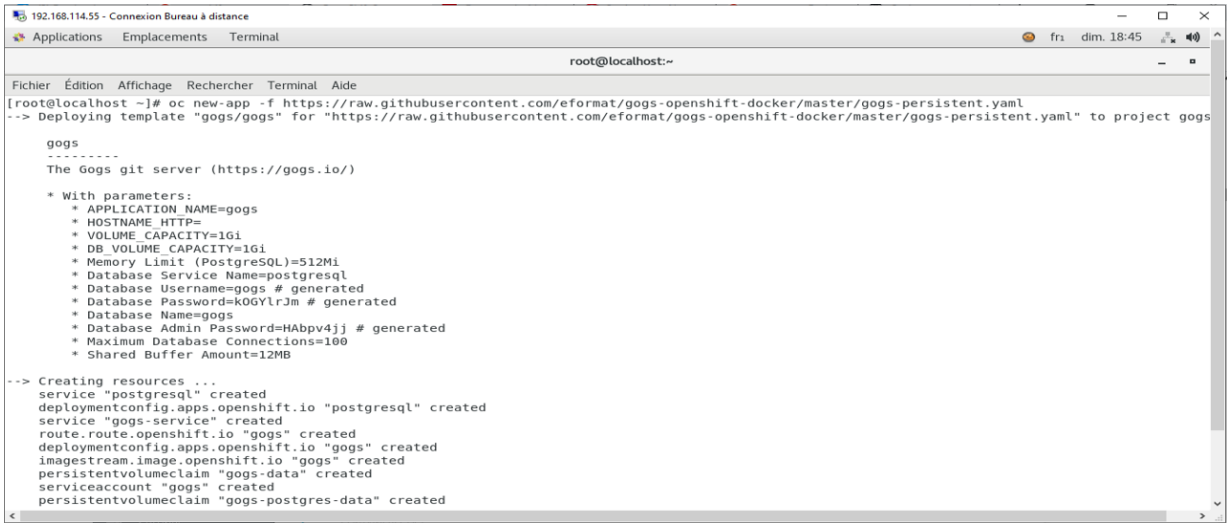


Figure 4.37: "Déploiement de Gogs"

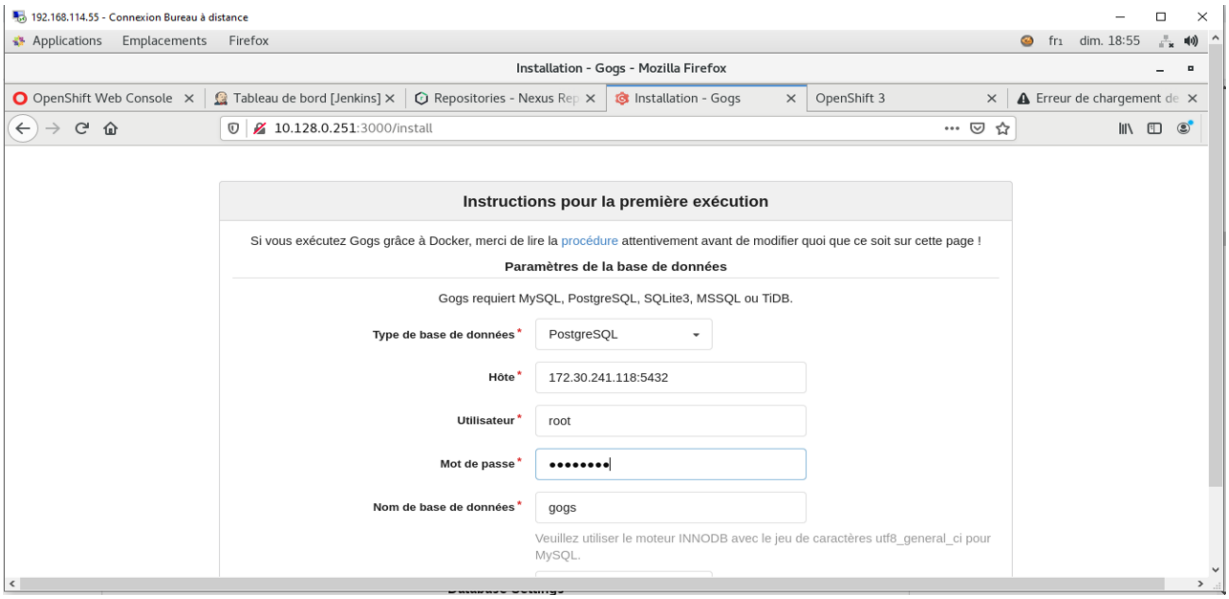


Figure 4.38: "Gogs installation 1"

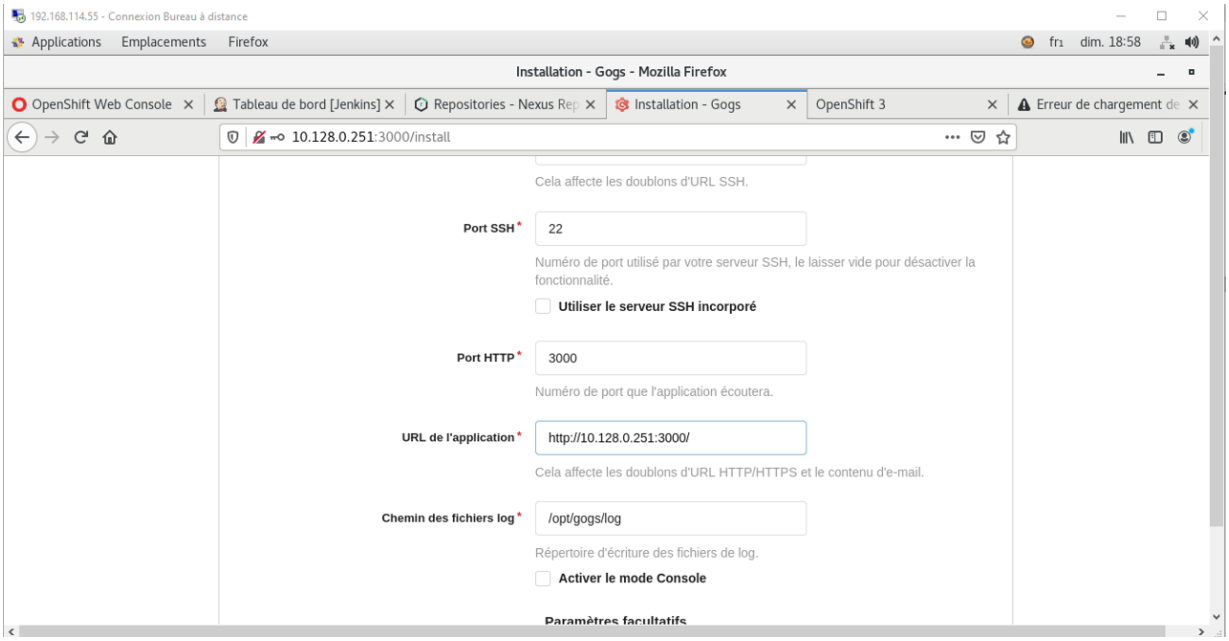


Figure 4.39: "Gogs installation 2"

Partie Frontend NodeJS Pipeline :

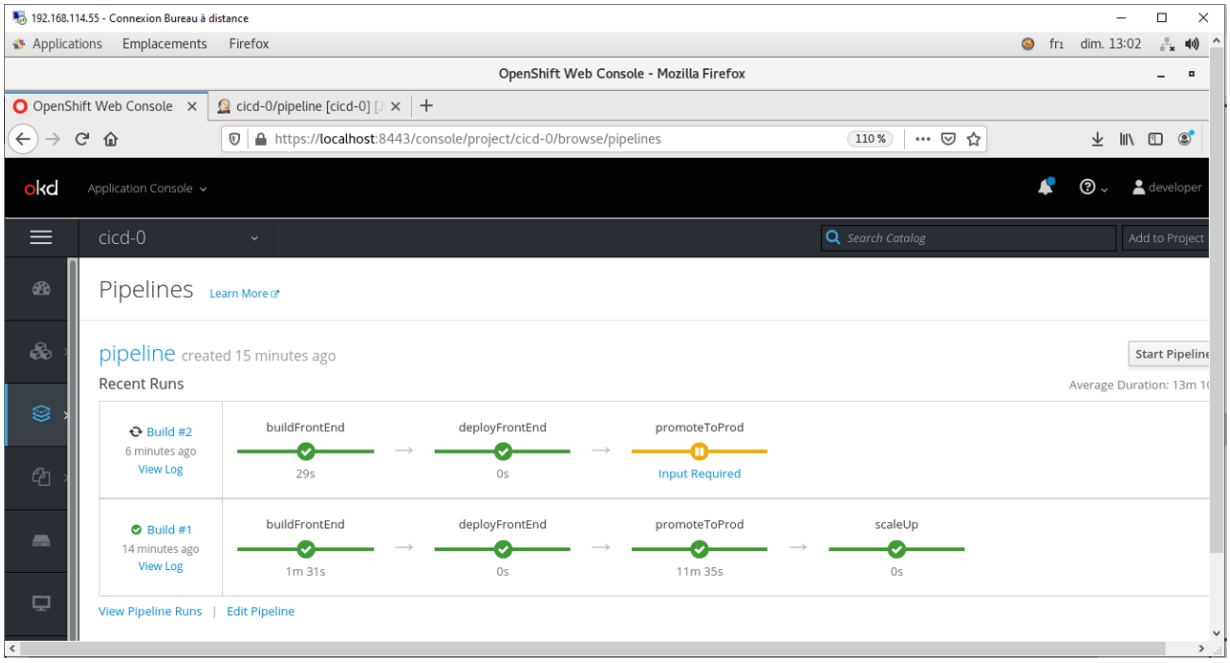


Figure 4.40: "Pipeline Frontend NodeJS 1"

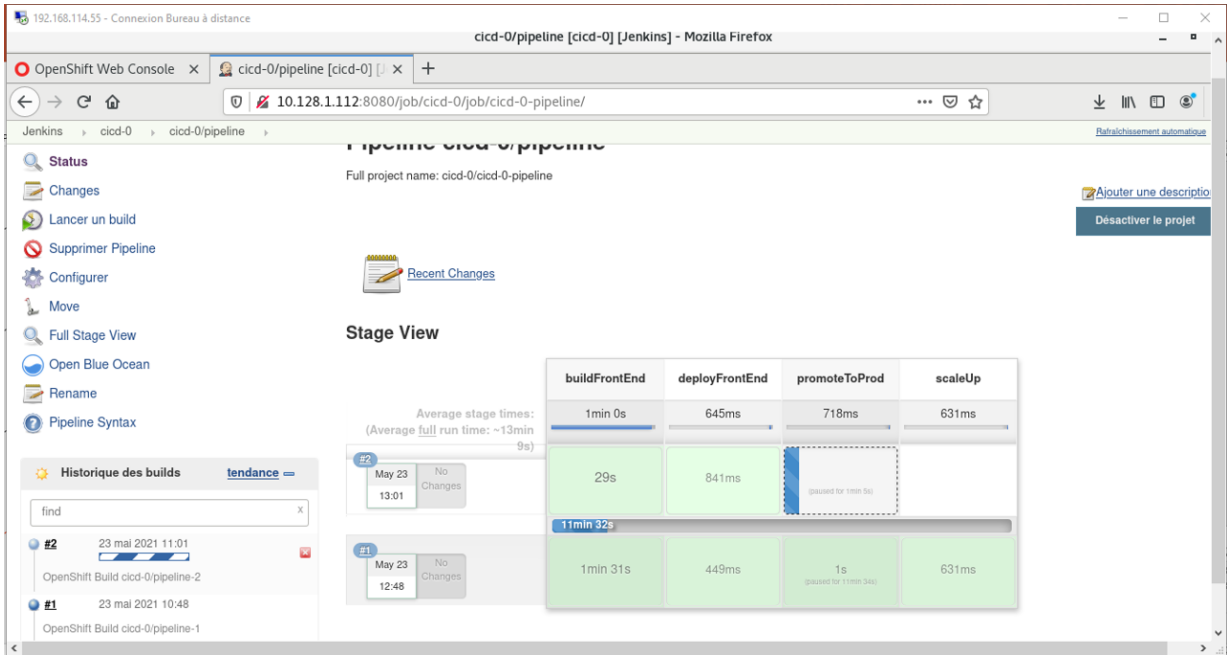


Figure 4.41: "Pipeline Frontend NodeJS 2"

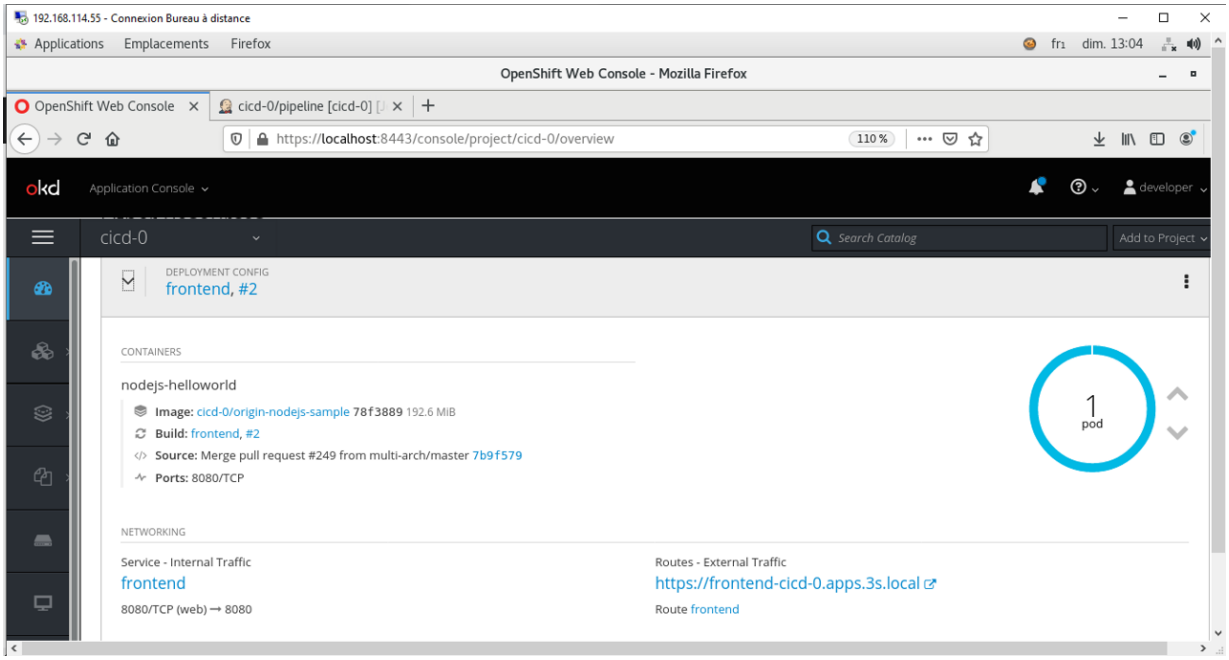


Figure 4.42: "Frontend NodeJS Pod"

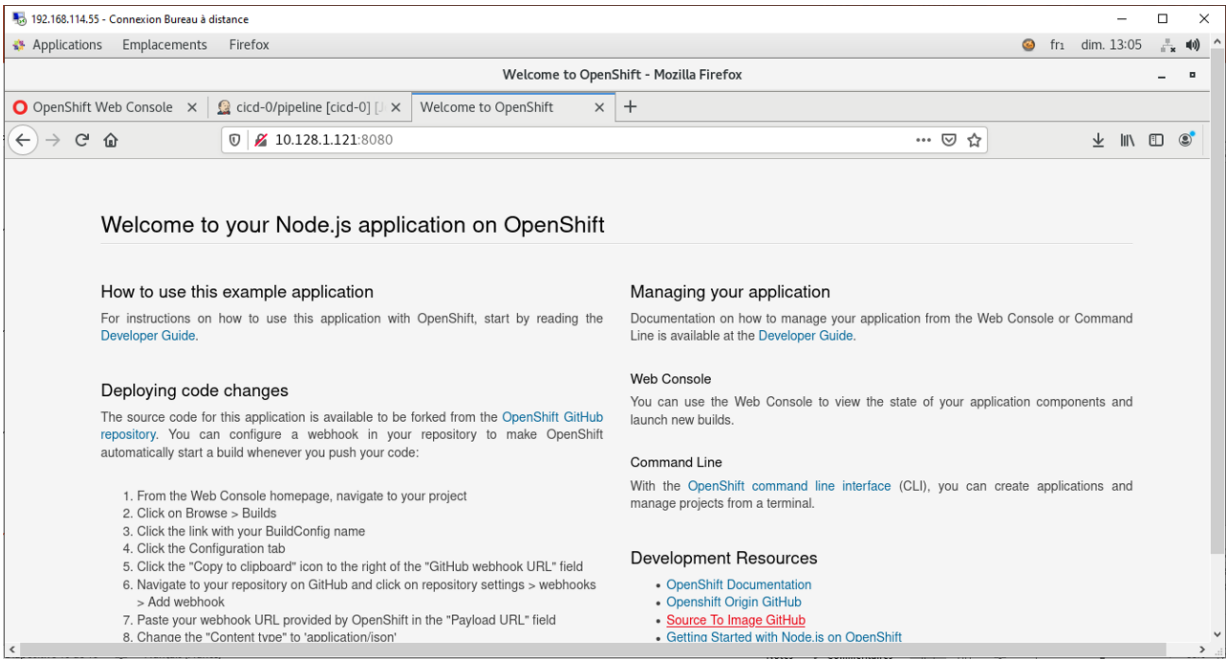


Figure 4.43: "Frontend NodeJS Web Interface"

Partie Backend NodeJS + MongoDB Pipeline :

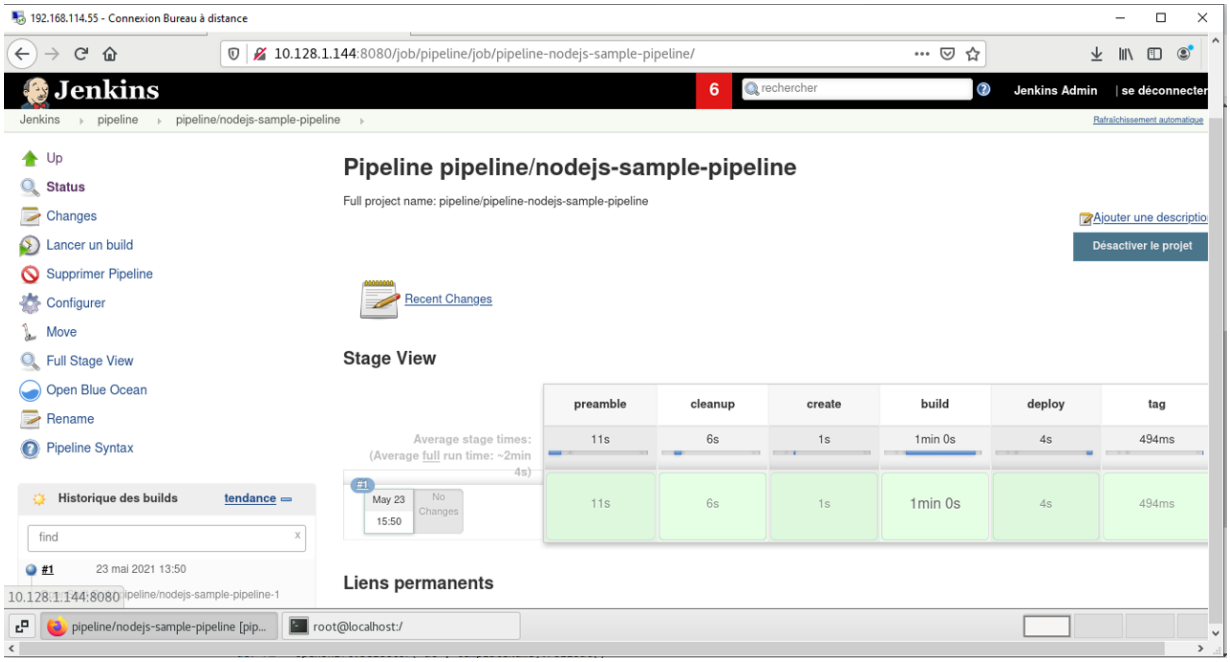


Figure 4.44: "Pipeline Backend NodeJS + MongoDB 1"

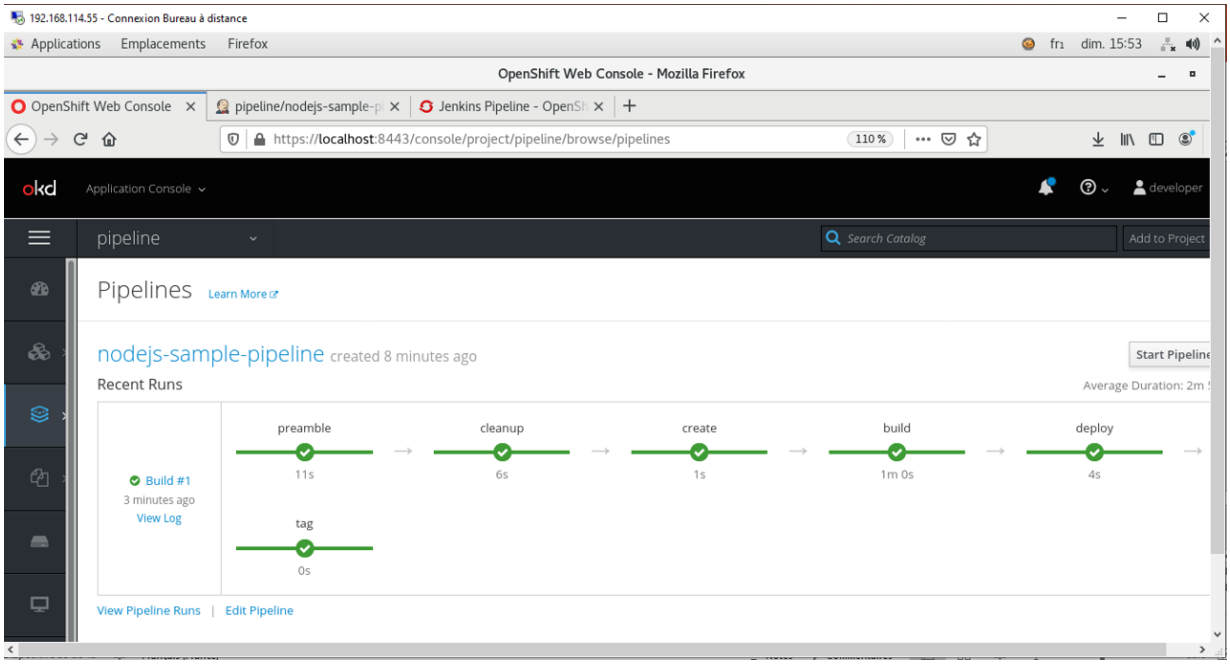


Figure 4.45: "Pipeline Backend NodeJS + MongoDB 1"

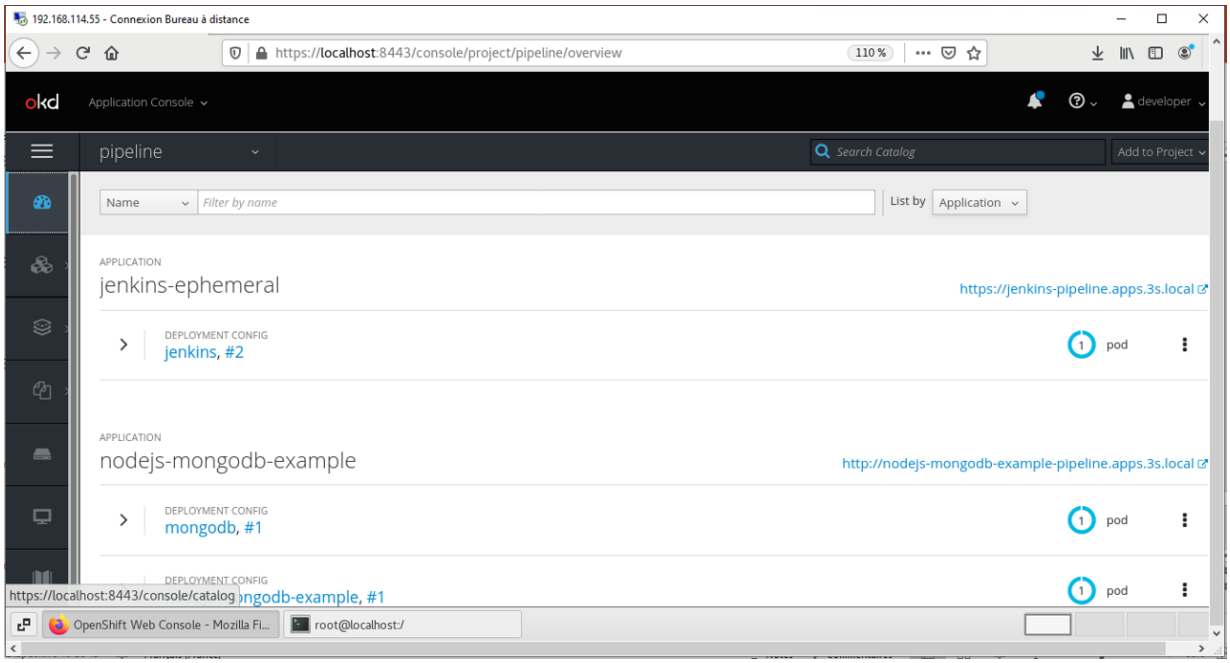


Figure 4.46: "Backend NodeJS + MongoDB Pods"

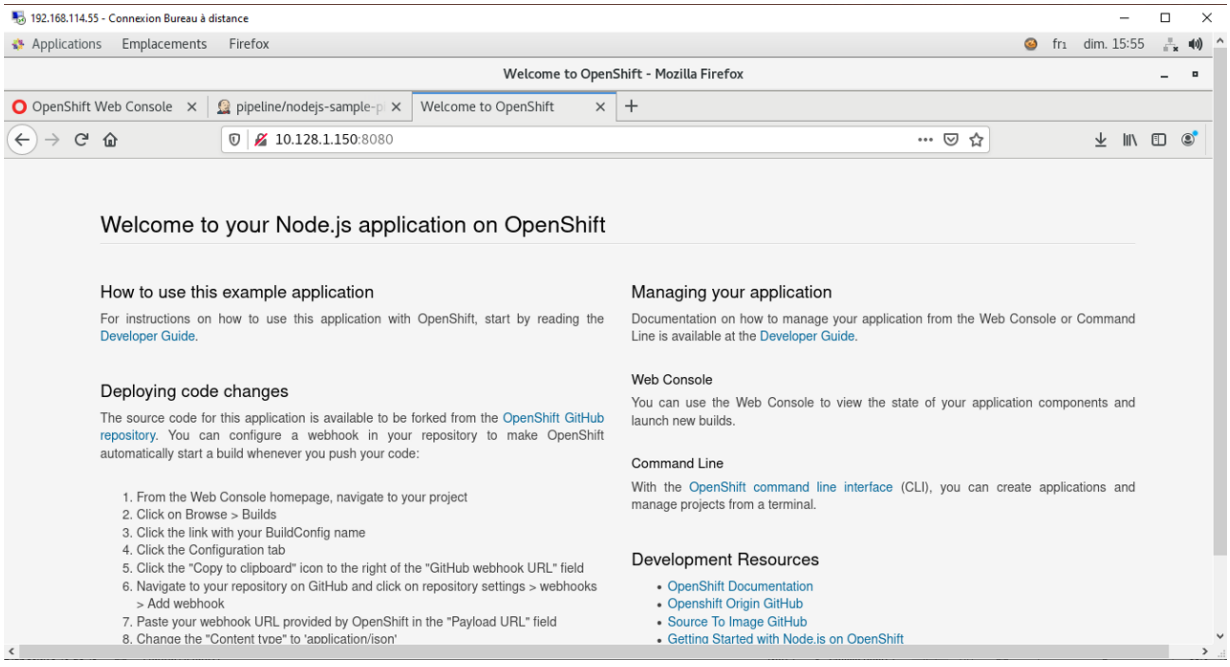


Figure 4.47: "Backend NodeJS Web Interface"